

Velran

a security-first web application language and server with Rust syntax.

Web Application Developer Guide

Zsolt Krüpl

September 2026

Preface

Project identity. The canonical full project name is **Velran — a security-first web application language and server with Rust syntax**. The shorter phrase “Rust Web” may be used descriptively, but it is not a separate project name.

This book is for web developers meeting Velran for the first time. It starts from the application itself—business data, allowed operations, and HTTP boundaries—rather than from syntax.

The learning order is deliberate: establish the mental model, install the toolchain, build one small end-to-end application, then deepen language, data, authentication, files, APIs, security, and production operations. CLI and `server.toml` material stays near the end as reference, not prerequisite.

Edition and verification status

This edition records a **Verified Development Milestone** for the current Velran implementation. On the current source tree, the trusted development environment has completed `cargo fmt`, the full workspace test suite, `./verify.sh`, and local test serving successfully. The language/package/security descriptions in this book are intended to match that verified tree.

This milestone is deliberately narrower than a production-release claim. Environment-specific deployment, recovery, performance evidence outside the repository gate, secret provisioning, TLS/reverse-proxy setup, and operator acceptance remain responsibilities of the target production environment. Documentation records the verified repository baseline; it does not replace production change control or environment-specific evidence.

Core terms

The book uses a few technical terms consistently. It is useful to establish the same meaning for them from the beginning:

- model** a named typed data shape known to the compiler; it does not automatically become a database table or schema;
- typed** the shape and type of the data are known to the compiler instead of being discovered only at runtime;
- capability** an explicit ability to access a resource or perform a sensitive operation, such as `Db`, `Transaction`, or `AppFs`;
- policy** a rule enforced at an external boundary, such as authentication, rate limiting, CORS, or trusted-proxy policy;
- runtime** the server-side environment that executes compiled Velran programs and enforces limits and capability boundaries.

Later chapters refine these definitions, but these meanings are enough for a first reading.

How to read this book

On a first pass, do not try to memorize every configuration key or production limit. The first goal is to be able to answer three questions consistently:

1. *What typed data does this feature operate on?*
2. *What operation reads or modifies that data?*
3. *What authority or capability permits that operation?*

Once those three questions are clear, the route, form, SQL, authentication, and configuration become different boundaries of the same model rather than unrelated tricks.

A useful discipline throughout the book is to separate what Velran proves from what the application designer must still decide:

Velran/platform responsibility	Application/operator responsibility
Typed boundaries, escaping, capability checks, bounded resources, secure session mechanics	Business rules, who may act on which object, retention and recovery objectives
Fail-closed compiler/runtime invariants	Choosing the correct access policy and production topology
Safe SQL, redirect, upload, egress and cryptographic primitives	Deciding when a business operation is allowed and how domain-specific conflicts are resolved

The model removes security plumbing; it does not invent the application's business authorization rules.

Choose a learning path

Do not read every chapter with the same priority. Choose one of these two paths and change paths later if your role changes.

Fast web-developer path. Use this when your immediate goal is to build and ship a conventional authenticated Velran application. Read the chapters in this order:

Stage	Chapters and outcome
Mental model and first vertical slice	1–4: understand boundaries, install/run the toolchain, build one complete request-to-response path, and learn the Rust-like language/HTTP surface.
Persistent application	5–7: forms/CRUD, database invariants, authentication and object authorization.
Choose the web surface you need	8 if you handle files/media; 9 for JSON/API work; 10 for static assets and browser enhancement. These are branches, not mandatory prerequisites for each other.
Make it trustworthy	14–15: testing/error diagnosis, then the consolidated security-hardening model.
Ship and operate	18–20: production deployment, recovery, and the CLI workflow. Finish with Chapter 23's release-readiness checklist.

You may postpone Chapters 11–13 and 16–17 until the application actually needs larger module structure, external integrations, caching/scaling, or deeper observability. Do not postpone Chapter 15 before shipping.

Complete Velran path. Read Chapters 1–23 in order. This path is intended for developers who will also own architecture, security review, platform integration, scaling, observability, or production operations. The full path keeps optional web features and operator material in context instead of treating them as isolated recipes.

How to switch paths. The fast path is not a lesser security mode. It skips breadth, not invariants. If a skipped capability becomes part of your application, read that chapter before implementing it, complete its exercise, and use its canonical companion example. At any point you can continue sequentially on the complete path without relearning earlier material.

Reuse your Rust knowledge

Velran is deliberately not a second general-purpose language that you must learn beside Rust. For ordinary computation, keep the Rust mental model wherever the web-security contract does not require a framework-owned construct. The recurring **Rust** → **Velran** blocks show three things: what transfers unchanged, what the framework adds at a boundary, and what you should avoid relearning as a separate Velran concept.

If you already understand `let`, `let mut`, structs, enums, `Vec<T>`, `Option<T>`, `Result<T,E>`, `match`, method calls, modules, and `?` error propagation, keep using those ideas. Spend your new-learning budget on routes, typed request boundaries, queries, capabilities, authorization, and production policy. Advanced Rust topics such as unsafe code, manual FFI, allocator internals, proc-macro authoring, and low-level async-runtime machinery are not prerequisites for ordinary Velran web development.

Minimum Rust for Velran

You do not need broad Rust mastery before you start building Velran applications. For the fast web-developer path, the useful minimum is a small set of everyday Rust ideas. The goal of this refresher is not to teach Rust in parallel with Velran; it is to tell you when your existing Rust intuition is already enough.

1. Bindings and ordinary values

Be comfortable reading immutable bindings and the occasional mutable local. Prefer immutable data flow unless a local computation genuinely changes state.

```
let title = "Release notes";
let mut retries = 0;
retries = retries + 1;
```

For Velran web work, this is enough to understand most local computation. You do not need to study ownership edge cases before writing your first page or action.

2. The core data shapes: `String`, `Vec`, `Option`, `Result`

Four types carry a large part of ordinary application code:

Rust idea	What it means in Velran application code
<code>String</code>	owned text that can cross normal application boundaries; use the framework's typed request types at HTTP boundaries.
<code>Vec<T></code>	zero or more typed values, for example query results or rendered items.
<code>Option<T></code>	a value may be present or absent; absence remains explicit instead of becoming a magic sentinel.
<code>Result<T,E></code>	an operation may succeed or fail; propagate or handle the failure deliberately.

The `?` operator is especially useful: read it as “return this error now; otherwise continue with the successful value.”

```
let note = queries::loadNote(db, id)?;
let owner = note.ownerUsername;
```

You do not need advanced error-type design to use this pattern effectively in early Velran code.

3. Structs and enums are the domain vocabulary

Use a struct when several named fields belong together. Use an enum when the domain permits one of a known set of states.

```
struct NoteSummary {
    id: i64,
    title: String,
}

enum PublicationState {
    Draft,
    Published,
}
```

This is the right level of Rust knowledge for modeling most web-domain data. The important Velran addition is what happens when those types cross a request, query, authorization, JSON, or persistence boundary.

4. Match, method calls, modules, and ?

You should be able to read a small `match`, call methods with `value.method()`, use qualified module names such as `queries::loadNote`, and follow `?` propagation. Those four skills cover a large fraction of the computation between framework-owned boundaries.

```
let label = match state {
    PublicationState::Draft => "draft",
    PublicationState::Published => "published",
};

let count = title.chars().count();
let note = queries::loadNote(db, id)?;
```

5. Borrowing: learn only the boundary you actually meet

For the first part of this book, it is enough to understand the practical distinction that an owned value such as `String` owns its data, while a borrowed reference such as `&str` temporarily views data owned elsewhere. If the compiler asks you to clarify a borrow in ordinary compute code, fix that local relationship; do not stop the web-learning path to master lifetime theory first.

In framework-owned web surfaces, prefer the documented Velran types and signatures. Do not invent reference-heavy handler APIs merely because Rust permits them.

6. What you can postpone

You can become productive in Velran before learning unsafe Rust, manual lifetime design, custom allocators, pinning, advanced trait machinery, procedural macros, FFI internals, or async-runtime implementation details. These topics can matter when contributing to the Velran compiler/runtime itself, but they are not prerequisites for application development.

Ready-to-start test. You know enough Rust to begin the fast path if you can read the four listings above, explain the difference between `Option` and `Result`, recognize a struct and enum, follow a small `match`, and understand that `?` propagates failure. Everything else can be learned when the book actually needs it.

Velran in 60 minutes

Use this as a first-session vertical slice, not as a substitute for the rest of the book. The goal is to connect the pieces once before studying them separately. Work against the verified Secure Notes baseline at `examples/secure-notes/checkpoints/01-baseline/main.vrn`; inspect the surrounding module files whenever a step mentions code that is not in `main.vrn` itself.

Pass	Focus	What to notice
1	Page and route	Find the page handler and the route that exposes it. Confirm that access is explicit rather than implicit.
2	Typed input and form	Follow one form value from the request boundary into typed application code. Identify its length/resource bound.
3	Database read/write	Follow one query and one mutation. Notice typed bind values, transaction scope, and the separation between SQL structure and user data.
4	Authentication and authorization	Locate the trusted principal, then find the separate ownership/authorization check before a protected read or mutation.
5	Response	Follow the successful path back to HTML or a typed redirect. Relate this to the request lifecycle: parse, validate, authenticate, authorize, effect, response.
6	Check and explain	Run <code>velran-cli check</code> on <code>examples/secure-notes/checkpoints/01-baseline/main.vrn</code> . Then explain which boundary would reject an unsafe variant before trying to add a feature.

The one-hour success criterion. Do not measure success by how much syntax you memorized. You are ready to continue when you can trace one request end to end, identify where trust changes, make one small safe edit, and use the compiler/verifier result to decide what to fix next. If a step is unfamiliar, continue with Chapters 1–7 rather than detouring into advanced Rust.

Ten security invariants to memorize

The book repeats the following rules on purpose. Treat them as a compact mental checklist at every external boundary:

1. every route has an explicit access policy;
2. external input is parsed, normalized, validated, and bounded before it reaches a protected effect;
3. SQL structure is compiler-owned and user data enters only through typed bind parameters;
4. authentication proves who the caller is; authorization proves what that caller may do, and must happen before the protected effect;
5. client file names and MIME declarations are metadata, not trust evidence;
6. CORS is a browser-origin policy, not authentication or authorization;
7. outbound network access exists only through an explicit named/configured capability;
8. a security rule needs negative evidence too: forbidden variants must fail closed;
9. logs and traces must not become a second copy of secrets or request bodies;
10. a production candidate is compiled and validated before activation; failure preserves the known-good generation.

When one of these rules appears again in a chapter, it is shown as a **Security invariant** block. Repetition is intentional: the goal is recall under implementation pressure, not novelty.

Book conventions

- `main.vrn` is the application entry point.
- The trusted TOML file is the primary production configuration surface.
- Secret values do not go on the command line or in plaintext TOML fields; configuration uses `*_file` references.
- SQL text is compiler-owned; user values may enter queries only as typed bind parameters.
- In HTML, `{ ... }` interpolation is escaped; there is no raw-HTML escape hatch.
- Business mutations run inside transactions.

The book targets the V1 surface. If a capability is not part of that surface, it should not be implemented through an ad-hoc bypass; it should become an explicit language/runtime feature.

Edition scope and source of truth

This edition describes the current Rust-first, native-only Velran surface. Ordinary compute syntax intentionally follows Rust wherever the web security model does not require a framework-owned construct: attributed callables use `#[page] fn`, `#[action] fn`, and `#[query] fn`; collections use Rust-like `Vec<T>` and `Option<T>`; mutable compute locals use `let mut` and direct assignment; floating-point code uses `f32` and method syntax such as `x.sin()`. Unsupported native lowering fails closed; there is no bytecode/VM execution fallback for application code.

The runnable files under `examples/` and the focused documents under `docs/` are the copy/paste and feature-level source of truth. Listings in this book are teaching material and may omit surrounding setup, but they must use the same accepted language surface. The repository intentionally separates learner-facing programs from verification-only material: `examples/` contains applications worth reading and adapting, while `tests/fixtures/` contains rejection/security cases that exist to prove fail-closed behavior and `tests/manifests/` contains verification metadata. When a release changes syntax or a security invariant, update those canonical examples, fixtures where relevant, and this English/Hungarian book together.

Progressive exercise curve

The exercises remove guidance in four stages. The later *Exercise mode* labels refer to these definitions:

Guided, Chapters 1–4. Identify the boundary or invariant, make the smallest change, then compare with the verified companion. Done when you can explain the change, its protecting boundary, and one rejected failure.

Supported, Chapters 5–10. Try first without the companion; if blocked, inspect only the interface, policy, or data shape before the implementation. Done when the feature works, its security invariant is explicit, and one negative variant is rejected.

Independent, Chapters 11–17. Treat the task as a requirement, design and implement it yourself, then review against the canonical example. Done when you can defend the design and both success and failure paths.

Operational, Chapters 18–23. State the invariant at risk, collect evidence, choose the smallest safe action, and define rollback or recovery first. Done when another operator could follow your evidence and post-action record.

Optimize for needing less help on the next exercise, not for finishing the current one fastest.

Contents

Preface	ii
I Getting started: a mental model and a working application	1
1 How to think about a Velran application	2
1.1 Concepts first, pages second	2
1.2 What does <code>model</code> mean?	3
1.3 The model is a contract between layers	3
1.4 Business concepts have operations	4
1.5 Capability: access is part of the model too	4
1.6 A route is not the business model	4
1.7 A simple design method	5
1.8 What to carry into the next chapters	5
1.9 Practice: draw the boundary map	5
1.10 Common mistake: starting from routes instead of the model	6
1.11 Running project checkpoint: Secure Notes	6
2 Installation, binaries, and runtime topologies	7
2.1 Build from source	7
2.2 Where to install the binaries	8
2.3 Your first application	9
2.4 At first, learn only the minimum configuration	10
2.5 Production process and reverse proxy	10
2.6 Runtime topologies in detail	11
2.7 Direct single-domain HTTPS	11
2.8 Behind an Apache reverse proxy	12
2.9 Behind an Nginx reverse proxy	13
2.10 Recommended production directory structure	13
2.11 First validation	13
2.12 Practice: verify the deployment boundary	14
2.13 Common mistake: treating configuration as application data	14
2.14 Running project checkpoint: Secure Notes	14
3 A to Z: a minimal web application	15
3.1 The request lifecycle: one mental model for most web work	16
3.2 Create the project directory	16
3.3 The database schema	16
3.4 The model and queries	17
3.5 The GET page	17

3.6	The POST action	18
3.7	Routes and validation	18
3.8	The complete application in one place	19
3.9	SQLite connection for development	19
3.10	Source validation and migration	19
3.11	Minimal development configuration	20
3.12	The first HTTP request	20
3.13	What happens with invalid input?	20
3.14	From development toward production	21
3.15	What did this one example teach us?	21
3.16	Verified companion example	21
3.17	Practice: trace one request end to end	21
3.18	Common mistake: mixing read and mutation flows	22
3.19	Running project checkpoint: Secure Notes	22
II	Core web application development	23
4	Language essentials and HTTP boundaries	24
4.1	Source structure and statement boundaries	24
4.2	Typed computation for web application code	27
4.3	Domain values and handler code	31
4.4	Routing, typed input, and HTML	32
4.5	Practice: classify syntax by owner	34
4.6	Common mistake: inventing convenience syntax	34
4.7	Running project checkpoint: Secure Notes	34
5	Data entry: typed SQL, CRUD, forms, and PRG	35
5.1	Apply the lifecycle to a CRUD mutation	35
5.2	Typed SQL and CRUD	36
5.3	Forms, validation, and PRG	37
5.4	A complete edit cycle: validation, authorization, concurrency	38
5.5	Database invariants and user-friendly validation	40
5.6	Mutation review checklist	40
5.7	Verified companion examples	40
5.8	Practice: design an edit conflict	40
5.9	Common mistake: validating only before the transaction	40
5.10	Running project checkpoint: Secure Notes	41
6	Database deep dive: transactions, migrations, and concurrency	42
6.1	Three separate responsibilities	42
6.2	The query return type is a contract	43
6.3	Bind values; do not build SQL	44
6.4	Read and mutation authority boundaries	45
6.5	Optimistic locking: do not overwrite someone else's work	46
6.6	Design indexes for access patterns	47
6.7	Migration: schema is part of the release	47
6.8	Expand/contract: preserve rollback options	48
6.9	Handling database errors	49
6.10	SQLite, PostgreSQL, or MariaDB?	49
6.11	Backup from a database developer's perspective	49

6.12	A sample CMS database layer	49
6.13	Chapter checklist	50
6.14	Practice: review a query as an interface	50
6.15	Common mistake: using the database only as storage	51
6.16	Running project checkpoint: Secure Notes	51
7	Authentication, sessions, and authorization	52
7.1	Where authentication and authorization sit in the lifecycle	52
7.2	The complete login path	53
7.3	Local auth: a separate identity database	53
7.4	Account lifecycle through the operator CLI	54
7.5	Session invalidation: auth_generation	54
7.6	Session store: memory or Redis	55
7.7	Cookie policy	55
7.8	Logout	55
7.9	LDAP / LDAPS for enterprise identity	55
7.10	TOTP and recovery codes	56
7.11	Route authorization	56
7.12	Using the principal on a page	57
7.13	A route role is not always enough	57
7.14	Login rate limiting	58
7.15	CSRF, Origin, and Fetch Metadata	58
7.16	Authentication audit	58
7.17	Local auth or LDAP?	59
7.18	Rich text in an authenticated CMS	59
7.19	Production checklist	59
7.20	Verified companion examples	59
7.21	Practice: build an authorization matrix	60
7.22	Common mistake: equating authentication with authorization	60
7.23	Running project checkpoint: Secure Notes	60
8	Files, uploads, and the media library	61
8.1	The mental model: three different things	61
8.2	General file upload: Upload	61
8.3	AppFs: the application's writable filesystem	63
8.4	What happens during an upload?	63
8.5	Image upload: prefer the Image type	64
8.6	Byte limits and pixel limits are different	64
8.7	Image references in the database	64
8.8	Safe rendering in HTML	66
8.9	The built-in media endpoint	66
8.10	CMS example: uploading a hero image	66
8.11	Database and AppFs may form one business state	67
8.12	Static assets and uploaded files are different	67
8.13	What not to do	67
8.14	Current media-surface limits	67
8.15	Developer upload checklist	67
8.16	Verified companion examples	68
8.17	Practice: threat-model one upload	68
8.18	Common mistake: putting uploads under the static root	68
8.19	Running project checkpoint: Secure Notes	68

9	JSON API, CORS, and browser client integration	69
9.1	The same lifecycle also applies to APIs	70
9.2	The first typed JSON endpoint	70
9.3	JSON lists from the database	71
9.4	Typed JSON POST body	71
9.5	Content negotiation and 406	72
9.6	API errors	72
9.7	Same-origin: the default and simplest model	73
9.8	CORS: exact-origin allowlist	73
9.9	Preflight	73
9.10	Credentialed CORS with a session cookie	74
9.11	CSRF on a JSON API	74
9.12	Complete browser POST flow	75
9.13	Reverse proxy and origin	75
9.14	What not to do	75
9.15	Which architecture should you choose?	76
9.16	Runnable example	76
9.17	Verified companion examples	76
9.18	Practice: separate API design from CORS design	76
9.19	Common mistake: enabling permissive CORS by default	76
9.20	Running project checkpoint: Secure Notes	76
10	Static assets and frontend integration	77
10.1	Three separate file lifecycles	77
10.2	Configuring the static root	78
10.3	Fingerprints: the key to long caching	78
10.4	A minimal asset build pipeline	79
10.5	ETag and conditional GET	79
10.6	Precompressed Brotli and gzip	79
10.7	MIME, nosniff, and path confinement	80
10.8	CSS, images, and fonts in Velran HTML	80
10.9	Important V1 boundary: JavaScript may be served, but not executed in Velran HTML	80
10.10	Should Velran serve assets, or Apache/Nginx?	81
10.11	Nginx example for direct static serving	81
10.12	Apache example for direct static serving	82
10.13	Deploy application code and assets as one release	82
10.14	Checking with curl	83
10.15	Practical decision rule	83
10.16	Verified companion example	83
10.17	Practice: classify every frontend file	83
10.18	Common mistake: letting frontend convenience erase trust boundaries . . .	83
10.19	Running project checkpoint: Secure Notes	84
11	Organizing larger applications	85
11.1	Three separate questions	85
11.2	mod is not textual include	85
11.3	When should you switch to feature-based structure?	86
11.4	Domain object: a business namespace, not an OOP class	87
11.5	Package boundaries for reusable application code	88
11.6	Dependencies remain explicit	88

11.7	Why should routes remain top-level?	89
11.8	Recommended naming convention	89
11.9	One possible CMS structure	89
11.10	What not to do	90
11.11	Refactoring path: from one file to a larger application	90
11.12	Verified companion examples	91
11.13	Practice: refactor by change locality	91
11.14	Common mistake: modularizing by file count alone	91
11.15	Running project checkpoint: Secure Notes	91
III	Applied development, integration, and assurance	92
12	Practical applications: wiki and CMS	93
12.1	A simple wiki	93
12.2	CMS and news site	94
12.3	Server-side Markdown rendering and cache	96
12.4	From demo CMS to editorial workflow	97
12.5	A practical CMS layering model	97
12.6	CMS review questions	98
12.7	Verified companion examples	98
12.8	Practice: define an editorial state machine	98
12.9	Common mistake: representing workflow as free-form text	98
12.10	Running project checkpoint: Secure Notes	99
13	External services and egress policy	100
13.1	The current Velran boundary	100
13.2	Why arbitrary URLs are intentionally unrepresentable	101
13.3	Named target: configuring the network authority	101
13.4	Policy checks multiple layers	102
13.5	IPv4 and IPv6 use the same authority model	102
13.6	Typed outbound operations	102
13.7	Size, protocol and time limits	102
13.8	Secrets for external services	103
13.9	Retries, idempotency and exceptional outcomes	103
13.10	Webhooks are the inverse boundary	103
13.11	Synchronous request or background work?	103
13.12	Minimum testing	104
13.13	The essence of this chapter	104
13.14	Verified companion example	104
13.15	Practice: write an egress contract	104
13.16	Common mistake: allowing arbitrary outbound destinations	104
13.17	Running project checkpoint: Secure Notes	104
14	Testing, error models, and developer diagnostics	105
14.1	Read a security diagnostic in four passes	105
14.2	The first gate: <code>velran-cli check</code>	106
14.3	A negative test is as valuable as a positive test	106
14.4	Four different worlds of failure	107
14.5	Test configuration separately	107
14.6	Migration: status, verify, apply, verify	107

14.7	Business tests: prove state, not implementation	108
14.8	Database tests: SQLite does not prove everything	108
14.9	Authentication and authorization test matrix	108
14.10	File, media, API, and cache tests	109
14.11	An HTTP error code is not the same thing as a program bug	109
14.12	Test exhaustive exceptional states	110
14.13	Debugging with request IDs	111
14.14	Deployment smoke test	111
14.15	What does the repository <code>verify.sh</code> mean?	111
14.16	Recommended development cycle	111
14.17	Debugging decision order	112
14.18	Minimum application release checklist	112
14.19	Practice: derive negative tests from boundaries	113
14.20	Common mistake: testing only successful handlers	113
14.21	Running project checkpoint: Secure Notes	113
15	Security hardening: the current secure-by-construction model	114
15.1	What the secure model does – and does not – decide	114
15.2	Authority and access control	114
15.3	Critical operations, idempotency and transaction outcome	115
15.4	Input, injection and browser boundaries	115
15.5	Trust, domain types and disclosure	115
15.6	Authentication and session hardening	116
15.7	Credentials, tokens and cryptography	116
15.8	Verified webhooks and safe uploads	116
15.9	Resource safety is a security invariant	117
15.10	Production configuration is a security contract	117
15.11	Supply-chain state is part of the release	117
15.12	Security events, redaction and alerts	117
15.13	Production security gate	118
15.14	Review questions for every new feature	118
15.15	Verified companion examples	118
15.16	Practice: map controls to threat paths	118
15.17	Common mistake: treating hardening as a final deployment phase	119
15.18	Running project checkpoint: Secure Notes	119
IV	Production and operations	120
16	Cache, Redis, rate limiting, and multi-instance operation	121
16.1	Why not a general Redis client?	121
16.2	Production Redis configuration	121
16.3	Shared sessions across server instances	122
16.4	Public page cache	122
16.5	What may go into public cache?	123
16.6	Operator-controlled cache boundaries	123
16.7	Cache invalidation: generations, not key search	124
16.8	Stampede protection and its limits	124
16.9	Redis failure during cached requests	124
16.10	Route rate limiting	124
16.11	Rate-limit scopes	125

16.12	Fixed-window model across instances	125
16.13	Memory backends are development exceptions	126
16.14	A typical multi-instance topology	126
16.15	Rolling restart and cache generation	126
16.16	What should you monitor?	127
16.17	Redis in recovery	127
16.18	Practical example: cached news front page with publishing	127
16.19	Checklist	128
16.20	Practice: classify cacheable data	128
16.21	Common mistake: caching before defining consistency	128
16.22	Running project checkpoint: Secure Notes	128
17	Logging, audit, metrics, and troubleshooting	129
17.1	The role of the three log classes	129
17.2	Request ID: following one request end to end	130
17.3	What may and may not be logged?	130
17.4	Security audit and HTTP denials	131
17.5	Business audit: durable business history	131
17.6	Platform security events, redaction, and burst alerts	132
17.7	Prometheus metrics on a separate management socket	132
17.8	Who scrapes metrics behind Apache or Nginx?	133
17.9	Health: liveness and readiness are different	133
17.10	Logrotate and SIGHUP	134
17.11	What if the log file is not writable?	135
17.12	Practical production troubleshooting workflow	135
17.13	Minimum monitoring baseline	135
17.14	Retention, SIEM, and audit protection	136
17.15	Checklist	136
17.16	Practice: design an evidence trail	136
17.17	Common mistake: logging data instead of events	136
17.18	Running project checkpoint: Secure Notes	137
18	Production deployment and service lifecycle	138
18.1	The application production contract	138
18.2	Release path	139
18.3	systemd service	139
18.4	Controlled restart and readiness	139
18.5	Migration and rollback are not the same operation	140
18.6	Recovery gate before deployment	140
18.7	Minimum post-deploy checks	140
18.8	Release invariants	140
18.9	What can reload and what requires restart	141
18.10	Candidate activation: the operator rule	141
18.11	Pre-deploy evidence	142
18.12	Deployment anti-patterns	142
18.13	Practice: rehearse an activation failure	142
18.14	Common mistake: equating build success with deployability	142
18.15	Running project checkpoint: Secure Notes	142
19	Backup, restore, and disaster recovery	144
19.1	Start with a state inventory	144

19.2	What is not business source of truth?	144
19.3	RPO and RTO are business decisions before they are technical settings . .	145
19.4	Safe V1 baseline: a controlled maintenance backup	145
19.5	Application DB: use the backend’s native backup tool	145
19.6	The local-auth DB is a security-sensitive backup	146
19.7	AppFs: the entire data root may be durable state	146
19.8	Every recovery point needs a manifest	147
19.9	Restore drill: prove the backup	147
19.10	Make a recovery decision before an upgrade	148
19.11	Expand/contract keeps the fast rollback window open	148
19.12	Rollback and restore are different operations	148
19.13	Forbidden shortcuts	149
19.14	Production recovery checklist	149
19.15	Practice: prove a restore, not a backup	149
19.16	Common mistake: backing up components independently without a recovery model	150
19.17	Running project checkpoint: Secure Notes	150
V	Reference and operator quick guide	151
20	The Velran CLI and the daily developer/operator workflow	152
20.1	Which tool for which task?	152
20.2	The CLI is not an alternative server	153
20.3	The first command: <code>velran-cli check</code>	153
20.4	Running from the source tree versus using the installed CLI	153
20.5	Migration workflow	154
20.6	Local-auth administration	154
20.7	TOTP administration is especially sensitive	155
20.8	A secret file is also an input contract	156
20.9	Server preflight follows CLI checks	156
20.10	Daily development cycle	156
20.11	Application release workflow	157
20.12	The Velran platform release workflow is different	157
20.13	Automation: separate read-only and mutating steps	157
20.14	Common failures	158
20.15	Operator memory line	158
20.16	Practice: make the safe path the shortest path	158
20.17	Common mistake: relying on remembered command sequences	159
20.18	Operator checkpoint: Secure Notes	159
21	The <code>server.toml</code> configuration reference	160
21.1	Loading, precedence, and fail-closed rules	160
21.2	How to read the tables	161
21.3	[server] and [tls]	161
21.4	[database], [redis], and [auth]	162
21.5	[web]	163
21.6	[storage] and [static_assets]	163
21.7	[lifecycle], [observability], and [logging]	164
21.8	[rate_limit] and [cache]	164
21.9	[native]	165

21.10	[reload]	165
21.11	[limits]	166
21.12	[cgroup]	167
21.13	Production baseline	167
21.14	Automatic source reload	168
21.15	Change workflow	168
21.16	Practice: review configuration by trust domain	168
21.17	Common mistake: copying a large configuration and changing only obvious values	169
21.18	Operator checkpoint: Secure Notes	169
22	Developer reference and where to go next	170
22.1	Canonical documentation map	170
22.2	Daily secure development checklist	171
22.3	How to use the documentation set	172
22.4	Task-oriented reading paths	172
22.5	Executable companion examples	172
22.6	Keeping examples trustworthy	173
22.7	Practice: choose the right authority	174
22.8	Common mistake: resolving documentation conflicts by guessing	174
22.9	Reader checkpoint: Secure Notes	174
A	Appendix and production checklist	175
A.1	Current verification note	175
A.2	Recommended production directory layout	175
A.3	Complete minimal Apache reverse-proxy example	176
A.4	Complete minimal Nginx reverse-proxy example	177
A.5	systemd service example	177
A.6	logrotate example	178
A.7	Pre-release production gate	179
A.8	Quick troubleshooting map	180
A.9	What to carry into production	181
A.10	Velran web developer competency checklist	181
A.11	Practice: run the checklist as evidence collection	183
A.12	Common mistake: treating a checklist as a substitute for engineering judgment	183
A.13	Final Secure Notes checkpoint	183

Part I

Getting started: a mental model and a working application

Chapter 1

How to think about a Velran application

Learning path: Fast path: core. Build the mental model used by every later chapter.

Rust → **Velran**

Reuse from Rust: Structs, enums, `Option`, `Result`, and explicit function inputs remain familiar typed building blocks.

Velran adds: Models make application data shapes compiler-visible, while capabilities make authority to external resources explicit.

Do not relearn: You do not need a new object model. Keep thinking in typed values and explicit inputs; learn where Velran adds security boundaries around them.

Before writing routes or connecting a database, ask: *what business data exists, what shape does it have, what operations may be performed on it, and within what boundaries?* Velran derives many security and correctness guarantees from making those shapes, inputs, queries, and resource accesses explicit contracts.

1.1 Concepts first, pages second

For a product list, start with *Product*, its typed shape, the operations that read or mutate it, the capabilities and policies those operations require, and only then the HTTP surface. The direction of thought is approximately:

```
business concept
  -> typed data shape
    -> operations
      -> capabilities and policies
        -> HTTP route
          -> HTML / JSON
```

This order helps keep business rules out of ad-hoc routes, templates, and string-built SQL.

1.2 What does `model` mean?

The book uses two closely related ideas:

- **data model or domain model:** the human description of the business concepts and relationships in the application;
- **model declaration:** Velran's concrete data shape known to the compiler.

For example:

```
model Product {
  id: i64
  name: String
  price: i64
}
```

This tells the compiler that a `Product` record has exactly these fields and types. A typed SQL query that returns a product must match this contract, and the same typed value can later be used in HTML or JSON output.

A `model` is not an automatic database schema

The declaration above does not create a `products` table. The persistent database schema is created by a migration, for example:

```
CREATE TABLE products (
  id INTEGER PRIMARY KEY,
  name TEXT NOT NULL,
  price INTEGER NOT NULL
);
```

The layers are related but separate: migrations own persistent schema, `model` owns the application's typed data contract, and queries map rows to models explicitly. A model therefore does not automatically create a database entity or same-named table.

A `model` is not a mutable object

`Product` is not a traditional object-oriented class. It has no hidden `save()` method, implicit database connection, or mutable object lifecycle. To change data, the program uses an explicit operation and an explicit resource.

At this stage, the signatures are the important part: `listProducts` receives `Db` read access, while `createProduct` receives an explicit `Transaction` for mutation. Chapter 3 shows the complete SQL bodies once the full request path is in view.

1.3 The `model` is a contract between layers

Think of a `model` as a typed contract that moves between layers.

```
Database row
| typed query
v
Product model
|-----|
v                v
```

HTML page	JSON API
v	
user	

If a query promises `Product`, its returned columns must match that shape. The same typed-record contract later carries into object authorization and JSON output.

1.4 Business concepts have operations

Business data becomes useful through operations such as `Product.list`, `Product.create`, or `Article.publish`. Small programs may use top-level functions; larger programs can group operations in the `object` namespace introduced later. A useful distinction is:

Element	Question it answers
<code>model</code>	What shape does <code>Product</code> data have?
<code>#[query] fn</code>	How do we read or mutate persistent data?
<code>object</code>	Which business concept owns these operations?
<code>#[page] fn/#[action] fn</code>	How does the business operation connect to a web handler?
<code>route</code>	On what HTTP path, with what input and policy, is it reachable?

1.5 Capability: access is part of the model too

Velran makes not only data types explicit but also the resources an operation may access. A function does not obtain the database, filesystem, or current user from an invisible global source.

```
#[page]
fn home(ctx: PageContext, db: Db)
  -> Result<Html, PageError> {
  let products = listProducts(db)?;
  return Ok(html {
    <ul>
      @for product in products {
        <li>{{ product.name }}</li>
      }
    </ul>
  });
}
```

The `db: Db` capability is visible in the signature. File handling uses an `AppFs` capability, mutating database work uses a `Transaction`, and authenticated operations depend on `auth` policy and the corresponding typed context.

Design rule: if an operation needs a sensitive resource, that requirement should be visible in code and policy.

1.6 A route is not the business model

A route matters, but it is the external boundary of the system. For example:

```
route create POST "/products"  
  form name<String> price<i64>  
  validate name length 1 100 price range 0 100000000  
  public => create;
```

The route defines the external HTTP boundary: method, path, typed input, validation, and policy. The `Product` itself remains reusable from HTML, JSON, admin, or other routes.

For larger applications, the following boundary is especially useful:

domain/model	: what is the data and what operations make sense on it?
HTTP route	: who may call it, how, and under what policy?
runtime config	: within what infrastructure and resource limits may it run?

1.7 A simple design method

When starting a new feature, these five questions are a good default:

1. **What is the business noun?** – Product, Article, Invoice, WikiPage.
2. **What typed data is actually needed?** – do not automatically mirror the full database row.
3. **Which operations read, and which mutate?**
4. **Which capabilities do they need?** – Db, Transaction, AppFs, auth, and named network-egress integrations.
5. **Through what HTTP/policy surface do we expose them?** – route, input, validation, auth, cache, rate limit.

For CRUD this takes minutes; in larger systems it prevents routes, SQL fragments, and authorization rules from drifting apart.

1.8 What to carry into the next chapters

Carry one rule forward: keep typed data, queries, transactions, capabilities, and HTTP/security policy explicit, without inventing an elaborate object hierarchy. The next chapters install the system and apply this model to a complete `Product` request flow.

1.9 Practice: draw the boundary map

Exercise mode: guided. Use the guided method defined in the preface.

Choose a small feature you already know, such as notes, tasks, or inventory items. Before writing code, list its trusted state, request inputs, persistent records, mutation points, and externally visible responses. Then mark which transitions require validation, authorization, a transaction, or an explicit capability.

The goal is to expose hidden assumptions before they become code.

1.10 Common mistake: starting from routes instead of the model

A route-first design often produces handlers that each rediscover the same business rules. If several routes repeatedly parse the same state, perform the same ownership check, or reconstruct the same invariants, move the thinking back to typed models, queries, and explicit mutation boundaries.

1.11 Running project checkpoint: Secure Notes

Use the compiler-checked *Secure Notes* application in `examples/secure-notes/checkpoints/01-baseline/main.vrn` as the running project. Its baseline already enforces authenticated, principal-scoped access, bounded input, CSRF-protected forms, ownership checks, and transactional mutations; later checkpoints evolve it without weakening those invariants.

Chapter 2

Installation, binaries, and runtime topologies

Learning path: **Fast path:** **core.** Get to a repeatable local check/build/run workflow; defer backend internals on first reading.

2.1 Build from source

The release source package is a Rust workspace. The normal release build that produces the two installable programs is:

```
cargo build --locked --release
```

The repository's own full self-check is `./verify.sh`; run it when you validate or release the Velran source tree itself. It is not the first step required merely to use an application.

`-locked` ensures that the build uses the dependency graph captured in the release's `Cargo.lock`. If you only need the two executable programs:

```
cargo build --locked --release -p velran-server -p velran-cli
```

After the build, the two important programs are:

```
target/release/velran-server
target/release/velran-cli
```

- **velran-server:** the long-running service process that compiles Velran applications and serves HTTP(S);
- **velran-cli:** an operator/administrative program for tasks such as authentication administration and migrations.

Their roles are intentionally different. A reverse proxy connects only to the **velran-server** process. **velran-cli** is not a web server and does not need a public or proxied listener.

Runtime requirement: rustc is part of the Velran runtime toolchain

Every host that runs Velran applications must also have a usable **rustc**. Velran does not ship an interpreter or VM fallback: its application backend lowers verified Velran code to safe Rust

and invokes `rustc` to produce the immutable native `cdylib` that is activated by the server. Installing only `velran-server` without an available Rust compiler is therefore insufficient for application activation or source reload.

By default the server discovers `rustc` from the active toolchain. Production deployments may instead configure an explicit absolute compiler path in the `[native]` section of `server.toml`. Cargo is needed when building Velran itself from source; the application execution backend invokes `rustc` directly.

What you need to know about application activation

For normal web development, remember only this rule: Velran verifies and compiles a complete candidate before it can replace the currently active application. If the candidate fails, the last known-good version keeps serving requests. There is no interpreter or VM fallback that silently runs unsupported application code.

You do not need the native compiler pipeline to build your first application. The implementation details are intentionally deferred to the “Under the hood” note below and to the production chapter.

During development, these may be sufficient:

```
cargo check --workspace
cargo test --workspace
```

2.2 Where to install the binaries

On a production host, do not run permanently from Cargo’s `target/`. For manual/source installs this book uses `/usr/local/bin` for executables and `/usr/local/etc/velran` for machine-local configuration, keeping them separate from distribution-managed files.

A simple, auditable layout is:

```
/usr/local/bin/velran-server
/usr/local/bin/velran-cli
/usr/local/etc/velran/server.toml
/srv/velran/current/
/srv/velran/data/
/run/secrets/velran/
/var/log/velran/
```

For example:

```
sudo install -d -m 0755 /usr/local/bin
sudo install -d -m 0755 /usr/local/etc/velran
sudo install -m 0755 target/release/velran-server /usr/local/bin/velran-server
sudo install -m 0755 target/release/velran-cli /usr/local/bin/velran-cli
sudo install -m 0644 config/server.toml.sample \
  /usr/local/etc/velran/server.toml
```

`/usr/local/bin` is normally already part of the system `PATH`, so no additional symlink is necessary. The examples in this book use the actual binary names, `velran-server` and `velran-cli`.

Debian package installation is intentionally different

The layout above is for manual/source installation. If Velran is installed through a Debian package, the files are owned by `dpkg`, so Debian filesystem conventions apply instead of the local-administrator `/usr/local` prefix:

```
/usr/bin/velran-server
/usr/bin/velran-cli
/etc/velran/server.toml
/usr/lib/systemd/system/velran.service
```

Build the package on a Debian/Ubuntu build host with:

```
make deb
sudo dpkg -i dist/velran_1.0.0-1_amd64.deb
```

The package deliberately does not start the service automatically: deploy the application and secrets, configure and validate `/etc/velran/server.toml`, then enable `velran.service`. Package-managed files use `/usr` and `/etc`; manual installs use `/usr/local`. See `docs/36-debian-package.md` for the full workflow.

2.3 Your first application

A minimal recommended project is:

```
myapp/
|-- main.vrn
|-- pages.vrn
`-- public/
```

`main.vrn`:

```
mod pages;
```

`pages.vrn`:

```
#[page]
fn home(ctx: PageContext) -> Result<Html, PageError> {
    return Ok(html {
        <main><h1>Hello Velran</h1></main>
    });
}

route home GET "/" public => home;
```

Start it directly from the workspace during development:

```
cargo run -p velran-server -- \
  --app /abs/path/myapp/main.vrn \
  --listen 127.0.0.1:8080 \
  --insecure-dev-cookies
```

The same command using the installed release binary is:

```
/usr/local/bin/velran-server \
--app /abs/path/myapp/main.vrn \
--listen 127.0.0.1:8080 \
--insecure-dev-cookies
```

`-insecure-dev-cookies` is for local development only.

2.4 At first, learn only the minimum configuration

At this point in the book the goal is not to learn the entire `server.toml`. As a beginner, it is enough to distinguish three things:

- the *application code* defines the models, queries, pages, and routes that exist;
- the *server configuration* defines where the process listens, which external resources it may access, and what operator-controlled limits apply;
- *secret files* hold sensitive values that must not live in application source or plaintext TOML.

The A-to-Z chapter provides a minimal development configuration. Production-specific keys are introduced only after there is an application worth operating; the complete key/default/sample reference is near the end of the book. On a first reading, do not try to memorize the full configuration surface.

Under the hood: native application activation

This subsection is optional on a first reading. Internally, activation is deliberately staged so that verification happens before new code can become live:

```
.vrn source
-> parser + security/type verification
-> verified executable intermediate representation (IR)
-> generated safe Rust
-> rustc
-> immutable cdylib generation
-> atomic activation
```

Generated application Rust forbids unsafe code. Unsupported constructs fail closed instead of taking an interpreter/VM path. During source reload, a failed candidate therefore leaves the last known-good generation active. The native compiler cache avoids rebuilding an unchanged verified source/toolchain/policy identity.

2.5 Production process and reverse proxy

Behind Apache or Nginx, the typical request path is:

```
Internet
|
| HTTPS :443
v
Apache or Nginx
|
| HTTP 127.0.0.1:8080
v
```

```
/usr/local/bin/velran-server
|
+-- /usr/local/etc/velran/server.toml
+-- /srv/velran/current/main.vrn
+-- DB / Redis / AppFs
```

The proxy neither starts nor replaces the Velran process. Run **velran-server** as a separate service, for example under systemd, and let the proxy forward requests to its loopback listener. **velran-cli** runs only for operator commands.

A simple systemd **ExecStart** fragment is:

```
ExecStart=/usr/local/bin/velran-server \
  --config /usr/local/etc/velran/server.toml
```

The process user should not be **root**; grant access only to the required application, secret, data, and log files. If TLS is terminated by Apache or Nginx, keep the Velran listener on loopback, for example **127.0.0.1:8080**.

2.6 Runtime topologies in detail

The binary locations and process roles stay the same whether Velran terminates TLS itself or runs behind Apache or Nginx. The difference is the listener, TLS termination, and trusted-proxy boundary configuration.

2.7 Direct single-domain HTTPS

When Velran terminates TLS itself, a public listener may use port 443:

```
[server]
app = "/srv/velran/current/main.vrn"
listen = "0.0.0.0:443"
insecure_dev_cookies = false

[tls]
cert_file = "/run/secrets/velran/tls-fullchain.pem"
key_file = "/run/secrets/velran/tls-key.pem"
handshake_timeout_ms = 5000
http_redirect_listen = "0.0.0.0:80"
public_host = "www.example.com"
```

public_host comes from trusted configuration. Only GET/HEAD requests may be redirected from HTTP to HTTPS.

Because this direct configuration binds ports 80/443, an unprivileged systemd service needs the narrow **CAP_NET_BIND_SERVICE** capability. Do not run the service as root merely to bind these ports. Behind a reverse proxy, with a listener such as **127.0.0.1:8080**, that capability is unnecessary.

2.8 Behind an Apache reverse proxy

In this topology Apache handles public TLS and Velran listens only on loopback. Example Velran configuration:

```
[server]
app = "/srv/velran/current/main.vrn"
listen = "127.0.0.1:8080"
insecure_dev_cookies = false

[tls]
public_host = "www.example.com"

[web]
trusted_proxy_cidrs = ["127.0.0.1/32", "::1/128"]
allow_missing_origin = false
cors_origins = []
cors_allow_credentials = false
```

In reverse-proxy mode there is no Velran-side certificate/key, but `tls.public_host` is still required. It is the trusted source for the canonical host visible to the browser and anchors Host and Origin/Referer validation. Production plain-HTTP upstream is allowed only on a loopback listener with an explicit trusted-proxy list and such a public host. Every normal proxied request must carry trusted metadata such as `X-Forwarded-Proto: https` or the equivalent standard `Forwarded: proto=https` from a trusted proxy.

A minimal Apache VirtualHost is:

```
<VirtualHost *:443>
    ServerName www.example.com

    SSLEngine on
    SSLCertificateFile /etc/letsencrypt/live/www.example.com/fullchain.pem
    SSLCertificateKeyFile /etc/letsencrypt/live/www.example.com/privkey.pem

    ProxyPreserveHost On
    ProxyPass / http://127.0.0.1:8080/
    ProxyPassReverse / http://127.0.0.1:8080/

    RequestHeader set X-Forwarded-Proto "https"
</VirtualHost>
```

This typically requires the `proxy`, `proxy_http`, `headers`, and `ssl` modules. Velran accepts forwarding headers only from ranges listed in `trusted_proxy_cidrs`; do not configure an overly broad range such as the entire Internet.

The port-80 VirtualHost may exist solely to redirect to HTTPS:

```
<VirtualHost *:80>
    ServerName www.example.com
    Redirect permanent / https://www.example.com/
</VirtualHost>
```

2.9 Behind an Nginx reverse proxy

The Nginx topology uses the same trust model: Nginx handles public TLS and Velran listens on loopback. The Velran-side `server.toml` may be the same as in the Apache example.

A minimal Nginx server block is:

```
server {
    listen 443 ssl;
    server_name www.example.com;

    ssl_certificate      /etc/letsencrypt/live/www.example.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/www.example.com/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto https;
        proxy_set_header X-Forwarded-For $remote_addr;
    }
}
```

This example has a single direct reverse proxy, so `X-Forwarded-For` is the client address as seen by the proxy and `X-Forwarded-Proto` is `https`. Do not let arbitrary forwarding headers arriving from the Internet become authority. Velran only trusts forwarding headers received from the exact `trusted_proxy_cidrs` ranges.

The HTTP listener may exist only to redirect to HTTPS:

```
server {
    listen 80;
    server_name www.example.com;
    return 301 https://$host$request_uri;
}
```

2.10 Recommended production directory structure

```
/usr/local/bin/velran-server
/usr/local/bin/velran-cli
/srv/velran/releases/2026-09-04.1/
/srv/velran/current -> /srv/velran/releases/2026-09-04.1
/srv/velran/data/
/usr/local/etc/velran/server.toml
/run/secrets/velran/
/var/log/velran/
```

The binary, release tree, and configuration should be runtime read-only. The application should write only to declared data/log locations.

2.11 First validation

Before sending traffic to the server:

```
/usr/local/bin/velran-server \  
--config /usr/local/etc/velran/server.toml \  
--check-config
```

This validates configuration, secret-file references, application compilation, and TLS files, among other checks, without opening a listener.

2.12 Practice: verify the deployment boundary

Exercise mode: guided. Use the guided method defined in the preface.

On a test machine, identify the binary, immutable release tree, configuration, secret files, application data, and logs. Confirm that only the intended data and log locations are writable by the service account. Run `-check-config` before opening a listener.

2.13 Common mistake: treating configuration as application data

Configuration and secrets should not drift together with mutable application state. Keeping the release tree and configuration read-only makes rollback, review, and incident response much easier to reason about.

2.14 Running project checkpoint: Secure Notes

For Secure Notes, decide where the application source/release artifact lives, where its database or persistent data lives, and which secrets are supplied by files rather than embedded in source. Record those paths before implementing features.

Chapter 3

A to Z: a minimal web application

Learning path: **Fast path:** **core.** Complete the first end-to-end vertical slice.

Rust → **Velran**

Reuse from Rust: Functions, `Result`, `?` propagation, local bindings, and ordinary control flow work with the same intent as in Rust.

Velran adds: Callable attributes, routes, typed request fields, HTML construction, contexts, and capabilities connect that computation to HTTP.

Do not relearn: Do not invent a separate error-propagation or local-variable style for Velran; concentrate on the web boundary around the Rust-like core.

Security invariant: **explicit route policy**

A route is never implicitly public. Every HTTP entry point declares the authority required to reach it; missing policy is an error, not a default.

Compiler diagnostic: **SEC-A01-001**

Meaning: A route has no explicit access policy. Velran refuses to guess whether the endpoint should be public or authenticated.

Fix: Declare the intended policy on the route, for example `public` or `auth user`; choose from the application contract, not merely to silence the diagnostic.

Now build one complete but intentionally small application before studying each language feature separately. It uses only minimal development configuration and shows the path from source through database and HTTP to deployment; it is not a finished webshop.

The example is a miniature product list. The home page lists products from the database and contains a form for adding a new product. In one small program we therefore see a model, typed SQL, GET page, POST action, form validation, CSRF protection, transaction, migration, configuration, and deployment behind a reverse proxy.

The finished example is available in the repository at:

```
examples/a-z-products/  
  main.vrn  
  migrations/0001_create_products.sql  
  server.toml.example  
  README.md
```

3.1 The request lifecycle: one mental model for most web work

Most application code in this book can be reviewed with the same seven-stage model:

Stage	Question to ask
Request	Which route and HTTP method received the request?
Parse	Which declared query, form, JSON, upload, or path values become typed inputs?
Validate	Which bounds, refinements, or cross-field rules must hold?
Authenticate	Who is the caller, and is a session or machine identity required?
Authorize	May this caller perform this operation on this specific object or tenant?
Query/transaction	Which read or state-changing effect is allowed after the previous gates passed?
Response	Which typed HTML, JSON, redirect, status, or error leaves the application?

The stages are a reasoning order, not a promise that every request executes seven runtime functions. A public GET may not need authentication; a read-only page may not open a transaction. The discipline is to ask every question explicitly and skip a stage only because the contract says it is unnecessary.

For state-changing requests, preserve the security order: parse and validate before protected work, authenticate before identity-dependent decisions, authorize before the protected effect, then mutate under the intended transaction or critical-operation contract. Construct the response only from data that survived those boundaries.

When debugging, walk the same chain from left to right. Ask where the observed behavior first differs from the declared contract instead of treating “the handler” as one opaque step.

3.2 Create the project directory

Create a separate application directory for development:

```
mkdir -p my-products/migrations
cd my-products
```

For this example all Velran code stays in a single `main.vrn`. Larger applications will later be split into mod files.

3.3 The database schema

For SQLite, `migrations/0001_create_products.sql` contains:

```
CREATE TABLE products (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  price INTEGER NOT NULL CHECK (price >= 0)
);
```

This migration creates the table used by the application’s typed SQL queries. In production, migration is a separate deployment step; the server does not modify the schema automatically at startup.

3.4 The model and queries

At the top of `main.vrn`, following the model-centered approach from Chapter 1, define the typed business data shape first:

```
model Product {
  id: i64
  name: String
  price: i64
}
```

The list query is:

```
#[query]
fn listProducts(db: Db) -> Result<Vec<Product>, DbError> sql {
  SELECT id, name, price
  FROM products
  ORDER BY id DESC
  LIMIT 50
}
```

`Vec<Product>` states that the query may return zero or more `Product` rows. The selected columns exactly follow the model field names and order.

The INSERT receives a transaction capability:

```
#[query]
fn createProduct(
  tx: Transaction,
  name: String,
  price: i64
) -> Result<(), DbError> sql {
  INSERT INTO products(name, price)
  VALUES (:name, :price)
}
```

`:name` and `:price` are bind parameters. We do not construct SQL from user input with string concatenation.

3.5 The GET page

The home page fetches the products, displays a form, and renders the result list:

```
#[page]
fn home(ctx: PageContext, db: Db) -> Result<Html, PageError> {
  let products = listProducts(db)?;

  return Ok(html {
    <main>
      <h1>Mini product list</h1>

      <form method="post" @action(create)>
        <input type="hidden" name="_csrf" value="{ { csrfToken } }">
        <label>
          Name
```

```

        <input name="name" maxlength="100">
    </label>
    <label>
        Price
        <input name="price">
    </label>
    <button type="submit">Add</button>
</form>

<ul>
    @for product in products {
        <li>
            #{{ product.id }} - {{ product.name }} - {{ product.price }}
        </li>
    }
</ul>
</main>
});
}

```

Dynamic text is HTML-escaped before it enters the response. The form action is not a hand-written URL; it is generated by the `@action(create)` helper. The hidden `_csrf` field protects the POST request against CSRF.

3.6 The POST action

A successful form submission writes to the database in a transaction, then redirects to the home page:

```

#[action]
fn create(
    ctx: ActionContext,
    db: Db,
    name: String,
    price: i64
) -> Result<Redirect, PageError> {
    transaction db {
        createProduct(tx, name, price)?;
    }
    return Ok(redirect(home()));
}

```

The redirect target is the typed GET route helper `home()`, not a string URL. This is the Post/Redirect/Get pattern: if the user refreshes the resulting page, the browser does not automatically repeat the INSERT.

3.7 Routes and validation

Two routes at the end of the file connect HTTP to the handlers:

```

route home GET "/" public => home;

route create POST "/products"
    form name<String> price<i64>

```

```
validate name length 1 100 price range 0 100000000
public => create;
```

The POST route turns both fields into typed input before the handler runs. An empty or over-100-character name, a negative or excessively large price, or a non-integer value never reaches the INSERT as a valid business operation.

The public write is deliberate for this tiny teaching example. In a real public submission endpoint, decide explicitly whether authentication, route rate limiting, abuse monitoring, or a stronger business permission is required. Do not copy `public` merely because the example uses it.

3.8 The complete application in one place

The preceding pieces form the complete application: one `model`, two `#[query] fn` declarations, one `#[page] fn`, one `#[action] fn`, and two `route` declarations. We do not repeat the entire long listing here; the exact single-file version is available directly in the repository:

```
examples/a-z-products/main.vrn
```

This is also a deliberate editorial choice. Later chapters examine the individual language features in depth; here the goal is to follow the entire application path once.

3.9 SQLite connection for development

Do not put the database URL into source code. Create a separate file containing an absolute SQLite path:

```
printf '%s\n' 'sqlite:///tmp/velran-a-z-products.db' > dev-db-url
chmod 600 dev-db-url
```

This file does not contain a real secret in this example, but it uses the same `*_file` pattern that production uses for password-bearing database URLs.

3.10 Source validation and migration

If the release binaries are already installed under `/usr/local/bin`, for example:

```
/usr/local/bin/velran-cli check main.vrn

/usr/local/bin/velran-cli migrate verify \
  --dir migrations \
  --db-url-file "$PWD/dev-db-url"

/usr/local/bin/velran-cli migrate apply \
  --dir migrations \
  --db-url-file "$PWD/dev-db-url"
```

`check` verifies the language and static contracts. The migration CLI is a separate process; the application server is still not running.

3.11 Minimal development configuration

For example, `server.toml` may be:

```
[server]
app = "/ABS/PATH/my-products/main.vrn"
listen = "127.0.0.1:8080"
insecure_dev_cookies = true

[database]
url_file = "/ABS/PATH/my-products/dev-db-url"
```

Replace both `/ABS/PATH` values with real absolute paths. `insecure_dev_cookies` is acceptable only for loopback development; never enable it in production.

Validate the configuration first:

```
/usr/local/bin/velran-server --config "$PWD/server.toml" --check-config
```

Then start the server:

```
/usr/local/bin/velran-server --config "$PWD/server.toml"
```

3.12 The first HTTP request

From another terminal:

```
curl -i http://127.0.0.1:8080/
```

Open the same URL in a browser. After the form is submitted, the flow is:

1. the browser sends POST to the `/products` route;
2. the runtime validates the request schema, typed fields, validation rules, and CSRF token;
3. the `create` action opens a transaction;
4. `createProduct` runs an INSERT with bind parameters;
5. success commits the transaction;
6. the handler redirects to `/`;
7. the new GET renders the row read back from the database.

The same basic chain appears later in the wiki and CMS examples; they simply add more models, queries, routes, authorization, and components around it.

3.13 What happens with invalid input?

Try a negative price or a non-numeric value. The important rule is that browser-side HTML attributes are only a usability aid; the real contract is the server-side route schema and validation. The SQL query receives only already-typed values.

It is also useful to try the following:

- submit a request without the `name` field;
- submit an unknown extra form field;
- modify the migration after applying it, then run `migrate verify` again;

- make the database unavailable and observe readiness and server-log behavior.

3.14 From development toward production

The application code stays the same in a release environment; it does not need to be rewritten merely because the server now runs from installed binaries. After the build and installation described in Chapter 2, the server typically starts as:

```
/usr/local/bin/velran-server \  
--config /usr/local/etc/velran/server.toml
```

The application entry point may be `/srv/velran/current/main.vrn`. With Apache or Nginx, the public TLS connection terminates at the proxy and Velran listens on loopback. The full Apache/Nginx configuration is not repeated here: the installation chapter explains the trust boundary, while the appendix provides near-copy/paste production configurations.

3.15 What did this one example teach us?

A few dozen lines already show the entire basic Velran path:

```
HTTP GET  
-> route  
-> page  
-> typed query  
-> database  
-> escaped HTML  
  
HTTP POST  
-> route schema + validation + CSRF  
-> action  
-> transaction  
-> typed SQL bind  
-> commit  
-> redirect  
-> GET
```

The next chapters decompose this picture. Whenever a language or runtime detail seems overly technical, return to two questions: *where does data enter the system, and through what typed/capability boundaries does it pass before becoming persistent state or an HTTP response?*

3.16 Verified companion example

The complete, compiler-checked companion for this chapter is `examples/a-z-products/main.vrn`. Treat the in-book listings as guided excerpts; when copying a full program, prefer the repository file because `./verify.sh` checks that exact source.

3.17 Practice: trace one request end to end

Exercise mode: guided. Use the guided method defined in the preface.

Take one GET and one POST from the example application and narrate every boundary they cross: route matching, typed input decoding, authorization, query or transaction execution,

response construction, and redirect. If you cannot name a boundary, revisit that part of the example until the data flow is explicit.

3.18 Common mistake: mixing read and mutation flows

A GET page and a POST action may touch the same domain object, but they have different security and lifecycle semantics. Reads should remain side-effect free; mutations should pass through validation, authorization, transactional state change, and Post/Redirect/Get (PRG) where appropriate.

3.19 Running project checkpoint: Secure Notes

Implement the smallest useful vertical slice: list the current user's notes and add one note. Do not add editing, sharing, search, uploads, or caching yet. The objective is to prove the complete request-to-storage-to-response path with the fewest moving parts.

Part II

Core web application development

Chapter 4

Language essentials and HTTP boundaries

Learning path: **Fast path:** **core.** Learn the minimum Rust-like language and HTTP surface needed for daily work.

Rust → **Velran**

Reuse from Rust: Use Rust-like `let`, `let mut`, assignment, `Vec<T>`, `Option<T>`, `String`, scalar types, methods, and `match`.

Velran adds: Velran owns the verified web surface: attributed callables, route/input declarations, capabilities, HTML/JSON boundaries, and the subset that can be lowered safely.

Do not relearn: Ordinary compute syntax is not a competing DSL. When a normal Rust-shaped expression is accepted, prefer it instead of searching for a Velran-specific spelling.

The A-to-Z example already showed the main building blocks. On a first reading, focus on modules, typed values, handlers, routes, optional/list values, and safe HTML; return to the detailed arithmetic/text catalogue when you need it. Working rule: *parse and normalize at the boundary, compute in typed values, persist through explicit queries, and render as late as possible.*

4.1 Source structure and statement boundaries

Modules

`main.vrn` is the entry point of the module graph:

```
mod models;
mod queries;
mod components;
mod pages;
mod actions;
```

`mod` loads an application-root-relative module without injecting its declarations into the global namespace. Across module boundaries, ordinary library items use qualified names such

as `models::Article` and are private by default; explicit `pub` exposes structs, enums, pure functions, inherent methods, or struct fields. Web authority remains route/capability-owned, and `pub mod` re-export semantics are handled separately from item visibility.

Module namespaces and cross-module calls

A module declaration adds one application-root-relative source module to the explicit source graph; it does not copy that module's declarations into the current scope. The canonical mapping is direct:

<code>models.vrn</code>	<code>-> models</code>
<code>catalog/queries.vrn</code>	<code>-> catalog::queries</code>
<code>catalog/pages.vrn</code>	<code>-> catalog::pages</code>

For example, an entry point may load several modules:

```
mod models;
mod catalog::queries;
mod catalog::pages;

route catalogIndex GET "/catalog"
  public => catalog::pages::index;
```

Across a module boundary, use the qualified name. For example:

```
let recent = catalog::queries::recent(db);
// type across a module boundary: models::Article
```

The declaration in `catalog/queries.vrn` is reached through its qualified module name. A type from `models.vrn` is likewise referenced as `models::Article`. Inside a declaration's own module, the short local name remains valid. This keeps local code concise while making dependencies between source modules visible.

Module loading remains explicit and confined to the application/package root: there is no `mod.vrn` fallback, automatic directory discovery, `../`, `./`, or wildcard import. Rust-like namespace roots `crate::`, `self::`, and `super::` are supported inside the verified module graph. A narrow explicit import form, `use path as alias;`, is supported. Package-root module re-export is intentionally narrower than Rust: `pub use path as alias;` may expose only an already-public module namespace; wildcard re-export, item re-export, private-module re-export, and private transitive-dependency re-export fail closed.

Language operations such as `text.trim()`, `x.sin()`, and `values.len()` use Rust-like method syntax where the operation naturally belongs to a value. Framework helpers remain explicit functions only where Velran adds a web-safety contract, such as `splitBounded(...)`. These are not application-module declarations and therefore remain unqualified.

Rust-style structs, inherent methods, and constructors

For ordinary pure/domain computation, prefer the Rust-shaped object model rather than inventing a second object syntax. Public visibility is explicit and private by default:

```
pub struct Summary {
  pub length: i64,
  hidden: i64,
```

```

}

impl Summary {
    pub fn new(length: i64) -> Self {
        return Self { length, hidden: 0 };
    }

    pub fn length_value(&self) -> i64 {
        return self.length;
    }
}

let summary = Summary::new(42);
let n = summary.length_value();

```

Associated functions such as `Type::new(...)` and `Self` in an inherent `impl` use the same verified pure/native lowering path as ordinary functions. Public method signatures are visibility-checked recursively, so a public API cannot leak a private struct through a parameter or return type (including nested result/option forms).

The current safe surface deliberately keeps general mutation narrower than Rust: mutable/owned receivers (for example `&mut self` or consuming `self`), trait implementations, and generic implementations remain fail-closed until the compiler can prove the required ownership/alias semantics without weakening transaction, authorization, tenant-isolation, or idempotency boundaries.

Local packages and explicit dependencies

A workspace can use local packages declared by `velran.toml`. Dependencies are path-pinned and resolved as a bounded, cycle-checked DAG; there is no package registry, Git dependency, build script, or ambient network resolution in the language package layer. A direct dependency is visible through its package namespace, while a transitive dependency stays private to the package that declared it unless an explicitly permitted public module namespace is re-exported.

```

[package]
name = "demo_app"
kind = "app"
entry = "main.vrn"

[dependencies]
textkit = { path = "../libs/textkit" }

```

Library visibility and web authority are separate concepts. `pub` can expose a Rust-like library API; it cannot grant route/page/action/query/integration/filesystem/network authority. Package libraries remain subject to the compiler-owned capability boundary (including the fail-closed library web-authority rule).

Rust syntax versus framework-owned web syntax

A useful rule is to learn one Rust-like compute language and then learn only the web-specific additions Velran must own. The boundary is intentionally visible:

Area	Prefer Rust-like form	Velran-owned boundary
Functions and locals	<code>fn</code> , <code>let</code> , <code>let mut</code> , direct assignment	<code>#[page] fn</code> , <code>#[action] fn</code> , <code>#[query] fn</code>
Values and collections	<code>i64</code> , <code>f32</code> , <code>String</code> , <code>Vec<T></code> , <code>Option<T></code>	domain types, bounded request types and capability types
Control flow	<code>if</code> , <code>while</code> , exhaustive <code>match</code> , <code>?</code>	instruction/allocation budgets and fail-closed verification
Web/data boundary	ordinary typed expressions	<code>route</code> , <code>html</code> , <code>sql</code> , <code>transaction</code> , validation/auth/cache policy

This split is not an invitation to bypass the framework. Rust-like syntax is accepted only when it can preserve the same verified, bounded web semantics through native lowering.

Statement terminators

Velran uses an explicit semicolon as the terminator for simple statements. Newlines are whitespace, not statement boundaries; there is no automatic semicolon insertion. This keeps multiline expressions deterministic and makes generated/audited code unambiguous.

```
let subtotal = price
    * quantity
    + shipping;
let mut retries = 0;
retries = retries + 1;
authorize article owner authorUsername or role Publisher;
return Ok(json(subtotal));
```

Block statements and block declarations are closed by their brace and do not take a trailing semicolon:

```
let mut state = 0;
if ready {
    state = 1;
}
while state < 3 {
    state = state + 1;
}
```

The same rule applies inside a transaction: a standalone mutating query call and a business-audit statement are simple statements and therefore end in `;`. Top-level block declarations such as `model`, `#[page] fn`, `#[action] fn`, `#[query] fn`, and control-flow blocks do not use `;`. Non-block top-level declarations do: both `mod path;` and `route ... => handler;` are terminated explicitly with `;`. A multiline route is complete only at that final terminator.

4.2 Typed computation for web application code

Expressions and arithmetic

Velran expressions use explicit precedence. From tighter to looser: unary `!`; multiplication, division and remainder; addition and subtraction; shifts; comparisons; bitwise AND/XOR/OR;

logical AND; logical OR. Parentheses should be used whenever they make intent clearer.

```
let total = 12 + 3 * 4;
let remainder = 17 % 5;
let shifted = 3 << 2;
let masked = 12 & 10;
let allowed = ready && authorized;
let fallback = cached || fresh;
let denied = !allowed;
```

The operator contract is deliberately typed:

Operands	Operators	Result
i64, same type	+ - * / %	checked i64.
f32, same type	+ - * / %	f32; result must remain finite.
Decimal, same type	+ - * / %	checked Decimal.
i64, same type	« » & ^	shift/bitwise i64.
bool, same type	&&	bool; short-circuit evaluation.
bool	!	logical negation.
String, same type	+	String concatenation, allocation-budgeted.
Numeric, same type	< <= > >=	bool.
Same supported scalar type	== !=	bool.

There is no implicit mixing of i64, f32, and Decimal. Convert an integer explicitly with `value as f32` when floating-point computation is required. Numeric overflow, division/remainder by zero, invalid shifts, or a non-finite f32 result fail closed rather than silently wrapping or propagating NaN/Infinity.

The numeric builtins include:

Function	Type	Purpose
<code>x.sin()</code> , <code>x.cos()</code>	f32 -> f32	Trigonometric functions.
<code>x.sqrt()</code>	f32 -> f32	Square root; invalid domain fails.
<code>x.abs()</code>	i64 -> i64 or f32 -> f32	Absolute value.
<code>x.ln()</code> , <code>x.log10()</code>	f32 -> f32	Natural/base-10 logarithm.
<code>x.log(base)</code>	f32, f32 -> f32	Logarithm with explicit base.
<code>x.exp()</code>	f32 -> f32	Exponential function.
<code>x.powf(y)</code>	f32, f32 -> f32	Floating-point power.
<code>x.round()/x.floor()/x.ceil()</code>	f32 -> f32	Rounding operations.
<code>x as f32</code>	i64 -> f32	Explicit integer-to-f32 conversion.
<code>monotonicNanos()</code>	() -> i64	Process-local monotonic nanosecond counter.

All f32 builtin results must remain finite. Invalid-domain operations fail instead of creating NaN or infinity.

`monotonicNanos()` is for elapsed-time measurement, not wall-clock timestamps; public-cache output that depends on it is rejected.

Practical arithmetic patterns

Web code rarely needs advanced mathematics everywhere, but a few numeric patterns recur constantly. Keep them typed and visible rather than hiding them in string conversions or SQL fragments.

For pagination, compute an offset from validated integers:

```
let pageSize = 25;
let offset = (page - 1) * pageSize;
```

For flags and compact integer masks, bitwise operations are available, but they should represent a real integer-domain contract rather than replace readable booleans. Use `&&` and `||` for business conditions because they short-circuit:

```
let mayPublish = authenticated
    && ownsArticle
    && !locked;
```

Use `Decimal` for money. Use `f32` only when floating-point behavior is part of the problem, such as geometry, signal processing, statistics, or explicit math functions. Never convert monetary values to `f32` merely to reuse a math builtin.

String and text functions

String operations are typed and Unicode-aware, and are lowered into bounded native execution:

```
let cleaned = text.trim();
let left = text.trim_start();
let right = text.trim_end();
let normalized = cleaned.to_lowercase();
let chars = cleaned.chars().count();
let rewritten = normalized.replace(" ", "-");
let part = substring(cleaned, 1, 3);
let first = indexOf(cleaned, "vrn");
let last = lastIndexof(cleaned, "a");
let ch = charAt("Velran", 1);
let repeated = "vrn".repeat(3);
let parts = splitBounded(rewritten, "-", 4096);
```

Function	Signature
<code>text.chars().count()</code>	<code>String -> i64</code>
<code>text.trim()/trim\(_start()/trim_end()</code>	<code>String -> String</code>
<code>text.to_lowercase()/to_uppercase()</code>	<code>String -> String</code>
<code>text.contains(needle)</code>	<code>String, String -> bool</code>
<code>text.starts_with(part) /</code> <code>text.ends_with(part)</code>	<code>String, String -> bool</code>
<code>text.replace(search, replacement)</code>	<code>String, String, String -> String</code>
<code>splitBounded(text, delimiter,</code> <code>maxItems)</code>	<code>String, String, i64 -> Vec<String></code>
<code>substring(text, start[, length])</code>	<code>String, i64[, i64] -> String</code>
<code>indexOf/lastIndexOf(text, needle)</code>	<code>String, String -> i64</code>
<code>charAt(text, index)</code>	<code>String, i64 -> String</code>
<code>text.repeat(count)</code>	<code>String, i64 -> String</code>

`text.chars().count()`, `substring`, `indexOf`, `lastIndexOf`, and `charAt` use Unicode scalar-value positions rather than UTF-8 byte offsets. Missing searches return `-1`; invalid negative/out-of-range indices fail closed. `replace` rejects an empty search string. `splitBounded` rejects an empty delimiter, requires a compile-time item limit in the supported range, and fails rather than silently truncating when that bound would be exceeded. `repeat` is allocation-budgeted before materializing its result. `Vec<String>` remains a typed compute collection with `parts.len()` and checked `parts[index]` access.

For bounded regular-expression work, Velran also provides `regexMatch`, `regexReplace`, and `regexCaptures`. Regex patterns and runtime work remain subject to validation and resource limits.

Practical text-handling patterns

Treat text manipulation as boundary work. Normalize user-facing search/filter input before comparing it, but keep the original value when the exact spelling matters to the domain.

```
let queryText = q.trim();
let normalizedQuery = queryText.to_lowercase();
let hasVelran = normalizedQuery.contains("velran");
```

Use `replace` for deterministic substitutions, not for general parsing:

```
let displayTag = tag.trim().replace("_", " ");
```

When slicing or locating text, remember that Velran uses Unicode scalar-value positions rather than UTF-8 byte offsets. This makes `substring`, `charAt`, and the index functions suitable for application text without exposing byte-level encoding details. For structured identifiers such as URLs, slugs, emails, dates, or UUIDs, prefer the dedicated Velran types and validators instead of rebuilding their rules with string functions.

Choosing the right tool in application code

Web-development task	Preferred Velran approach
Normalize search/filter text	<code>trim</code> , then <code>lower/upper</code> only when comparison should ignore case.
Replace a known token	<code>replace</code> ; do not use regex unless pattern matching is actually required.
Parse email, URL, slug, date, UUID	Use the dedicated typed value and validation path, not ad-hoc string parsing.
Money/totals	<code>Decimal</code> .
Pagination/counters	<code>i64</code> with checked arithmetic.
Business conditions	<code>bool</code> with short-circuit <code>&&</code> , <code> </code> , and <code>!</code> .
Scientific/statistical math	<code>f32</code> and explicit math builtins; check that floating point is appropriate for the domain.
Elapsed-time measurement	<code>monotonicNanos()</code> , never as a wall-clock timestamp.

4.3 Domain values and handler code

Models and business types

```
model Article {
  id: i64
  slug: Slug
  title: String
  body: String
}
```

Common types include `String`, `i64`, and `bool`, as well as `Date`, `DateTime`, `Uuid`, `Decimal`, and `Slug`. Use `Decimal` for money rather than floating-point values.

Page and action

A page contains GET-like presentation logic:

```
#[page]
fn show(ctx: PageContext, db: Db, id: i64)
  -> Result<Html, PageError> {
  let article = articleById(db, id)?;
  return Ok(html { <h1>{{ article.title }}</h1> });
}
```

An action handles state-changing requests:

```
#[action]
fn create(ctx: ActionContext, db: Db, title: String)
  -> Result<Redirect, PageError> {
  transaction db {
    insertArticle(tx, title)?;
  }
  return Ok(redirect(adminArticles()));
}
```

The example assumes a GET route named `adminArticles`; redirect targets are route calls, not copied URL strings.

Variables and error propagation

`let` introduces an immutable typed local value. Use Rust-like `let mut` when the value must change, then use ordinary assignment such as `count = count + 1` or `samples[i] = value`. The static type cannot change, bounds checks and resource accounting remain enforced, and the legacy `set` syntax is rejected. The `?` operator propagates an error from a typed `Result`.

Enum and exhaustive match

Use an enum for a closed set of choices rather than free-form text. This gives a stronger contract in forms, routes, JSON, and domain logic. Enums are also closed sum types: when control flow depends on a variant, use an exhaustive `match`.

```
enum PublishState {
    Draft
    Review
    Published
}

match state {
    Draft => { return Ok(json("draft")); }
    Review => { return Ok(json("review")); }
    Published => { return Ok(json("published")); }
}
```

There is no wildcard fallback for this security-relevant form. Adding a new variant therefore makes old matches fail compilation until the new state is handled explicitly. Later, the transaction chapter uses exactly this property for `CommitUnknown`.

Object

Related model/query/page operations can be grouped under a domain object. This is particularly useful in larger applications because it defines a clear domain API boundary.

4.4 Routing, typed input, and HTML

Route

```
route article GET "/article/:slug<Slug>" public => article;
```

The path parameter type is declared in the route. Invalid input never reaches the handler as a typed value.

HTML escaping

```
return Ok(html {
    <h1>{{ article.title }}</h1>
});
```

`{ ... }` is HTML-escaped. A string read from the database is no exception. There is no raw-HTML API.

Lists and optional values

A list must be iterated, and an optional model may be dereferenced only after an explicit presence check. The compiler does not allow a `Vec<Model>` or an unchecked `Option<Model>` to behave like a single record.

```
@for article in articles {
  <li>{{ article.title }}</li>
}

@if article {
  <h1>{{ article.title }}</h1>
}
```

The database chapter explains the exact runtime query-cardinality contract, including the fact that a `Option<Model>` result is an error if the query returns more than one row.

Typed URLs

```
<a @href(article, article.slug)>Open</a>
<form method="post" @action(deleteArticle, article.id)>
```

Prefer route helpers

Do not duplicate your own application-route URLs as hand-written strings. In HTML, `@action(...)` is the preferred form for POST targets and `@href(...)` for links, because the route name and typed parameters remain the source of truth. Built-in runtime endpoints – such as `/__velran/auth/logout` – are naturally not application route helpers.

Do not build a dynamic `href="{{ value }}"` link. A route helper knows both the destination and the parameter type.

Components and layouts

```
#[layout]
fn Main(title: String) -> Html {
  html {
    <html>
      <head><title>{{ title }}</title></head>
      <body><main>@content</main></body>
    </html>
  }
}
```

Inside a page:

```
return Ok(html {
  @layout(Main, article.title) {
    <article><h1>{{ article.title }}</h1></article>
  }
});
```

A layout may contain exactly one `@content` slot; components and layouts can only access their explicit typed parameters.

4.5 Practice: classify syntax by owner

Exercise mode: guided. Use the guided method defined in the preface.

Review a page, action, query, route, component, and ordinary expression. For each construct, decide whether it is ordinary Rust-like syntax or a Velran framework-owned web boundary. This distinction is useful when guessing syntax: prefer Rust-like forms unless the construct represents a security, HTTP, resource, or framework contract.

4.6 Common mistake: inventing convenience syntax

Do not assume undocumented aliases such as legacy callable forms or custom collection spellings. The language deliberately converges on Rust syntax where possible, while keeping only the framework constructs needed to make web boundaries explicit and verifiable.

4.7 Running project checkpoint: Secure Notes

Define the note list page and create action using current callable attributes, typed route inputs, and explicit access policy. Keep business data in typed values rather than unstructured strings passed between handlers.

Chapter 5

Data entry: typed SQL, CRUD, forms, and PRG

Learning path: **Fast path:** **core.** Add bounded forms and CRUD without weakening authorization or transaction boundaries.

Rust → **Velran**

Reuse from Rust: Typed values, structs, `Option/Result`, comparisons, and normal control flow remain the language of business rules.

Velran adds: Route form declarations, validation bounds, transaction capabilities, authorization proofs, and typed redirects define the protected mutation boundary.

Do not relearn: Do not rewrite business validation as framework magic. Keep domain logic Rust-like and use Velran constructs where data crosses trust or effect boundaries.

Security invariant: **bound input before effect**

Treat request data as untrusted until it is parsed into the expected type, normalized where needed, and checked against explicit bounds. Do this before database mutation, file write, redirect target selection, or another protected effect.

Verifier diagnostic: **SEC-A01-005**

Meaning: A mutation query received an object key without authorization proof for that model/key pair.

Fix: Load the object, authorize it, then pass the authorized key (or an unchanged local alias) into the mutation.

A CRUD mutation follows one cycle: read, accept typed form input, validate, authorize, mutate in a transaction, then redirect and read again. The key distinction is: *valid input is not yet authorized input*.

5.1 Apply the lifecycle to a CRUD mutation

For a typical edit request, read the seven-stage model concretely as follows:

```
POST /notes/42
-> parse: Form<NoteEdit>
```

```
-> validate: title/body bounds
-> authenticate: require a user session
-> authorize: load note 42, then prove ownership/permission
-> transaction: optimistic-lock update + audit
-> response: typed redirect to GET
```

Three rules matter most. Validation never proves authorization. Authentication never proves object ownership. A successful read performed before the transaction never replaces an authoritative concurrency check inside the mutation. Keeping these statements separate prevents many CRUD vulnerabilities from looking like ordinary form bugs.

5.2 Typed SQL and CRUD

Query

```
#[query]
fn articleBySlug(db: Db, slug: Slug)
  -> Result<Article, DbError> sql {
    SELECT id, slug, title, body
    FROM articles
    WHERE slug = :slug
  }
```

The SQL text is static and compiler-owned. A user-provided value may enter only as a typed bind, such as `:slug`.

Mutation in a transaction

```
#[query]
fn articleById(db: Db, id: i64)
  -> Result<Article, DbError> sql {
    SELECT id, slug, title, body
    FROM articles
    WHERE id = :id
  }

#[query]
fn updateArticle(
  tx: Transaction,
  id: i64,
  title: String,
  body: String
) -> Result<(), DbError> mutates Article by id sql {
  UPDATE articles
  SET title = :title, body = :body
  WHERE id = :id
}
```

The action loads the object, establishes authorization, and only then uses the authorized object key for the mutation:

```
#[action]
fn update(
  ctx: ActionContext,
```

```

    db: Db,
    id: i64,
    title: String,
    body: String
) -> Result<Redirect, PageError> {
    let article = articleById(db, id)?;
    authorize article authenticated;
    transaction db {
        updateArticle(tx, article.id, title, body)?;
    }
    return Ok(redirect(adminArticles()));
}

```

For an ordinary local transaction, success commits and a known database failure rolls back. A `Transaction` capability cannot escape the block. For retry-sensitive critical work, capture the explicit transaction outcome instead of assuming every failure proves rollback; Chapter 6 explains `CommitUnknown`.

Portable database schema

V1 aims for patterns that can remain portable across SQLite, PostgreSQL, and MariaDB. Canonical text may be used as the portable representation for `Date`, `DateTime`, `Uuid`, and `Decimal`.

Migration

The server does not migrate automatically at startup. The recommended flow is:

```

migrate verify
migrate apply
migrate verify

```

Keep migration credentials separate from normal application-runtime credentials.

5.3 Forms, validation, and PRG

Named form schema

```

form ArticleForm {
    title<String>
    body<String>
    published<bool>
    validate title length 2 200 body length 1 200000
}

```

Route:

```

route articleCreate POST "/admin/articles"
    form ArticleForm
    auth role Editor
    => create;

```

The handler parameter order and types should follow the form fields.

Invalid submission

For a named form, a missing required field, a field-level typed decode failure, or a declared validation failure results in 422 *Unprocessable Content*. The runtime refills field values in escaped form, attaches field errors, and inserts a CSRF token into the form. An unknown or duplicate field is instead a 400 because it violates the request schema.

This intentionally differs from inline form or JSON schemas: there a typed decode/validation failure is a request-contract violation and returns 400. Use a named form when you want to report the error on the same user-facing form with field-level feedback.

Successful POST and PRG

After a successful mutation, return a redirect so the browser loads the destination with GET. This Post/Redirect/Get pattern eliminates the common accidental resubmission problem.

A complete simple form-action pattern

```
#[action]
fn create(
  ctx: ActionContext,
  db: Db,
  title: String,
  body: String,
  published: bool
) -> Result<Redirect, PageError> {
  let articleSlug = slug(title);
  transaction db {
    insertArticle(tx, articleSlug, title, body)?;
  }
  return Ok(redirect(adminArticles()));
}
```

For a named form, a missing checkbox-like `bool` field is interpreted as `false`.

A practical decision order

For a state-changing form, use this order during design and review:

1. decode and validate the request at the route/form boundary;
2. load the target object if the operation concerns an existing record;
3. establish object/permission/tenant authorization;
4. perform the mutation inside the appropriate transaction or critical-operation contract;
5. redirect with a typed GET route helper;
6. invalidate only the caches that the mutation actually makes stale.

This ordering prevents a common mistake: treating a well-formed identifier or form as proof that the caller may modify the referenced object.

5.4 A complete edit cycle: validation, authorization, concurrency

For a create form, validation and a transaction may be enough. Editing an existing business object adds a third concern: the copy shown to the user may be stale by the time the POST

arrives. Velran keeps these concerns separate rather than turning one successful validation step into an implicit proof of everything that follows.

A practical edit flow is:

1. decode the request into the declared form types;
2. apply field and cross-field validation;
3. load the current object and establish authorization;
4. compare the submitted version through an optimistic-locking mutation;
5. commit the mutation and any business audit record in the same transaction;
6. redirect to a GET route after success.

The version check belongs in the mutation itself. A separate “does this version still exist?” SELECT would recreate a race between checking and writing.

```
model Article {
  id: i64
  title: String
  version: i64
}

#[query]
fn updateArticle(
  tx: Transaction,
  id: i64,
  title: String,
  version: i64
) -> Result<Changed, DbError> mutates Article by id sql {
  UPDATE articles
  SET title = :title, version = version + 1
  WHERE id = :id AND version = :version
}
```

Changed means that exactly one row must change. A zero-row result represents a stale edit and maps to a conflict instead of silently overwriting somebody else’s newer value. More than one affected row violates the mutation invariant and is treated as a database error.

Four boundaries that should not be collapsed

Boundary	Question	Typical outcome
Request schema	Is this request structurally allowed?	unknown/duplicate field: 400
Named-form validation	Can the user correct this value?	field/type/declared validation error: 422
Authorization	May this principal act on this object?	fail closed before mutation
Concurrency invariant	Is the submitted state still current?	stale optimistic-lock version: 409

This separation is useful during review: a 422 is not an authorization decision, and successful authorization is not proof that an old browser tab still contains the latest database state.

5.5 Database invariants and user-friendly validation

Application validation should improve feedback, but authoritative uniqueness belongs in the database. If a slug, email, external identifier, or similar value must be unique, enforce that invariant with a database `UNIQUE` constraint. A preflight `SELECT` can still be useful for friendly feedback, but it cannot replace the constraint because two concurrent requests can both pass the preflight check.

The same rule applies more generally: put presentation-oriented checks at the request boundary, domain refinement in typed/domain values, authorization before protected effects, and race-sensitive integrity in the database or another authoritative platform primitive.

5.6 Mutation review checklist

Before accepting a CRUD mutation, verify all of the following:

- the route has an explicit access policy;
- request fields are typed and bounded;
- the target object is loaded before object-level authorization;
- SQL values enter only through typed binds;
- state changes run under the intended transaction/critical-operation contract;
- concurrently edited records use optimistic locking where lost updates matter;
- uniqueness and other race-sensitive invariants are enforced authoritatively;
- business audit is transactional when the change must be reconstructable later;
- success returns through PRG rather than rendering a mutable POST response;
- cache invalidation is narrow and tied to the data actually changed.

5.7 Verified companion examples

Use these compiler-checked repository programs as the copy/paste source of truth for this chapter:

- `examples/crud/app.vrn` – CRUD and object-authorized mutation;
- `examples/forms/app.vrn` – typed forms and validation;
- `examples/optimistic-locking/main.vrn` – stale-edit protection;
- `examples/prg-flash/app.vrn` – PRG and flash state.

5.8 Practice: design an edit conflict

Exercise mode: supported. Use the supported method defined in the preface.

Simulate two browser sessions opening the same record. Session A saves first; session B then submits stale data. Decide where the version is carried, where the mismatch is detected, which HTTP outcome is returned, and what the user sees next.

5.9 Common mistake: validating only before the transaction

Application validation improves error messages, but race-sensitive invariants still need an authoritative enforcement point. Uniqueness, ownership-sensitive mutations, and optimistic concurrency must remain correct when two requests run at nearly the same time.

5.10 Running project checkpoint: Secure Notes

Use the compiler-checked snapshot in `examples/secure-notes/checkpoints/05-crud/main.vrn`. Add edit and delete operations to Secure Notes. Make ownership checks explicit, use optimistic locking for edits, keep mutation inside the intended transaction boundary, and return through PRG after success.

Chapter 6

Database deep dive: transactions, migrations, and concurrency

Learning path: **Fast path:** **core.** Learn schema/query invariants needed by persistent applications.

Rust → **Velran**

Reuse from Rust: Function signatures, **Result**, **Vec<T>**, typed parameters, and ? remain familiar.

Velran adds: **##[query]** plus compiler-owned SQL, typed bind parameters, static row bounds, and Db/transaction capabilities make database effects reviewable.

Do not relearn: Do not learn string-built SQL as a Velran technique. The new concept is the verified database boundary, not a new way to concatenate queries.

Security invariant: **data is not SQL structure**

SQL structure remains compiler-owned. User-controlled values enter queries only through typed bind parameters, and result size remains explicitly bounded; validation never licenses string-built SQL.

Compiler diagnostic: **SEC-A05-001**

Meaning: The source attempts unsafe SQL construction instead of compiler-owned SQL with typed binds.

Fix: Keep SQL structure static and move request-controlled values into typed bind parameters.

The previous chapter introduced typed SQL and CRUD. This chapter focuses on the database rules that preserve integrity across concurrent requests and releases. On a first reading, prioritize query contracts, typed binds, transactions, optimistic locking, and migrations; treat capacity, backup, and engine-selection details as later reference.

6.1 Three separate responsibilities

In a Velran application it is useful to separate three layers:

application code declares static typed SQL queries, supplies binds, and requests transactions;

runtime pools connections, binds parameters, applies timeouts and result limits, then validates returned rows against the declared type;
operator owns database credentials, TLS, database capacity, backups, and the migration process.

The application therefore does not open arbitrary sockets to the database or build its own connection pool. **Db** and **Transaction** are the capabilities through which database work occurs.

The current V1 data layer also has broad defense-in-depth limits, including a maximum pool size of 32 connections, a 5-second connection-acquire timeout, a 10-second query timeout, a 10,000-row backend result ceiling, and a 16 MiB result-size ceiling. Velran model-list queries are stricter: the compiler requires a literal **LIMIT**, and the compiler/runtime cap that list at 1,000 rows. These are safety ceilings, not pagination design; correct queries should still be short, paginated, and indexable.

With multiple server instances, the operator must budget total database connections across all instances:

```
total DB connections ~= number of Velran instances
                        * pool per instance
                        + admin/maintenance reserve
```

6.2 The query return type is a contract

Query cardinality is visible in the function type:

```
#[query]
fn articleById(db: Db, id: i64)
  -> Result<Article, DbError> sql {
  SELECT id, slug, title, body, version
  FROM articles
  WHERE id = :id
}

#[query]
fn articleBySlug(db: Db, slug: Slug)
  -> Result<Option<Article>, DbError> sql {
  SELECT id, slug, title, body, version
  FROM articles
  WHERE slug = :slug
}

#[query]
fn recentArticles(db: Db, offset: i64)
  -> Result<Vec<Article>, DbError> sql {
  SELECT id, slug, title, body, version
  FROM articles
  ORDER BY id DESC
  LIMIT 100 OFFSET :offset
}
```

The three forms mean:

Type	Expected result	Cardinality violation
Article	exactly one row	0 rows: 404; multiple rows: DB error
Option<Article>	zero or one row	multiple rows: DB error
Vec<Article>	zero or more rows	runtime result limits apply

This is more than convenient notation: result shape is part of the program contract and is enforced at runtime. **Option<Article>** does not mean “take the first row if one exists.” If the query returns two or more rows, the invariant is broken and the runtime fails closed with a DB error. Similarly, an **Article** query returns 404 **Not Found** for zero rows and does not arbitrarily choose a row if there are several.

A **Vec<Model>** query must also declare a static literal row bound in SQL. The bound belongs to the reusable query contract, not to a caller-supplied `limit` parameter; use a fixed `LIMIT` and paginate with a bounded offset or another explicit access pattern.

Make row shape explicit

The names and order of `SELECT` fields should match the model. Do not use `SELECT *` in application queries.

```
model Article {
  id: i64
  slug: Slug
  title: String
  body: String
  version: i64
}

// good: the same five fields, in the same order
SELECT id, slug, title, body, version
FROM articles
```

Explicit projection matters for two reasons. First, the compiler can verify the relationship between model and query. Second, later schema expansion does not silently alter an old query’s result shape.

6.3 Bind values; do not build SQL

Client-derived values enter SQL as typed binds:

```
#[query]
fn findByEmail(db: Db, email: String)
-> Result<Option<User>, DbError> sql {
  SELECT id, email, display_name
  FROM users
  WHERE email = :email
}
```

`:email` is not string substitution. The runtime translates it to a backend-specific placeholder and binds it as a parameter.

Do not think in terms of:

```
// wrong approach:
// "SELECT ... ORDER BY " + userOrder
// "WHERE " + userFilter
```

Column names, `ORDER BY` direction, SQL operators, and entire `WHERE` fragments must not come from user input. If several sort modes are supported, choose among a small number of static queries in application code.

6.4 Read and mutation authority boundaries

A read query receives a `Db` capability. A mutating query receives a `Transaction`:

```
#[query]
fn renameArticle(
    tx: Transaction,
    id: i64,
    title: String
) -> Result<(), DbError> mutates Article by id sql {
    UPDATE articles
    SET title = :title
    WHERE id = :id
}
```

The caller opens an explicit transaction block:

```
#[action]
fn rename(
    ctx: ActionContext,
    db: Db,
    id: i64,
    title: String
) -> Result<Redirect, PageError> {
    let article = articleById(db, id)?;
    authorize article authenticated;
    transaction db {
        renameArticle(tx, article.id, title)?;
    }
    return Ok(redirect(adminArticles()));
}
```

For ordinary best-effort application code, a successful block commits and a known database failure rolls back. For retry-sensitive or critical work, do not collapse the outcome to a Boolean success/failure. Capture the transaction outcome explicitly:

```
let article = articleById(db, id)?;
authorize article authenticated;
let outcome = transaction db {
    renameArticle(tx, article.id, title)?;
};

match outcome {
    Committed => { return Ok(redirect(adminArticles())); }
    RolledBack => { fail conflict; }
    CommitUnknown => { fail conflict; }
```

```
}

```

`CommitUnknown` means the client/runtime did not receive enough evidence to prove whether the commit took effect. It must not automatically trigger a retry. Critical idempotent transactions are compiler-required to capture and exhaustively match this outcome. `Transaction` still cannot escape the block, so a closed transaction handle cannot accidentally be reused.

One business operation should be one transaction

Do not split related mutations across separate actions or separate transactions:

```
transaction db {
  Article.publishChanged(tx, article.id, version)?;
  audit Article article.id action publish;
    from article.status to ArticleStatus.Published
}
```

Here publishing and the business audit trail commit together. If the audit record cannot be written, publishing rolls back too. This prevents a state where the business change happened but its required audit record is missing.

6.5 Optimistic locking: do not overwrite someone else's work

In a CMS or wiki, two editors may open the same record. A simple

```
UPDATE articles SET title = :title WHERE id = :id
```

makes the last writer win and can silently discard the first editor's changes.

Use a version column and `Changed`:

```
model Article {
  id: i64
  title: String
  version: i64
}

#[query]
fn updateArticle(
  tx: Transaction,
  id: i64,
  title: String,
  version: i64
) -> Result<Changed, DbError> mutates Article by id sql {
  UPDATE articles
  SET title = :title,
      version = version + 1
  WHERE id = :id AND version = :version
}
```

At the call site, load and authorize the article first and pass `article.id`, not the request ID directly, into this mutation. The edit form sends the `version` value the user saw when opening the page. The `Changed` contract is strict: the mutation must affect **exactly one row**. One

changed row is success; zero changed rows become **409 Conflict**; two or more rows become a DB error because the query violated its own invariant. **Changed** is only valid as the return type of a mutating query, and in V1 it cannot be combined with a **RETURNING** clause.

With optimistic locking, if another editor saved in the meantime, **WHERE id = :id AND version = :version** matches zero rows and the request becomes a conflict. The correct UX is not an automatic retry. Tell the user that the data changed, load the new version, and let the user decide how to merge.

Typical uses include:

- wiki pages and CMS articles;
- tickets or customer records;
- business objects with workflow state;
- rarely changed administrator configuration.

It is generally unnecessary for append-only events or database-side atomic counters.

6.6 Design indexes for access patterns

An index is useful not because there are many of them, but because it serves a real query.

Natural CMS access patterns might justify:

```
CREATE UNIQUE INDEX articles_slug_uq
  ON articles(slug);

CREATE INDEX articles_status_id_idx
  ON articles(status, id);
```

If the public site loads by slug, make slug unique/indexed. If the admin list filters by status and pages by identifier, a matching composite index may be appropriate.

Some rules:

- paginate every large list;
- index frequently used lookup fields;
- align index order with **WHERE/ORDER BY** patterns;
- do not index every column automatically: indexes cost writes and storage;
- investigate performance with the real backend's **EXPLAIN** plan.

Velran typed SQL protects query shape; it does not replace the database optimizer.

6.7 Migration: schema is part of the release

In production, the server does not run migrations automatically at startup. This is deliberate: a schema change is a separate operator action with separate credentials and a separate failure boundary.

A typical directory is:

```
migrations/
  0001_create_articles.sql
  0002_articles_slug_index.sql
  0003_articles_add_version.sql
```

File names use:

```
NNNN_name.sql
```

Do not modify an already-applied migration. The CLI records a SHA-256 checksum and stops on mismatch.

Production commands using the installed CLI:

```
/usr/local/bin/velran-cli migrate status \  
--dir /srv/myapp/current/migrations \  
--db-url-file /run/secrets/migration-db-url  
  
/usr/local/bin/velran-cli migrate verify \  
--dir /srv/myapp/current/migrations \  
--db-url-file /run/secrets/migration-db-url  
  
/usr/local/bin/velran-cli migrate apply \  
--dir /srv/myapp/current/migrations \  
--db-url-file /run/secrets/migration-db-url \  
--lock-timeout-secs 30
```

Migration credentials may have more privileges than application DML credentials. The server process should therefore not receive them.

What does the migration CLI guarantee?

- applied file name and checksum are verified;
- an old version number cannot silently receive a new migration later;
- a database-level migration lock is acquired;
- migration state advances only after successful application.

Backend-specific differences remain. PostgreSQL uses a transaction per migration; SQLite keeps the batch in a transaction using a `BEGIN IMMEDIATE` writer lock; MariaDB DDL is not transactional in every case, so partial DDL failure may require operator repair.

6.8 Expand/contract: preserve rollback options

A fast application rollback is straightforward only when the new schema remains compatible with the old application.

For larger changes, use expand/contract:

1. **expand**: add a new nullable/defaulted field or a new table while keeping the old application functional;
2. deploy the new application;
3. perform a controlled backfill if necessary;
4. in a later release, **contract**: remove the old field or constraint after the rollback window has closed.

Immediate rename/drop, a new mandatory field, or an incompatible type change can create a forward-only release. Treat that as an explicit release-review decision.

6.9 Handling database errors

`DbError` is a technical error. Do not expose driver messages, SQL text, host names, or connection strings to the end user.

At application level distinguish at least:

- record not found** normal domain state, often handled with an optional query;
- 409 concurrent modification** optimistic-lock conflict; the user resolves it by reloading/-comparing;
- database unavailable** infrastructure failure; readiness may report it and the server log can be searched by request ID;
- constraint/domain failure** preferably prevent it with validation, while keeping the database constraint as the final integrity boundary.

Do not wrap an entire mutating action in an unlimited automatic retry loop. Repeating a request may repeat side effects, and a transaction conflict is not necessarily transient.

6.10 SQLite, PostgreSQL, or MariaDB?

Velran V1 supports all three backends, but that does not make every SQL dialect feature portable.

SQLite excellent for simple or single-host deployments; back up a live application database using an SQLite-aware method rather than plain copying the live file.

PostgreSQL a strong default for larger or multi-instance production systems; use TLS for remote connections.

MariaDB also supported, but account for DDL transactional differences especially during migrations.

Portable applications should use a common SQL subset. Use backend-specific features only deliberately, and document the backend dependency in the project.

6.11 Backup from a database developer's perspective

As a database developer, remember one key point here: migration and backup are separate lifecycles. Migration describes how schema changes; backup/restore proves that persistent state can be recovered. For PostgreSQL, MariaDB, and SQLite, use the backend's own consistency-aware backup method and do not assume that a plain file copy is automatically a valid backup.

Full application state – database, AppFs, local-auth store, recovery manifest, recovery point objective (RPO), recovery time objective (RTO), and restore drills – is covered in Chapter 19. It matters here because schema-change planning must already know what state is available for rollback or restoration.

6.12 A sample CMS database layer

A small CMS might start with:

Article	
id	
slug	UNIQUE
title	

```
body
status
hero_image
version
created_at
updated_at

migrations/
  0001_create_articles.sql
  0002_article_indexes.sql
  0003_add_version.sql
```

Application patterns:

- public read: `Option<Article>` by slug;
- admin list: paginated `Vec<Article>`;
- edit: `Changed + version`;
- publish/archive: business mutation + audit in one transaction;
- schema change: separate CLI migration;
- before deploy: migration verify + backup/restore readiness;
- after deploy: readiness, logs, metrics, and smoke test.

With this model the database layer is not a separate collection of SQL snippets; it becomes part of the web application’s type, concurrency, release, and recovery contract.

6.13 Chapter checklist

Before considering a database-backed feature complete, verify:

- every user value enters SQL as a typed bind;
- there is no `SELECT *`; projections match models;
- large lists are paginated;
- important lookup/sort patterns have appropriate indexes;
- every mutation runs with a `Transaction` capability;
- related mutations share one transaction;
- human-concurrently-edited records use optimistic locking;
- already-applied migrations are immutable;
- migration credentials are not available to the application server;
- destructive schema changes have a known rollback consequence;
- DB errors do not leak SQL or credentials to the client;
- backups have regularly proven restores.

6.14 Practice: review a query as an interface

Exercise mode: supported. Use the supported method defined in the preface.

For each query in a feature, write down its input types, projected columns, cardinality expectation, ordering, and index assumption. Treat that list as an interface contract rather than as an implementation detail hidden inside SQL.

6.15 Common mistake: using the database only as storage

A production database also enforces invariants and concurrency semantics. If correctness depends on a condition remaining true under concurrent requests, encode the authoritative part in the database or transaction design rather than relying only on a prior application-side check.

6.16 Running project checkpoint: Secure Notes

Start from `examples/secure-notes/migrations/0001_init.sql`. Review the title/body bounds, publication-state constraint, timestamps, and owner/id index as database-level invariants. Then add pagination while preserving deterministic ordering and principal scoping; document which guarantees belong to SQL/schema design and which remain handler-level policy.

Chapter 7

Authentication, sessions, and authorization

Learning path: **Fast path:** **core.** Distinguish authentication, route policy, and object authorization.

Rust → **Velran**

Reuse from Rust: Enums, typed identifiers, **Result**, branching, and ordinary domain predicates remain useful for policy logic.

Velran adds: The framework supplies a trusted principal, route-level auth policy, and explicit authorization boundaries for protected objects/effects.

Do not relearn: Do not replace domain rules with framework vocabulary. Authentication provides identity; your typed business rule still decides authority, then Velran enforces where that proof must occur.

Security invariant: identity is not authority

Authentication establishes the caller identity. Authorization establishes whether that caller may perform this operation on this object. Complete authorization before the protected read or mutation.

Verifier diagnostic: SEC-A01-022

Meaning: The route identity is authenticated, but its role/permission proof does not satisfy the handler requirement.

Fix: Strengthen the route policy or change the handler requirement only if the business authorization contract actually permits it.

Authentication proves identity; authorization decides permitted actions. A session carries trusted identity across requests. Velran keeps application route policy in `.vrn`, while identity and session policy remain server configuration.

7.1 Where authentication and authorization sit in the lifecycle

Use the lifecycle model to keep identity questions separate from object authority:

```
request
-> parse / validate
```

```
-> authenticate: who is calling?  
-> authorize: may this caller act on this object?  
-> query / transaction  
-> response
```

Route policy answers broad admission. Object, tenant, permission, MFA, or critical-operation checks can narrow authority further: authenticated means *trusted principal available*, not *operation allowed*.

Login establishes authentication rather than consuming it: validate the credential form and CSRF state, verify the identity backend, rotate the session, audit, then redirect. Credential policy belongs to the identity backend, not object authorization.

7.2 The complete login path

The built-in HTML login endpoints are:

```
GET  /__velran/auth/login  
POST /__velran/auth/login  
POST /__velran/auth/logout
```

These are server endpoints, not routes defined by the application. The GET login page issues a CSRF token bound to the session. The POST login form expects exactly the `_csrf`, `username`, `password`, and `totp` fields.

At a high level, a successful login:

1. validates form structure and the CSRF token;
2. canonicalizes the username;
3. applies the login rate limit;
4. verifies local-auth or LDAP credentials;
5. if required, verifies TOTP or, for local auth, a recovery code;
6. rotates to a new authenticated session;
7. writes an audit event;
8. redirects to the root with 303 **See Other**.

Session rotation matters: after successful authentication, the anonymous session identifier is not kept. This is a basic defense against session fixation.

7.3 Local auth: a separate identity database

A single-node application may use the local-auth SQLite store. It is deliberately separate from the application database. Velran typed SQL queries therefore cannot access credential tables.

For example, the secret file may be:

```
/run/secrets/velran/local-auth-db-url
```

with contents:

```
sqlite:///srv/velran/auth/users.db?mode=rwc
```

Initialize it with:

```
/usr/local/bin/velran-cli auth init \  
--db-url-file /run/secrets/velran/local-auth-db-url
```

When creating a user, do not pass the password as a command-line argument; read it from a file:

```
printf '%s\n' 'a-very-long-random-password' \  
> /run/secrets/velran/new-user-password  
chmod 600 /run/secrets/velran/new-user-password  
  
/usr/local/bin/velran-cli auth user-add \  
--db-url-file /run/secrets/velran/local-auth-db-url \  
--username alice \  
--password-file /run/secrets/velran/new-user-password \  
--role User \  
--role Editor
```

The local store uses Argon2id password hashing with a unique random salt. The current implementation accepts passwords between 12 and 1024 characters. Do not confuse this with application-side form validation: credential policy belongs to the identity backend.

7.4 Account lifecycle through the operator CLI

User state does not need to be modified with direct SQL. The CLI provides explicit operations:

```
velran-cli auth password-set --db-url-file ... \  
--username alice --password-file ...  
  
velran-cli auth disable --db-url-file ... --username alice  
velran-cli auth enable --db-url-file ... --username alice  
velran-cli auth roles-set --db-url-file ... --username alice \  
--role User --role Publisher
```

Disabling an account does not delete it. From the outside, failed login does not reveal whether the cause was a wrong password, nonexistent user, or disabled account, because such distinctions can become account-enumeration channels.

7.5 Session invalidation: `auth_generation`

With local auth, credential or authorization changes increment `auth_generation`. Each session records its creation generation; a mismatch on a later request invalidates that session and replaces it with a fresh anonymous one.

Consequently, the following operator actions affect existing sessions too, not merely the next login:

- password change;
- role change;
- account disable/enable;
- TOTP enrollment or disable.

This is particularly important for role revocation: there is no need to wait for the old session to expire naturally.

7.6 Session store: memory or Redis

An in-memory session store may be used for development and testing. In production, especially with multiple server instances, Redis should be the shared session backend.

```
[redis]
url_file = "/run/secrets/velran/redis-url"
allow_insecure = false
```

Redis may hold several kinds of short-lived infrastructure state in Velran:

- shared sessions;
- TOTP replay protection;
- login rate limiting;
- public cache.

Redis is not a business source of truth. After disaster recovery, an empty session/cache state is acceptable: users sign in again and caches rebuild.

7.7 Cookie policy

In normal production mode the session cookie is named:

```
__Host-velran_session
```

The server uses `Path=/`, `HttpOnly`, and `Secure`. The default is `SameSite=Lax`; with credentialed cross-origin configuration it becomes `SameSite=None`; `Secure`.

The explicit development escape hatch `-insecure-dev-cookies` changes the cookie name to `velran_session` and omits `Secure`. Never use this for an Internet-facing production deployment.

7.8 Logout

Logout is intentionally a POST operation:

```
<form method="post" action="/__velran/auth/logout">
  <input type="hidden" name="_csrf" value="{{ csrfToken }}">
  <button type="submit">Logout</button>
</form>
```

The server verifies the CSRF token, invalidates the old session, creates a fresh anonymous session, sends a new cookie, and audits the logout event. Logout is therefore not simply a client-side cookie deletion.

7.9 LDAP / LDAPS for enterprise identity

For shared or enterprise identity, an LDAP backend may be used. Use an LDAPS URL in production, with service-bind credentials loaded from secret files.

Example configuration:

```
[auth]
ldap_url = "ldaps://ldap.example.com:636"
ldap_search_base = "ou=people,dc=example,dc=com"
ldap_username_attribute = "uid"
ldap_service_bind_dn_file = "/run/secrets/velran/ldap-bind-dn"
ldap_service_bind_password_file = "/run/secrets/velran/ldap-bind-password"
require_totp = true
```

Do not configure local auth and LDAP as simultaneous primary identity backends in one process. That avoids ambiguous backend priority and username collisions.

With LDAP, role mapping should come from trusted server-side configuration. Client-supplied `role`, `username`, or similar fields are never authorization authority.

7.10 TOTP and recovery codes

Enroll TOTP for a local-auth user with:

```
/usr/local/bin/velran-cli auth totp-enroll \
  --db-url-file /run/secrets/velran/local-auth-db-url \
  --username alice
```

The CLI prints the secret/URI data required for enrollment and the recovery codes. Recovery codes remain in the local-auth database only as hashes, and a successfully used code is consumed.

Disable TOTP with:

```
/usr/local/bin/velran-cli auth totp-disable \
  --db-url-file /run/secrets/velran/local-auth-db-url \
  --username alice
```

If `require_totp = true`, a user with a correct password still cannot sign in without a usable second factor.

The login form's `totp` field may contain either a six-digit TOTP or, with local auth, a recovery code. With Redis, replay state is shared, so multiple server instances see the same TOTP replay protection.

7.11 Route authorization

Applications commonly declare three route-level policies:

```
route account GET "/account" auth user => account;
route admin GET "/admin" auth role Admin => admin;
route security GET "/security" auth mfa => security;
```

Their meanings are:

- `auth user`: any authenticated principal;
- `auth role Admin`: an authenticated principal with the trusted `Admin` role in the session;

- `auth mfa`: an authenticated principal that has successfully proven a second factor.

For an HTML route, an unauthenticated request may redirect to the login page. For a JSON API, the same condition appears as 401 `Unauthorized` rather than an HTML redirect, which is more predictable for API clients.

7.12 Using the principal on a page

An authenticated page receives trusted session state:

```
#[page]
fn account(ctx: PageContext) -> Result<Html, PageError> {
  return Ok(html {
    <main>
      <h1>Account</h1>
      <p>Signed in as {{ authPrincipal }}</p>
      <form method="post" action="/__velran/auth/logout">
        <input type="hidden" name="_csrf" value="{{ csrfToken }}">
        <button type="submit">Logout</button>
      </form>
    </main>
  });
}

route account GET "/account" auth user => account;
```

`authPrincipal`, session roles, and MFA state are server-side trusted state. Do not reconstruct them from hidden form fields or query parameters.

7.13 A route role is not always enough

A route-level role answers whether a user may enter an operation. Object-level authorization may require more.

For example, an editor may be allowed to modify only their own draft. In that case `auth role Editor` alone is insufficient: the action must compare the owner of the record loaded from the database with the trusted session principal.

The core rule is:

identity from the session, object state from the database, decision on the server

Client-supplied `owner`, `actor`, or `role` fields are data at most, never authority.

The typed object-authorization guard

The `authorize` guard may be used only on an already-loaded, **non-optional** model record. The compiler also verifies that the `owner` field exists and has type `String`. A query that returns `Option<Article>` must therefore be handled explicitly first; authorization does not interpret a “no record” state.

```
let article = loadArticle(db, id)?;
authorize article owner authorUsername or role Publisher;
```

A handler containing this guard cannot be exposed through a public route, and public cache cannot sit in front of object authorization. The owner or one of the listed trusted roles is allowed; every other authenticated actor receives **403 Forbidden**.

7.14 Login rate limiting

Login has its own rate-limit policy. For example:

```
[auth]
login_max_attempts = 8
login_principal_max_attempts = 16
login_source_max_attempts = 64
mfa_max_attempts = 5
login_window_secs = 300
```

After trusted-proxy processing, the server uses the effective client IP together with the canonical username as part of the limiter key. This is why `trusted_proxy_cidrs` is also an authentication-security setting behind a reverse proxy.

A failed credential or second factor does not produce detailed failure information for the client. Use **429 Too Many Requests** for rate limiting and **503 Service Unavailable** when the authentication backend is unavailable.

The limiter tracks source+principal, principal across sources, and source across principals; MFA/recovery attempts use a stricter stage. This covers distributed guessing and credential stuffing. Shared limiter-store failure is fail closed.

7.15 CSRF, Origin, and Fetch Metadata

With session-cookie authentication, the browser sends the cookie automatically, so state-changing requests require CSRF protection. Velran uses defense in depth:

- a session-bound CSRF token;
- same-origin Origin/Referer policy;
- Fetch Metadata controls;
- strict request parsing;
- trusted-proxy policy.

CORS does not replace CSRF protection. If you allow credentialed CORS for a separate frontend origin, that is a distinct security policy and requires HTTPS.

7.16 Authentication audit

Login and logout events go to the audit log. Useful fields include request ID, actor, outcome, and effective client IP. The purpose of the audit log is event reconstruction, not credential debugging.

Never log:

- passwords;
- TOTP secrets or recovery codes;
- session IDs or full Cookie headers;
- CSRF tokens;

- LDAP bind or DB/Redis credentials.

A login result may distinguish success, invalid credentials, invalid second factor, or rate limiting internally, while the client-facing message remains intentionally less detailed.

7.17 Local auth or LDAP?

	Local auth	LDAP/LDAPS
Identity store	separate SQLite DB	enterprise directory
Typical deployment	single-node	shared/enterprise identity
User administration	Velran CLI	directory admin workflow
Role source	local-auth store	trusted mapping
TOTP	local enrollment + recovery	server-side TOTP policy/secrets
Multi-instance identity	not the target use case	yes

If the application needs public self-registration, email password reset, or self-service MFA enrollment, do not bolt a thin route onto the operator CLI. Those features require token lifecycle, email security, rate limiting, and their own recovery threat model.

7.18 Rich text in an authenticated CMS

An authenticated CMS editor interface still does not justify storing raw HTML. Editor authorization and content sanitization are separate defenses. Store Markdown as content and render it with the safe renderer:

```
<article class="article-body">
  @markdown(article.body)
</article>
```

`@markdown` does not permit raw HTML and filters link targets through an allowlist.

7.19 Production checklist

Before enabling authentication in production, verify:

- exactly one primary identity backend is active;
- the local-auth DB is separate from the application DB;
- passwords and service credentials come only from secret files;
- production sessions use Redis when multiple server instances run;
- cookies are Secure/HttpOnly and `-insecure-dev-cookies` is not enabled;
- `trusted_proxy_cidrs` covers only the real Apache/Nginx proxy addresses;
- the login limiter is configured;
- state-changing forms use CSRF protection;
- object-level authorization supplements route roles where necessary;
- credentials and session secrets never enter logs;
- account disable, role changes, and TOTP changes have been tested for session revocation.

7.20 Verified companion examples

The canonical runnable companions are:

- `examples/auth/app.vrn` – route authentication and role policy;
- `examples/object-authorization/app.vrn` – object-level authority;
- `examples/mfa-elevation/main.vrn` – MFA elevation;
- `examples/critical-operations/main.vrn` – critical-operation contracts.

Together they keep these authority mechanisms distinct instead of collapsing them into one mechanism.

7.21 Practice: build an authorization matrix

Exercise mode: supported. Use the supported method defined in the preface.

Create a small table with anonymous user, authenticated owner, authenticated non-owner, privileged role, and disabled account. List what each actor may do to a note. Use the table to distinguish route-level access policy from object-level authorization.

7.22 Common mistake: equating authentication with authorization

Knowing who the caller is does not prove that the caller may access a particular record. Route policy should establish the broad admission rule; ownership, tenant, or domain-specific checks must still be evaluated against the loaded object.

7.23 Running project checkpoint: Secure Notes

Use `examples/secure-notes/checkpoints/07-auth/main.vrn`. Require authentication for all note operations. Enforce ownership on view, edit, and delete paths. Then test the negative case deliberately: a valid logged-in user must still be unable to access another user's note by guessing an identifier.

Chapter 8

Files, uploads, and the media library

Learning path: **Fast path:** **optional branch.** Read now if the application accepts uploads or serves application-owned media.

Security invariant: client file metadata is untrusted

The submitted file name and Content-Type are descriptive metadata, not proof of safe storage or safe content. Keep storage behind AppFs/capability boundaries and derive serving decisions from trusted detection and policy.

Compiler diagnostic: SEC-FILE-001

Meaning: An `Image` upload is declared without the explicit publish transition required before serving it as media.

Fix: Use the typed image upload form with `publish`; raw/unverified uploads remain private.

Uploads cross request, resource, filesystem, authorization, persistence, and serving boundaries. Velran therefore separates general streamed `Upload` values from the stricter validated `Image` type used for publishable media.

8.1 The mental model: three different things

Keep these three concepts separate during upload handling:

1. **client metadata:** original filename and client-declared MIME type;
2. **storage:** the server-side file selected by the runtime inside AppFs;
3. **business reference:** the value the application stores in the database and uses later.

The client filename is not a storage path, and the client's `Content-Type` header is not security evidence. Treat both at most as display or audit metadata.

8.2 General file upload: `Upload`

The route declaration tells the runtime that a multipart file is expected and names the static AppFs subdirectory that receives it:

```
route uploadFile POST "/upload"
  upload file<Upload> to "uploads"
  public => uploadFile;
```


The action does not receive the whole file in memory. `Upload` exposes upload metadata:

```
#[action]
fn uploadFile(ctx: ActionContext, file: Upload)
  -> Result<Redirect, PageError> {
  let savedPath = file.path;
  let originalName = file.filename;
  let contentType = file.contentType;
  let byteCount = file.bytes;
  return Ok(redirect(uploaded()));
}
```

The redirect uses the typed `uploaded()` GET route helper. More importantly, a raw `Upload` remains private application data; it does not become a public asset merely because the handler can inspect its metadata. This is also an important resource-protection property: the runtime streams multipart content, so a large file does not automatically become an equally large in-memory request allocation.

8.3 AppFs: the application's writable filesystem

AppFs is the restricted filesystem area assigned to the application. Give it a dedicated data root in production; do not mix it with the program binary, static assets, or other writable system areas.

Example server configuration:

```
[storage]
data_root = "/srv/velran/data"
fs_mode = "rwc"
max_upload_bytes = 16777216
max_image_pixels = 40000000
```

The short capability letters mean:

r read;
w write;
c create and the operations required to commit an upload.

Uploads typically require `rwc`. The byte limit is an operator hard limit; application code cannot raise it arbitrarily for one route.

On Linux, AppFs confinement is intended to prevent an application from escaping the assigned root through path traversal, symlinks, or magic-link tricks. Normal Unix ownership and permissions are still required: the service user should have write access only to directories it actually needs.

8.4 What happens during an upload?

From a developer perspective, the safe upload path is:

1. the runtime checks request-size limits and the route upload schema;
2. it streams the file into a confined staging location;
3. the server generates the storage key rather than deriving it from the client filename;
4. typed uploads perform additional server-side content validation;

5. after successful processing the file can be committed atomically;
6. on handler/runtime failure the runtime attempts cleanup.

Do not bypass AppFs with a parallel filename-sanitizing write path; that would bypass the framework boundary.

8.5 Image upload: prefer the Image type

For images in a CMS, wiki, news site, or admin interface, use `Image` rather than the general `Upload` surface:

```
route saveHero POST "/admin/hero"  
  upload hero<Image> to "media" publish  
  auth role Editor  
=> saveHero;
```

`Image` uses the same streaming multipart and AppFs path, but the server also validates actual file content. V1 accepts PNG and JPEG. The client-declared MIME type is not authority.

SVG is intentionally not a general image-upload format because it can carry active content and script/XSS surface. Do not assume GIF, WebP, or AVIF support unless the documentation for the runtime version explicitly states it.

8.6 Byte limits and pixel limits are different

Images have two distinct resource risks. A file may be small in bytes but decode into an enormous number of pixels. The pixel ceiling (`max_image_pixels`) is therefore separate from the upload-byte ceiling (`max_upload_bytes`).

```
max_upload_bytes = 16777216  
max_image_pixels = 40000000
```

The second limit caps `width * height`. Set both consciously for the application's requirements; a "16 MB upload limit" alone is not image-decode protection.

8.7 Image references in the database

`Image` is a first-class scalar, so it can be stored as a model field and in the database using a canonical representation. The application does not need – and should not attempt – to assemble or parse internal storage-path components manually.

A typical business model:

```
model Article {  
  id: i64  
  slug: Slug  
  title: String  
  body: String  
  hero: Image  
}
```

The important point is that the database stores a *reference*, not a client filename and not a hand-built public URL. Storage and HTTP-serving policy therefore remain under runtime control.

8.8 Safe rendering in HTML

A validated image is rendered using the dedicated renderer rather than direct string interpolation:

```
#[page]
fn article(ctx: PageContext, article: Article)
  -> Result<Html, PageError> {
  return Ok(html {
    <article>
      <h1>{{ article.title }}</h1>
      @image(article.hero, article.title)
      @markdown(article.body)
    </article>
  });
}
```

`@image(image, alt)` generates HTML from the validated image reference. It escapes the alt text, uses validated dimensions, and targets the built-in read-only media endpoint. Direct `{ image }` interpolation of an `Image` is a compiler error, so the typed reference cannot accidentally become a raw URL or markup value.

8.9 The built-in media endpoint

The runtime reserves this URL prefix:

```
/__velran/media/
```

Applications cannot declare conflicting routes. The media endpoint does not make the entire `data_root` browsable over HTTP; it serves images only from AppFs directories declared by an `upload ...<Image> to "..."` publish route in the program.

When serving media, the server re-detects content based on the image detector rather than trusting the client's original MIME type. Responses receive security and cache headers; media serving does not run an application route and does not create a session.

8.10 CMS example: uploading a hero image

An editor workflow normally consists of two business steps:

1. the editor uploads a validated image;
2. the application associates the resulting typed image reference with the article.

The route should at least require authentication; an admin/editor function normally also deserves role policy:

```
route saveHero POST "/admin/articles/:id<i64>/hero"
  upload hero<Image> to "media" publish
  auth role Editor
  => saveHero;
```

Do not assume that “it is only an image, so it is harmless.” An upload route is still a state-changing POST: authentication, CSRF/origin protection, request limits, and an auditable

business operation may all apply.

8.11 Database and AppFs may form one business state

If database rows reference AppFs images, database and AppFs form one recoverable business state. Use a coordinated application DB + local-auth DB + AppFs snapshot under controlled stop/drain; unrelated snapshots are not automatically consistent. Redis session/cache/rate-limit state is not business source of truth.

8.12 Static assets and uploaded files are different

CSS, JavaScript, icons, and other versioned assets shipped with the program have a different lifecycle from user uploads. The static root should be a read-only deployment artifact; the AppFs data root should be a separate, controlled writable runtime area.

This separation prevents several classes of mistakes:

- deployment cannot overwrite user media;
- upload cannot modify application code or versioned assets;
- backup clearly distinguishes business data from reinstallable release artifacts;
- reverse-proxy rules can be designed more cleanly.

8.13 What not to do

- Do not use the client filename directly as a filesystem path.
- Do not trust multipart `Content-Type`.
- Do not expose the entire AppFs data root through an Apache/Nginx alias.
- Do not manually interpolate an `Image` into a URL or HTML.
- Do not treat DB and AppFs as unrelated recovery states when references exist between them.
- Do not bypass operator upload/pixel limits from application code.

8.14 Current media-surface limits

The V1 media surface is intentionally small. Do not build on undocumented assumptions. The current baseline does not promise an automatic thumbnail/resize pipeline, WebP/AVIF conversion, EXIF normalization or metadata stripping, a complete admin media browser, S3/object-storage backends, or general Markdown image syntax.

If an application needs one of these, treat it as a separate architectural feature with its own resource, security, and lifecycle rules.

8.15 Developer upload checklist

- | | |
|---|--|
| 1. Auth/authorization correct? | 2. AppFs root separate and non-public? |
| 3. Reasonable byte limit? | 4. Pixel limit for images too? |
| 5. Using <code>Image</code> for images? | 6. Filename/MIME treated only as metadata? |
| 7. Typed reference in DB? | 8. <code>@image</code> in HTML? |
| 9. DB + AppFs backed up together? | 10. No reverse-proxy alias for data root? |

8.16 Verified companion examples

For exact current syntax, use:

- `examples/upload/app.vrn` – general upload boundary;
- `examples/media/app.vrn` – validated media surface;
- `examples/typed-multipart/main.vrn` – typed multipart decoding.

8.17 Practice: threat-model one upload

Exercise mode: supported. Use the supported method defined in the preface.

For an image upload, enumerate size, pixel count, filename, MIME metadata, decoded format, storage location, authorization, backup, and serving path. Decide which values are trusted only after framework validation and which remain untrusted metadata forever.

8.18 Common mistake: putting uploads under the static root

Static assets are deployment artifacts; uploads are mutable application data. Combining the two weakens rollback and backup semantics and can accidentally bypass authorization or content-handling rules.

8.19 Running project checkpoint: Secure Notes

Optionally attach one image to a note using the typed media path. Keep the file outside the static release tree, store only the typed reference needed by the application, and verify that deleting or replacing a note cannot expose an unrelated user's media.

Chapter 9

JSON API, CORS, and browser client integration

Learning path: **Fast path:** **API branch.** Read now for JSON/API clients or a separate frontend origin.

Rust → **Velran**

Reuse from Rust: Typed scalars, structs/models, collections, **Result**, and reusable business functions are the same application core you use for HTML routes.

Velran adds: JSON input mode, closed request schemas, explicit public projection, content negotiation, and CORS policy define the API boundary.

Do not relearn: Do not build a second business layer for JSON. Reuse the same typed logic and change only the representation and boundary contract.

Security invariant: **CORS is not authorization**

CORS controls which browser origins may make or read cross-origin requests. It does not authenticate a caller, authorize an object, or replace the normal request lifecycle.

Security diagnostic: **SEC-DATA-004**

Meaning: A model is being exposed without an explicit public projection, so internal fields could cross the response boundary unintentionally.

Fix: Define and return the intended public projection instead of serializing the internal model shape directly.

Velran does not force you to choose between classic server-rendered HTML and a JSON API. The same application can contain pages returning **Html**, actions processing forms, and typed JSON endpoints. This is useful both when a small JavaScript component needs some data and when a separate frontend application uses the Velran server as an API.

This chapter cleanly separates three operating models:

1. **same-origin HTML + API:** the browser loads the page and calls the API from the same origin;
2. **cross-origin public API:** another origin calls the API, but there is no session cookie;

3. **cross-origin session auth**: a separate frontend origin uses cookie-based sessions, so CORS and CSRF policy are both required.

The most important rule is: **CORS is neither authentication nor CSRF protection**. CORS is a browser-side access policy; authentication decides who the user is, while CSRF protection prevents an authenticated session from being abused for unauthorized state changes from another origin.

9.1 The same lifecycle also applies to APIs

JSON changes the representation, not the trust model. A state-changing API request should still be reasoned about in the same order:

```
HTTP request
-> parse: closed typed JSON schema
-> validate: bounds/refinements
-> authenticate: session or machine identity when required
-> authorize: object/permission/tenant decision
-> query / transaction
-> response: explicit JSON projection or stable error envelope
```

CORS sits beside this lifecycle rather than replacing one of its stages. It answers whether a browser origin may make the response available to frontend code. It does not establish identity, object authority, CSRF safety, or database integrity. Keeping CORS outside the core authorization chain prevents a common design error: treating an allowed origin as an allowed user.

9.2 The first typed JSON endpoint

A JSON endpoint handler returns `Json`. `json(...)` converts a supported runtime value into a JSON representation.

```
#[page]
fn statusApi(ctx: PageContext) -> Result<Json, PageError> {
  let healthy = true;
  return Ok(json(healthy));
}

route statusApi GET "/api/status" public => statusApi;
```

A successful response is JSON, for example:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

true
```

`json(value)` is not an arbitrary object serializer. Scalars can be encoded directly, but database/domain models cross a public JSON boundary through an explicit `expose(...)` projection. This prevents a newly added model field—especially a sensitive one—from silently appearing in an existing API. The response shape is therefore deliberate rather than a reflection of an arbitrary runtime object.

9.3 JSON lists from the database

Typed SQL and JSON responses connect directly. The query still returns an exact model shape, and the JSON layer represents that typed value.

```
model Product {
  id: i64
  name: String
  price: i64
}

#[query]
fn listProducts(db: Db, offset: i64)
  -> Result<Vec<Product>, DbError> sql {
  SELECT id, name, price
  FROM products
  ORDER BY id
  LIMIT 100 OFFSET :offset
}

#[page]
fn productsApi(ctx: PageContext, db: Db,
               offset: i64)
  -> Result<Json, PageError> {
  let products = listProducts(db, offset)?;
  return Ok(json(expose(products, id, name, price)));
}

route productsApi GET "/api/products"
  query offset<i64>
  validate offset range 0 1000000
  public => productsApi;
```

The important point is not merely JSON encoding. The reusable list query owns its static row bound; the client supplies only a bounded offset, not SQL cardinality authority. Route input remains typed and validated, and the response uses an explicit public projection. A newly added model field therefore does not silently become part of the API.

9.4 Typed JSON POST body

A route declares `json` input mode for a JSON request body.

```
#[action]
fn echoApi(ctx: ActionContext,
           name: String,
           age: i64,
           active: bool)
  -> Result<Json, PageError> {
  return Ok(json(name));
}

route echoApi POST "/api/echo"
  json name<String> age<i64> active<bool>
  validate name length 1 100 age range 0 150
  auth user
  => echoApi;
```

A valid request is:

```
POST /api/echo HTTP/1.1
Content-Type: application/json
Accept: application/json
X-CSRF-Token: <session CSRF token>

{"name": "Alice", "age": 42, "active": true}
```

The JSON input schema is **closed**. Unknown or missing keys, duplicate keys, wrong scalar types, and unsupported nested objects, arrays, floats, or `null` values are errors. These are request-contract violations and therefore return **400 Bad Request**. A missing or incorrect `Content-Type` on a JSON route returns **415 Unsupported Media Type**. Field and request-size limits apply just as they do to other request bodies.

The `json`, `form`, and `upload` body modes are mutually exclusive on the same POST route, and the compiler enforces this. JSON expects `application/json`. Simple forms use URL-encoded form data; uploads use multipart form data. Each endpoint should have one unambiguous request contract.

9.5 Content negotiation and 406

The client can use the `Accept` header to state what representation it can receive. For a JSON route, supported examples include:

```
Accept: application/json
Accept: application/*
Accept: */*
```

If a handler would return JSON but the client accepts only `text/html`, the server returns **406 Not Acceptable**. This is preferable to silently returning a representation the client explicitly rejected.

9.6 API errors

On JSON routes, runtime-classified application errors use a simple common envelope: a stable **error** code, while detailed diagnosis remains in server-side logs. Clients should not parse the English text of error messages; they should use the HTTP status and stable error code.

Once a route is known to return JSON, errors use a JSON envelope, for example:

```
HTTP/1.1 401 Unauthorized
Content-Type: application/json; charset=utf-8

{"error": "unauthorized"}
```

Typical error codes include:

- `bad_request`;
- `unauthorized`;
- `forbidden`;

- `csrf_failed`;
- `unsupported_media_type`;
- `database_unavailable`;
- `resource_limit`;
- `internal_error`.

This is especially important for browser clients. An authenticated JSON route returns `401` to an anonymous request rather than an HTML login redirect. The frontend can therefore decide based on status code without having to interpret HTML as an API error.

9.7 Same-origin: the default and simplest model

If HTML and API are served from the same scheme + host + port combination, they are same-origin from the browser's point of view.

```
https://app.example/  
https://app.example/api/products
```

No CORS configuration is normally needed. A simple client is:

```
const response = await fetch("/api/products?limit=20&offset=0", {  
  headers: { Accept: "application/json" }  
});  
  
if (!response.ok) {  
  throw new Error(`HTTP ${response.status}`);  
}  
  
const products = await response.json();
```

Prefer same-origin unless a separate frontend origin is a real requirement; it removes a trust boundary and most CORS complexity.

9.8 CORS: exact-origin allowlist

Cross-origin browser access is denied by default. Trusted server configuration must enumerate exact origins; there is no wildcard origin.

```
[web]  
cors_origins = ["https://frontend.example"]  
cors_allow_credentials = false
```

The CLI equivalent is:

```
velran-server --cors-origin https://frontend.example ...
```

List every permitted origin explicitly; never reflect an arbitrary request `Origin`.

9.9 Preflight

Before some cross-origin requests, the browser sends an `OPTIONS` preflight. Velran allows the preflight only when:

- the `Origin` is configured;
- the target GET/POST route exists;
- the requested request headers are supported.

Supported headers include `Content-Type`, `Accept`, and `X-CSRF-Token`; preflight remains part of route/origin policy, not a generic allow-all `OPTIONS` endpoint.

9.10 Credentialed CORS with a session cookie

When a frontend on a different origin uses Velran session authentication, credentialed CORS is required:

```
[web]
cors_origins = ["https://frontend.example"]
cors_allow_credentials = true
```

In production this is allowed only over HTTPS. With this configuration the session cookie uses `SameSite=None; Secure`, because the browser must send it on cross-site requests.

The JavaScript fetch call also needs explicit credential mode:

```
const response = await fetch("https://api.example/api/account", {
  credentials: "include",
  headers: { Accept: "application/json" }
});
```

`credentials: "include"` only allows matching cookies to be sent; it grants no application authority.

9.11 CSRF on a JSON API

With cookie-based sessions, the browser manages the session cookie automatically, so state-changing JSON requests need CSRF protection just like HTML forms.

For form POST, the token is a body field:

```
_csrf=<token>
```

For JSON POST, use a header:

```
X-CSRF-Token: <token>
```

For a separate frontend origin, an authenticated token endpoint can be defined:

```
#[page]
fn csrfApi(ctx: PageContext) -> Result<Json, PageError> {
  return Ok(json(csrfToken));
}

route csrfApi GET "/api/csrf" auth user => csrfApi;
```

The token belongs to the same session; CORS does not replace CSRF validation.

9.12 Complete browser POST flow

For a separate session-auth frontend, the browser sends the session cookie and session-bound CSRF token; the runtime verifies origin, authentication, CSRF, and typed input before the handler performs the business operation.

Example client:

```
const csrf = await fetch("https://api.example/api/csrf", {
  credentials: "include",
  headers: { Accept: "application/json" }
}).then(async r => {
  if (!r.ok) throw new Error(`CSRF HTTP ${r.status}`);
  return r.json();
});

const result = await fetch("https://api.example/api/echo", {
  method: "POST",
  credentials: "include",
  headers: {
    "Content-Type": "application/json",
    Accept: "application/json",
    "X-CSRF-Token": csrf
  },
  body: JSON.stringify({ name: "Alice", age: 42, active: true })
}).then(async r => {
  if (!r.ok) throw new Error(`API HTTP ${r.status}`);
  return r.json();
});
```

9.13 Reverse proxy and origin

Behind Apache or Nginx, public scheme/host and trusted-proxy configuration must agree. The application must not decide what origin is safe based on arbitrary forwarded headers supplied by the client. The installation chapter explains the trusted-proxy boundary; the full `server.toml` key reference appears near the end of the book.

Always place the **frontend origin as seen by the browser** in the CORS allowlist, for example:

```
https://frontend.example
```

not an internal proxy or container address.

9.14 What not to do

- Do not enable or reflect CORS permissively; enumerate exact origins.
- Do not treat CORS as authentication or CSRF protection, and do not send session credentials over HTTP in production.
- Keep each POST route to one typed body mode and bound API input.
- JSON clients should handle API status codes such as 401/403, not HTML login redirects.

9.15 Which architecture should you choose?

Situation	Recommended model
server-rendered page + small JS	same-origin HTML + JSON API
SPA on the same domain	same-origin API, no CORS
separate public frontend, no session	exact-origin CORS without credentials
separate frontend + session auth	exact-origin credentialed CORS + HTTPS + CSRF
server-to-server client	CORS is irrelevant; auth and network policy matter

9.16 Runnable example

The complete repository example is `examples/json-api/app.vrn`.

Start same-origin; add a separate frontend origin and CORS only when deployment requires it.

9.17 Verified companion examples

The repository companions are:

- `examples/json-api/app.vrn` – JSON API route flow;
- `examples/typed-query-form/main.vrn` – typed query/form structs;
- `examples/json-typed-response/app.vrn` – typed JSON response boundary.

Prefer these files when you need a complete request/response example rather than an isolated listing.

9.18 Practice: separate API design from CORS design

Exercise mode: supported. Use the supported method defined in the preface.

Describe the JSON request and response contract without mentioning browsers. Then, separately, decide whether a browser on another origin needs permission to call it. This prevents CORS from being mistaken for authentication or API authorization.

9.19 Common mistake: enabling permissive CORS by default

CORS is a browser-origin policy, not a general network security mechanism. Prefer same-origin deployment when practical; when cross-origin access is required, enumerate exact origins and credential behavior deliberately.

9.20 Running project checkpoint: Secure Notes

Start from `examples/secure-notes/checkpoints/09-json-api/main.vrn` and its verified JSON companions. Expose a read-only JSON representation of the authenticated user's notes. Keep the same authorization rules as the HTML page. Add CORS only if you intentionally deploy a separate frontend origin; otherwise leave the API same-origin.

Chapter 10

Static assets and frontend integration

Learning path: **Fast path:** **browser branch.** Read now when adding static assets or progressive browser enhancement.

A web application produces more than dynamic HTML and JSON responses. CSS, icons, images, fonts, source maps, or artifacts from a separate frontend build have a different lifecycle: they arrive as part of a release, ideally remain immutable, and can be cached aggressively by the browser.

Velran therefore serves static assets from a separate, **read-only static root**. This is not the same area as the writable AppFs data root, and it is not the same concept as user-uploaded media.

First read: learn the three file lifecycles, fingerprinted caching, and the V1 JavaScript/CSP boundary. Server-vs-proxy serving examples are reference material for deployment.

10.1 Three separate file lifecycles

In production, keep at least three file categories separate:

	Example	Writable?	Lifecycle
release	CSS, font, logo	read-only	replaced on deploy
runtime data	export, generated file	according to AppFs policy	changes while the app runs
user media	uploaded image	controlled writable	lives with business data

A typical directory structure is:

```
/usr/local/bin/velran-server
/usr/local/bin/velran-cli
/srv/myapp/releases/2026-09-04/
  main.vrn
  migrations/
  public/
    css/
    img/
    fonts/
```

```
/srv/myapp/data/
/run/secrets/velran/
```

Treat `public/` as a deployment artifact. `data/`, by contrast, is runtime state. If they are mixed, an upload may overwrite a release file, or a rollback may accidentally roll user data backward.

10.2 Configuring the static root

Static namespace configuration lives in a separate server-config section:

```
[static_assets]
root = "/srv/myapp/current/public"
url_prefix = "/assets/"
max_asset_bytes = 8388608
max_age_secs = 300
immutable_max_age_secs = 31536000
precompressed = true
```

The same options are available through the CLI. Important flags include:

```
--static-root
--static-url-prefix
--max-static-asset-bytes
--static-max-age-secs
--static-immutable-max-age-secs
```

The static prefix is a reserved route namespace. If the prefix is `/assets/`, the application must not declare a conflicting dynamic or static route. The server should fail closed on collision: startup failure is better than ambiguous route selection per request.

10.3 Fingerprints: the key to long caching

Tie static content and URL together. For example:

```
public/
  css/app.01234567.css
  img/logo.a91f028c.svg
  fonts/ui-f1e2d3c4.woff2
```

Velran treats a filename as fingerprinted when it contains a token of at least eight hexadecimal characters delimited by a dot, hyphen, or underscore. Such a file may receive a long immutable cache policy:

```
Cache-Control: public, max-age=31536000, immutable
```

For a non-fingerprinted file the safer default is short and revalidatable:

```
Cache-Control: public, max-age=300, must-revalidate
```

The key rule is: **do not replace content behind the same URL while using a one-year**

cache. A new CSS build should get a new fingerprinted filename. Old files may remain in the release long enough for old HTML representations to age out.

10.4 A minimal asset build pipeline

Velran does not prescribe a frontend build system. The pipeline only needs to produce deterministic output into `public/`.

For a small application, source might look like:

```
assets-src/  
  app.css  
  logo.svg  
public/
```

A release build may:

1. create a clean `public/` directory;
2. build or copy CSS/fonts/images;
3. compute a content hash;
4. create fingerprinted filenames;
5. optionally produce Brotli/gzip siblings;
6. use the fingerprinted URLs from the application.

The hash is not a security token. Its purpose is to identify the content version in the filename.

10.5 ETag and conditional GET

Every static representation receives its own ETag. The browser can later send:

```
If-None-Match: "velran-..."
```

If the representation has not changed, the server may return `304 Not Modified` without a body. HEAD is supported alongside GET.

A static request does not create a session and does not send a session cookie. This matters for both performance and security: downloading CSS or a font should not become a session-lifecycle event.

10.6 Precompressed Brotli and gzip

The Velran server does not compress static files during the request. A release pipeline may prebuild:

```
app.01234567.css  
app.01234567.css.br  
app.01234567.css.gz
```

Based on `Accept-Encoding`, the server chooses Brotli, then gzip, then the original representation. A compressed response includes headers such as:

```
Content-Encoding: br  
Vary: Accept-Encoding
```

This avoids per-request compression CPU cost and does not expose dynamic compression work as part of the web request resource surface.

10.7 MIME, nosniff, and path confinement

The server maps known web extensions to MIME types and uses `application/octet-stream` for unknown extensions. Responses always include:

```
X-Content-Type-Options: nosniff
```

Static paths are intentionally strict. Valid:

```
/assets/css/app.01234567.css
```

Rejected forms include `..`, encoded traversal, backslashes, empty path components, and control characters. V1 does not allow percent-encoding in static paths; the build pipeline should use URL-safe filenames.

On Linux, static-root access follows the same confinement philosophy as AppFs: serving assets must not become arbitrary host-filesystem reads through symlinks or traversal.

10.8 CSS, images, and fonts in Velran HTML

The V1 CSP allows same-origin style, image, font, and media assets. A normal release may therefore contain:

```
/assets/css/app.01234567.css  
/assets/img/logo.a91f028c.svg  
/assets/fonts/ui-f1e2d3c4.woff2
```

HTML should always reference the fingerprinted URL that belongs to the release. Templates must not construct asset URLs from user input; the asset name is build data, not request data.

10.9 Important V1 boundary: JavaScript may be served, but not executed in Velran HTML

The static server knows the MIME types for `.js` and `.mjs`, so those build artifacts can physically be served. This does **not** mean that HTML rendered by Velran may execute scripts.

In the V1 HTML security model:

- CSP does not grant a `script-src` permission;
- the compiler rejects `<script>` markup;
- there is no inline-script or “unsafe” escape hatch.

This is an intentional security boundary. JavaScript `fetch` examples in the previous chapter therefore have two valid interpretations:

1. a separate frontend application runs on its own origin and calls a Velran JSON API;
2. the code illustrates browser-API client behavior, not that a script tag can be embedded in V1 Velran HTML.

For a classic Velran server-rendered application, build around HTML + CSS + typed form/action. If a full JavaScript frontend is required, deploy it separately and use Velran as a typed JSON backend.

10.10 Should Velran serve assets, or Apache/Nginx?

Two deployment models work.

1. Velran-owned static namespace

This should be the default. The reverse proxy sends every request to Velran, which handles the `/assets/` namespace itself:

```
Internet
|
Apache / Nginx :443
|
127.0.0.1:8080 Velran
|-- application routes
`-- /assets/*
```

The advantage is that fingerprint, ETag, precompressed, nosniff, and path policy stay in one place. This is usually more than sufficient for small and medium applications.

2. The reverse proxy serves assets directly

At higher static traffic volumes, Apache or Nginx may serve the read-only release `public/` directory directly. If you do this, **you must reproduce** the important contract yourself:

- separate read-only static root;
- traversal/symlink-safe path handling;
- correct MIME + `nosniff`;
- long immutable cache for fingerprinted files;
- short/revalidatable cache for non-fingerprinted files;
- correct `Vary` for Brotli/gzip;
- no namespace collision with application routes.

This is an operator optimization, not a requirement for application correctness.

10.11 Nginx example for direct static serving

A simple starting point:

```
server {
    listen 443 ssl;
    server_name app.example;

    location /assets/ {
        alias /srv/myapp/current/public/;
        try_files $uri =404;
        add_header X-Content-Type-Options nosniff always;
    }

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto https;
    }
}
```

This is only a topology baseline. Production configuration must align cache policy, precompressed files, and symlink/release rules with the same immutable deployment model.

10.12 Apache example for direct static serving

The idea is the same with Apache:

```
Alias /assets/ /srv/myapp/current/public/

<Directory /srv/myapp/current/public/>
    Require all granted
    Options -Indexes
    AllowOverride None
    Header always set X-Content-Type-Options "nosniff"
</Directory>

ProxyPass          / http://127.0.0.1:8080/
ProxyPassReverse   / http://127.0.0.1:8080/
```

If Apache serves assets directly, the `Alias` rule must take effect before the proxy catch-all, and cache headers must be configured explicitly.

10.13 Deploy application code and assets as one release

A common asset failure is deploying HTML that references the new CSS filename while the static root still points to the old release – or vice versa.

Use this order:

```
build application
-> build/fingerprint assets
-> optional .br/.gz
-> immutable release directory
-> preflight
-> atomic current switch
-> controlled restart/reload
```

Application code and the `public/` directory should be part of the same release artifact. Rollback then restores both together.

10.14 Checking with curl

After deployment, check at least:

```
curl -I https://app.example/assets/css/app.01234567.css
curl -I -H 'Accept-Encoding: br' \
  https://app.example/assets/css/app.01234567.css
curl -I -H 'If-None-Match: "velran-..."' \
  https://app.example/assets/css/app.01234567.css
```

Verify `Content-Type`, `Cache-Control`, `ETag`, `X-Content-Type-Options`, and, for compressed representations, `Content-Encoding` and `Vary`.

10.15 Practical decision rule

Situation	Recommended approach
server-rendered Velran page	Velran static root + CSS/image/font
small/medium production	proxy forwards everything to Velran
high static traffic	Apache/Nginx direct static with equivalent policy
full JavaScript frontend	separate frontend deployment + Velran JSON API
user upload	AppFs/media pipeline, not static root

The most important rule is: **a static asset is a deployment artifact, not application data**. Preserving that boundary simplifies caching, rollback, backup, and security at the same time.

10.16 Verified companion example

The current static-asset companion is `examples/static-assets/app.vrn`. Keep release assets read-only and separate from AppFs/runtime state exactly as shown by the deployment model in this chapter.

10.17 Practice: classify every frontend file

Exercise mode: supported. Use the supported method defined in the preface.

Take the files used by a page and classify each as immutable deployment asset, generated HTML, JSON response, or mutable application data. Confirm that only immutable deployment assets live under the static root.

10.18 Common mistake: letting frontend convenience erase trust boundaries

A JavaScript frontend does not change server-side authorization requirements. Client validation and hidden controls are usability features; all authoritative validation, authorization, and mutation rules remain on the server boundary.

10.19 Running project checkpoint: Secure Notes

Add a small stylesheet and, if useful, progressive JavaScript enhancement to the notes UI. Keep the application functional without trusting client-side checks, and keep user-generated content outside the static asset tree.

Chapter 11

Organizing larger applications

Learning path: Complete path / scale-up chapter. Read when a single-module application starts becoming difficult to change safely.

Rust → **Velran**

Reuse from Rust: Module decomposition, qualified names, explicit dependencies, and keeping related code together follow familiar Rust design instincts.

Velran adds: Velran resolves an application-root source graph, adds domain object namespaces, and keeps HTTP route policy separate from source/module structure.

Do not relearn: Do not relearn modularity. Use familiar module boundaries; learn only the extra distinction between source location, domain namespace, and external HTTP exposure.

As an application grows, change locality matters more than file count. Use `mod` for source structure and `object` for domain API namespaces; keep HTTP exposure in `route`.

11.1 Three separate questions

Treat the structure of a larger application as three distinct layers:

Tool	Responsibility
<code>mod</code>	In which file or source module does the code live?
<code>object</code>	Which business concept owns the model, query, page, or action?
<code>route</code>	What HTTP surface and what auth/cache/rate-limit policy expose it?

This separation is deliberate. A domain object does not automatically receive a database, current user, filesystem, or network access, and a module does not imply HTTP policy.

11.2 `mod` is not textual include

A small multi-file project might look like:

```
myapp/  
|-- main.vrn  
|-- models.vrn
```

```
|-- queries.vrn
|-- components.vrn
|-- pages.vrn
|-- actions.vrn
|-- migrations/
`-- public/
```

The entry point:

```
mod models;
mod queries;
mod components;
mod pages;
mod actions;
```

`mod` is not C-style textual inclusion and it does not inject declarations into a global namespace. The compiler loads, validates, and compiles the explicit source graph into one program. A module owns a namespace: declarations from `queries.vrn`, for example, are referenced as `queries::name` across module boundaries. Module paths are application-root relative, so nested source files must be loaded explicitly with paths such as `mod article::admin;`.

11.3 When should you switch to feature-based structure?

Splitting by technical type – `models.vrn`, `queries.vrn`, `pages.vrn` – is a good first step, but in a larger application it can mean opening five or six files for one feature change. At that point, organize by business area instead:

```
main.vrn
article.vrn
article/
|-- admin.vrn
`-- components.vrn
account.vrn
account/
`-- profile.vrn
invoice.vrn
invoice/
`-- admin.vrn
routes.vrn
migrations/
public/
```

Then `main.vrn` becomes mostly application composition:

```
mod article;
mod article::admin;
mod article::components;
mod account;
mod account::profile;
mod invoice;
mod invoice::admin;
mod routes;
```

The goal is change locality: one business change should stay mostly inside one feature directory.

11.4 Domain object: a business namespace, not an OOP class

object groups related business names inside a domain namespace. For example:

```
object Article {
  model {
    id: i64
    slug: Slug
    title: String
    body: String
    authorUsername: String
  }

  #[query]
  fn bySlug(db: Db, slug: Slug)
    -> Result<Article, DbError> sql {
    SELECT id, slug, title, body, authorUsername
    FROM articles
    WHERE slug = :slug
  }

  #[page]
  fn show(ctx: PageContext, db: Db, slug: Slug)
    -> Result<Html, PageError> {
    let article = Article.bySlug(db, slug)?;
    return Ok(html {
      <article>
        <h1>{{ article.title }}</h1>
        @markdown(article.body)
      </article>
    });
  }
}
```

Usage becomes expressive:

```
let article = Article.bySlug(db, slug)?;
```

Short domain names become natural too, for example `Invoice.approve`, `Invoice.cancel`, and `UserAccount.disable`.

What may an object contain?

A framework domain object can group operations around one model. The following is only a structural sketch, not standalone compilable Velran source:

```
object Article {
  model { ... }
  #[query]
  fn bySlug(...) ...
  #[page]
  fn show(...) ...
  #[action]
```

```
fn publish(...) ...
    #[component]
fn Card(...) ...
    #[layout]
fn EditorLayout(...) ...
}
```

Exactly one model block belongs to a framework domain **object**. This construct groups framework-owned model/query/page/action names; it is separate from the Rust-like **struct** + inherent **impl** surface used for ordinary pure/domain values. Framework objects still do not imply inheritance, hidden capabilities, virtual dispatch, or reflection. For Rust-like values, associated constructors (for example **Type::new**) and immutable **&self** methods are supported; general mutable/owned receivers remain intentionally fail-closed.

11.5 Package boundaries for reusable application code

When code must be reused across applications or isolated behind a stable library boundary, move it into a local package instead of stretching the application module graph indefinitely. Each package has its own **velran.toml**, private-by-default API, and bounded local dependency graph. Direct dependencies are explicit path dependencies; transitive dependencies stay encapsulated rather than becoming ambient application namespaces.

Use **pub struct**, **pub fn**, public inherent methods, and the narrow package-root module re-export when designing the library surface. Keep framework authority in the application: a reusable library cannot gain HTTP routes, actions, database/network authority, or other web capabilities merely through **pub**. This makes package decomposition an architectural boundary, not a capability-escalation mechanism.

11.6 Dependencies remain explicit

The domain namespace does not hide capabilities. This is good:

```
#[query]
fn byId(db: Db, id: i64)
    -> Result<Article, DbError> sql {
    SELECT id, slug, title, body, authorUsername
    FROM articles
    WHERE id = :id
}
```

An action receives the capabilities it needs just as explicitly. The following is intentionally abbreviated structural code; the concrete typed state-changing query is shown in the CRUD and business-audit sections:

```
#[action]
fn publish(ctx: ActionContext, db: Db, id: i64)
  -> Result<Redirect, PageError> {
  let article = Article.byId(db, id)?;
  authorize article owner authorUsername or role Publisher;
  // ... state transition inside a transaction ...
  return Ok(redirect(adminArticles()));
}
```

The handler signature still reveals database access. The same principle applies to AppFs, auth, and named network egress: a domain object must not become a hidden service locator.

11.7 Why should routes remain top-level?

Routes deliberately do not disappear entirely into objects:

```
route articleShow GET "/article/:slug<Slug>"
  public cache public ttl 60
  => article::Article.show;

route articlePublish POST "/admin/article/:id<i64>/publish"
  auth role Publisher
  invalidate cache articleShow
  => article::Article.publish;
```

A route registry or code review can then see in one place:

- URL and HTTP method;
- authentication/authorization requirement;
- cache policy and invalidation;
- rate limit or resource profile;
- the global route name used in logs and audit.

A domain member name may be short and locally meaningful, such as `Article.publish` inside its own module; from a separate `routes.vrn`, use `article::Article.publish`. The route name should remain globally descriptive, such as `articlePublish`.

11.8 Recommended naming convention

Element	Form	Example
object	PascalCase business noun	Article, Invoice, UserAccount
member	camelCase, short inside domain	bySlug, publish, approve
route	globally auditable name	articleShow, invoiceApprove
module	feature or technical unit	article, account, routes

11.9 One possible CMS structure

A medium-size CMS might use:

```
main.vrn
routes.vrn
article.vrn          # Article object + base queries
article/
|-- admin.vrn        # edit/publish
`-- components.vrn   # Card, lists
media.vrn
media/
`-- admin.vrn
account.vrn
migrations/
public/
`-- app.css
```

Public and admin HTTP policy remain in the route layer. The article feature keeps the article business model and its operations together.

11.10 What not to do

Do not create one file per declaration

Overly small modules create more navigation than structure. A fifty-line feature does not need eight files.

Do not build a hidden “service” layer

If an operation uses DB, AppFs, or auth context, keep that visible in the typed signature. One of Velran’s strengths is precisely that capability boundaries are explicit.

Do not duplicate route policy inside handlers

Route-level auth, cache, rate limit, and resource policy are declarative contracts. Domain-specific object authorization naturally remains in the handler because it depends on the business record.

Do not use free-form String for business state

If a domain has `Draft`, `Review`, `Published`, or `Cancelled` states, use an enum. Project structure helps most when the domain contract is typed too.

11.11 Refactoring path: from one file to a larger application

You do not need to design the final directory hierarchy on day one. A good evolutionary path is:

1. Start with one `main.vrn` while the application is small.
2. Separate model/query/page/action groups when the file becomes hard to navigate.
3. If one feature is scattered across many technical files, switch to feature-based modules.
4. If too many global business names appear, group them under `object` namespaces.

11.12 Verified companion examples

For module structure, use `examples/module-namespaces/main.vrn`; for a generic multi-file starting point, use `examples/starter-project/main.vrn`; and for the running project in this book, use `examples/secure-notes/main.vrn`. All three are compiler-checked and therefore better copy/paste sources than shortened architectural sketches.

11.13 Practice: refactor by change locality

Exercise mode: independent. Use the independent method defined in the preface.

Pick one feature and identify every file that changes when the feature changes. If a small business change requires edits across many unrelated technical layers, consider regrouping the code around the feature or domain object. Refactor only after the dependency direction is clear.

11.14 Common mistake: modularizing by file count alone

Useful modules own coherent contracts and reduce concepts that must be held at once; avoid both giant files and artificial one-function modules.

11.15 Running project checkpoint: Secure Notes

Move Secure Notes from the initial single-file shape into a structure that keeps note-specific models, queries, pages/actions, and policies easy to find. Preserve the same externally visible routes and security behavior while improving change locality.

Part III

Applied development, integration, and assurance

Chapter 12

Practical applications: wiki and CMS

Learning path: Complete path / domain example. Use this as a richer workflow and publication-state case study.

The wiki and CMS examples combine earlier features into domain workflows. Their compiler-checked sources are `examples/wiki/main.vrn` and `examples/news-site/`; prefer those fixtures when a listing here is shortened.

12.1 A simple wiki

A minimal wiki domain consists of pages: stable identifier, slug, title, and Markdown body. The following example focuses on connecting the language elements.

Model

```
model WikiPage {
  id: i64
  slug: Slug
  title: String
  body: String
}
```

Query

```
#[query]
fn wikiBySlug(db: Db, slug: Slug)
  -> Result<WikiPage, DbError> sql {
  SELECT id, slug, title, body
  FROM wiki_pages
  WHERE slug = :slug
}
```

Public page

```
#[page]
```

```
fn wikiShow(ctx: PageContext, db: Db, slug: Slug)
  -> Result<Html, PageError> {
  let page = wikiBySlug(db, slug)?;
  return Ok(html {
    @layout(Main, page.title) {
      <article>
        <h1>{{ page.title }}</h1>
        @markdown(page.body)
      </article>
    }
  });
}

route wikiShow GET "/wiki/:slug<Slug>" public => wikiShow;
```

Editor form

```
form WikiForm {
  title<String>
  body<String>
  validate title length 2 200 body length 1 300000
}
```

The mutating route should require authentication, for example `auth role Editor`. The write query should receive a `Transaction` capability. Do not blindly regenerate and overwrite the slug from the title on every edit if preserving stable old URLs matters.

Canonical slug and aliases

A more mature wiki can maintain old names in a separate slug-alias/history table. The runtime can produce a 301 redirect from a canonical route, so the application does not need to build arbitrary redirect URLs.

What should a real wiki add?

- optimistic locking for concurrent edits;
- a business audit trail for changes;
- object-level authorization if not every page has identical access rules;
- backup/restore policy for the database and any AppFs state.

12.2 CMS and news site

The repository's `examples/news-site/` project already demonstrates a small CMS core: model, typed SQL, layout, Markdown, editor action, and public cache.

Project structure

```
main.vrn
models.vrn
queries.vrn
components.vrn
pages.vrn
actions.vrn
```

Article model

```
model Article {
  id: i64
  slug: Slug
  title: String
  body: String
}
```

Editor input gets a separate named form schema:

```
form ArticleForm {
  title<String>
  body<String>
  validate title length 2 200 body length 1 300000
}
```

Public article page

```
#[page]
fn article(ctx: PageContext, db: Db, slug: Slug)
  -> Result<Html, PageError> {
  let article = articleBySlug(db, slug)?;
  return Ok(html {
    @layout(Main, article.title) {
      <article>
        <h1>{{ article.title }}</h1>
        @markdown(article.body)
      </article>
    }
  });
}

route article GET "/article/:slug<Slug>"
  public cache public ttl 60
  => article;
```

Public cache is appropriate only for a page that does not depend on session or user state.

Editor-side creation

```
#[action]
fn create(ctx: ActionContext, db: Db,
          title: String, body: String)
  -> Result<Redirect, PageError> {
  let articleSlug = slug(title);
  transaction db {
    insertArticle(tx, articleSlug, title, body)?;
  }
  return Ok(redirect(adminArticles()));
}

route articleCreate POST "/admin/articles"
  form ArticleForm
  auth role Editor
```

```
invalidate cache article
=> create;
```

The invalidation above changes the route-level generation: it does not delete just one cache key for the new slug, but makes the previous generation of the `article` route unreachable. This is correct and predictable in a simple example; in a high-traffic CMS you should deliberately decide which public routes a given mutation really needs to invalidate.

From CMS to production system

The following additions form the backbone of a practical production CMS:

- separate validation for publication and domain invariants;
- draft/published state as an enum;
- canonical slug history;
- optimistic locking during editing;
- business audit trail;
- media library and AppFs for images;
- role-based editor/publisher policy;
- rate limits and resource profiles for expensive routes;
- backups, restore drills, and controlled deployment.

In a CMS, original Markdown should remain the source of truth; do not store raw rendered HTML produced by the application.

12.3 Server-side Markdown rendering and cache

For published content, prefer server-side Markdown rendering. The authoritative path is:

```
Markdown stored in SQL
-> typed query by stable id/slug
-> @markdown(...)
-> safe HTML response
-> optional route-level public cache
```

Keep the original Markdown in the database as the source of truth. Do not persist application-generated HTML merely to avoid rendering work. The server-side renderer rejects raw HTML and applies its link-target policy, so rendering remains inside the framework's HTML security boundary instead of moving trust into browser JavaScript.

A public, non-personalized document may use route caching when its freshness contract allows it. The cache stores the completed response; removing the cache clause gives the same rendering path without caching. A minimal pair looks like:

```
route markdownDocumentLive GET "/documents/:id<i64>"
  public
  => markdownDocument;

route markdownDocumentCached GET "/documents-cached/:id<i64>"
  public
  cache public ttl 120
  => markdownDocument;
```

The canonical end-to-end example is `examples/markdown-sql-cache/`. It includes SQLite, PostgreSQL, and MariaDB schema variants and demonstrates an ID-based typed SQL read feeding `@markdown`. Use browser-side Markdown rendering only for an editor preview when it improves interaction; the published representation should still be rendered by the server so the server remains authoritative for sanitization, cache policy, CSP assumptions, and no-JavaScript clients.

12.4 From demo CMS to editorial workflow

A production content system needs a workflow, not only CRUD. A useful boundary is to keep public rendering, editorial mutation, concurrent-edit protection, media publication, and audit as separate mechanisms that compose explicitly.

Concurrent editing

Human editors commonly keep a form open for minutes. Add a `version` column and use `Changed` on updates so an old edit receives a conflict instead of overwriting newer work. The edit page carries the current version as typed form input; the mutation checks `id + version` atomically and increments the version on success.

For important state changes such as publishing or archiving, put the mutation and business audit record in the same database transaction. The audit trail records the authenticated actor and the static business action; it is not a substitute for authorization.

```
transaction db {
  Article.publishChanged(tx, id, version)?;
  audit Article id action publish
    from article.status to ArticleStatus::Published;
}
```

A stale version stops before the audit statement, so a failed publication attempt cannot create a false success record.

Media belongs to a state machine

Do not treat a browser filename or client MIME type as a trusted path/type. Raw Upload values remain private. Browser-visible images require an explicit typed `Image` upload with `publish`; the platform stages the bytes privately, inspects supported image formats and dimensions, and publishes only after successful handler execution.

This creates a useful content rule: database metadata may refer to a published media object, but uninspected multipart bytes never become public merely because the form was accepted.

Public pages and editor previews are different cache domains

A public article route may use public caching only when its response does not depend on session or user state. Editor previews, draft pages, permission-sensitive views, and personalized navigation should not share that cache contract. Keep the public route intentionally narrow, and invalidate its generation after a mutation that changes the rendered article.

12.5 A practical CMS layering model

As the project grows, keep responsibilities separated:

1. **domain model**: article/page identifiers, status, slug and version;
2. **queries**: compiler-owned SQL and typed binds;
3. **editor boundary**: named forms, validation and authenticated routes;
4. **authorization**: role/permission plus object-level checks where required;
5. **transactional mutation**: state change, optimistic locking and business audit;
6. **presentation**: layouts, components and Markdown rendering;
7. **media**: staged inspection and explicit publication;
8. **public delivery**: canonical URL, public cache and narrow invalidation;
9. **operations**: migrations, backup/restore, monitoring and release evidence.

The layering keeps authorization, rendering, mutation, and cache behavior reviewable as the CMS grows.

12.6 CMS review questions

Before calling a content feature complete, ask:

- Can two editors overwrite each other silently?
- Is every protected state transition authorized independently of validation?
- Are publication/archive decisions reconstructable from business audit when required?
- Are old slugs redirected canonically rather than accepted as arbitrary redirect input?
- Can uninspected upload bytes become public?
- Can a session-dependent response enter public cache?
- Does a mutation invalidate the smallest correct public-cache scope?
- Can the content database and AppFs/media state be restored to a known consistent point?

12.7 Verified companion examples

Use these compiler-checked companions according to the question you are studying:

- `examples/wiki/main.vrn` – focused wiki flow;
- `examples/news-site/main.vrn` – multi-file CMS/editorial workflow;
- `examples/markdown-sql-cache/` – focused typed SQL to server-side Markdown rendering, with optional public response caching.

Together they validate that editorial workflow, typed SQL, Markdown rendering, authorization, caching, and module structure still compose under the current compiler.

12.8 Practice: define an editorial state machine

Exercise mode: independent. Use the independent method defined in the preface.

List the states of one publishable object and the actors allowed to move it between states. Include at least one rejected transition. Then decide which transitions require stronger authorization, audit evidence, or concurrency protection.

12.9 Common mistake: representing workflow as free-form text

Editorial state is business state. Model it as a closed, typed set and keep transition rules explicit. Otherwise invalid combinations gradually become data-cleanup problems instead of

compile-time or verification-time design decisions.

12.10 Running project checkpoint: Secure Notes

Use the compiler-checked publication checkpoint:

```
examples/secure-notes/checkpoints/12-publication/main.vrn
```

Extend Secure Notes with Draft and Published states. Keep publication as an explicit state transition, preserve ownership rules, and make the externally visible view depend on the typed state rather than on ad-hoc string comparisons.

Chapter 13

External services and egress policy

Learning path: Complete path / integration chapter. Read before adding outbound network integrations.

Security invariant: egress is an explicit capability

Application code does not gain arbitrary network access. Outbound communication crosses a named, configured capability boundary so destinations, credentials, limits, and audit policy remain reviewable.

Compiler diagnostic: SEC-EFFECT-002

Meaning: A handler performs a network effect without declaring the matching integration capability in `uses`.

Fix: Declare the named integration capability explicitly in the handler signature; do not replace it with ambient or raw network access.

Outbound HTTPS is a separate trust boundary because the application initiates the connection. Velran therefore exposes it as an explicit capability, not ambient network access.

13.1 The current Velran boundary

Velran intentionally has no unrestricted `http.get(userUrl)` style API. Instead, source declares a named integration and the handler explicitly receives the capability:

```
integration Billing {
    egress payments
}

#[action]
fn charge(
    ctx: ActionContext,
    amount: i64
) -> Result<Json, PageError>
    uses Billing
{
    let status = Billing.postJson("/v1/charges", amount)?;
    return Ok(json(status));
}
```

The declaration exposes only a trusted *name*. Host, IP, port, TLS and network ranges are not source-code authority. The compiler records the call as the `net.payments` effect; a handler cannot perform the call without the matching `uses Billing` capability.

The safe-path rule is therefore:

```
Velran business intent
    |
    v
named integration capability
    |
    v
trusted operator egress policy
    |
    v
bounded HTTPS transport
```

13.2 Why arbitrary URLs are intentionally unrepresentable

A user-controlled fetch target creates classic SSRF risk. Examples that must never become ordinary application destinations include loopback, link-local metadata services and internal infrastructure:

```
http://127.0.0.1/...
http://169.254.169.254/...
http://internal-db.example/...
http://[::1]/...
```

Velran source can choose a relative path within a declared integration, but not the scheme, host, IP or port. Paths must be compiler-known rooted relative literals; absolute, protocol-relative and dynamic destination URLs are rejected.

13.3 Named target: configuring the network authority

The trusted egress policy binds the Velran target name to concrete transport authority. A target may look like:

```
[[target]]
name = "payments"
hosts = ["api.payments.example"]
cidrs = ["203.0.113.0/24", "2001:db8:1200::/48"]
ports = [443]
tls_required = true
max_dns_answers = 8
max_sent_bytes = 262144
max_received_bytes = 2097152
connect_timeout_ms = 5000
total_timeout_ms = 15000
```

For the Velran typed call surface the startup policy must resolve the target unambiguously to the supported endpoint shape. An application using an outbound effect does not start if the required egress policy is missing or inconsistent.

13.4 Policy checks multiple layers

Egress policy is intentionally stronger than a hostname allowlist.

Host, port and TLS

The destination host and port come from trusted policy, and TLS identity verification uses the configured hostname. Application input cannot weaken TLS or choose a different port.

DNS and CIDR

Every A and AAAA answer is checked against the allowed network ranges. If any answer violates policy, the resolution is rejected fail closed. This protects against DNS rebinding and accidental split-horizon exposure.

Peer verification after connect

After TCP connection establishment, the actual peer IP is checked again. The selected address must still satisfy CIDR policy; DNS validation is not treated as permanent proof.

13.5 IPv4 and IPv6 use the same authority model

The policy is dual-stack. IPv4-mapped IPv6 addresses are normalized before policy evaluation so an IPv4 restriction cannot be bypassed through an IPv6 representation. Loopback, link-local, ULA, multicast and unspecified ranges receive no implicit trust.

13.6 Typed outbound operations

The first Velran call surface deliberately stays small:

```
let status = Billing.get("/v1/status"?);
let status = Billing.postJson("/v1/charges", amount)?;
```

Important constraints are part of the security model:

- page handlers may perform outbound GET but not outbound POST side effects;
- JSON POST is an action capability;
- generic outbound JSON cannot receive **Secret**, **Sensitive**, credential-purpose or upload values;
- server-side egress does not grant browser network authority – generated CSP keeps **connect-src** **'none'** unless a separately designed browser capability exists;
- the current typed call returns the HTTP status code, not an automatically trusted response body.

The last point is deliberate. External response data would have to enter as untrusted data and cross a typed validation/refinement boundary; the language does not silently convert arbitrary upstream JSON into trusted domain values.

13.7 Size, protocol and time limits

An upstream service may be broken or hostile. The transport therefore bounds DNS answers, request bytes, response bytes and time. In addition to operator target limits, the Velran status-

call surface applies its own hard upstream-body ceiling and rejects unsupported transfer/content encodings.

The complete request also participates in the runtime cumulative external-I/O budget. This prevents several individually legal network operations from composing into one unbounded request.

13.8 Secrets for external services

External credentials remain behind a dedicated secret-store boundary, for example:

```
/run/secrets/velran/  
payments-bearer
```

Secret names cannot become arbitrary filesystem paths. Credentials must never be written to response bodies, ordinary logs, security events or audit payloads. Velran’s typed `Secret<T>` rules and the trusted Rust secret store reinforce the same boundary at different layers.

13.9 Retries, idempotency and exceptional outcomes

Automatic retry of a side-effecting POST is intentionally not a default. Critical retry-sensitive application operations should use the Velran idempotency contract rather than ad-hoc retry loops.

For critical database work, transaction outcome is explicit:

```
let outcome = transaction db {  
    charge(tx)?;  
};  
  
match outcome {  
    Committed => { return Ok(json(true)); }  
    RolledBack => { fail conflict; }  
    CommitUnknown => { fail conflict; }  
}
```

An idempotent critical transaction must capture and exhaustively handle this outcome. `CommitUnknown` must not silently become “retry the POST”, because the side effect may already have committed.

13.10 Webhooks are the inverse boundary

An outbound integration and an inbound webhook solve opposite problems. A webhook route uses a declared verification policy, raw-body signature verification, timestamp/replay-window checks and an atomic replay claim before the handler receives authority. Do not model a webhook as merely a public JSON POST.

13.11 Synchronous request or background work?

Velran currently has no first-class background-job subsystem. A short, bounded integration can remain in the request/response cycle. A long or reliably retryable workflow should wait for

an explicit job/outbox design rather than being emulated with ad-hoc threads, subprocesses or unbounded request time.

13.12 Minimum testing

Prove at least:

- unknown integration or missing **uses** capability is rejected at compile time;
- dynamic and absolute destination paths are rejected;
- page-side POST is rejected;
- allowed host + allowed IPv4 and IPv6 work;
- mixed DNS answers with one forbidden address fail closed;
- forbidden port, TLS identity mismatch and peer-IP mismatch fail closed;
- request/response byte limits and timeouts are enforced;
- unsupported upstream transfer/content encoding is rejected;
- cumulative external-I/O exhaustion becomes a resource-limit failure;
- secrets cannot flow into generic outbound JSON.

13.13 The essence of this chapter

External HTTPS is a **named, typed, bounded capability**; trusted deployment policy owns the destination. SSRF resistance, TLS policy and resource limits therefore remain part of the secure path.

13.14 Verified companion example

The minimal native outbound-capability example is `examples/outbound-native/app.vrn`. It is intentionally small: the important contract is that source names an integration capability while trusted deployment policy owns the actual destination authority.

13.15 Practice: write an egress contract

Exercise mode: independent. Use the independent method defined in the preface.

For one external service, record the destination identity, allowed operation, timeout, retry policy, credential source, expected response shape, and failure behavior. Treat each of these as part of the integration contract rather than as incidental client configuration.

13.16 Common mistake: allowing arbitrary outbound destinations

An integration should not turn user-controlled input into an arbitrary network destination. Keep egress named, bounded, and reviewable so SSRF-style behavior cannot emerge from a generic HTTP helper.

13.17 Running project checkpoint: Secure Notes

Add one optional named integration, such as sending a publication notification. Keep it off the core persistence path: a failed external call must have an explicitly designed effect rather than leaving the note in an ambiguous half-updated state.

Chapter 14

Testing, error models, and developer diagnostics

Learning path: Fast path: core before shipping. Turn compiler, verifier, and runtime failures into a repeatable debugging workflow.

Security invariant: prove rejection too

A security rule is not demonstrated only by a valid program succeeding. Keep negative tests that show forbidden variants are rejected, and treat an unexpected acceptance as a regression.

Compilation proves source and policy constraints, not business correctness or environment health. Testing must therefore cover the application state, real dependencies, migrations, authorization expectations, and deployed topology.

14.1 Read a security diagnostic in four passes

When the compiler or verifier rejects a program with a stable security code, do not start by looking for a way around the check. Read the diagnostic in four passes: identify the code, name the violated invariant, find the authority or data boundary involved, then make the smallest source change that restores that invariant. Re-run the check and keep the rejected variant as a negative test when the rule is security-significant.

A useful debugging note therefore records four things: *code*, *boundary*, *why the program was rejected*, and *what evidence makes the corrected program acceptable*. This is more durable than memorizing message wording, and it teaches the same reasoning used during security review.

Testing in Velran is therefore best treated as a set of layers that build on one another:

1. **source check:** is the program valid in the language and policy sense;
2. **negative compile test:** is a dangerous or forbidden program actually rejected;
3. **application/database test:** does the business operation create the expected state;
4. **integration test:** do the real DB, Redis, LDAP, TLS, AppFs, or other dependencies work together correctly;
5. **system/smoke test:** does the installed system work over HTTP with the real configuration.

The goal is not for every test to prove the same thing, but for every class of failure to break at the cheapest possible layer.

14.2 The first gate: `velran-cli` check

The fastest development check is:

```
velran-cli check main.vrn
```

On success, the CLI summarizes the recognized program, for example the number of models, queries, pages, actions, and routes. The check does not start an HTTP server or mutate a database; it validates the source contract.

Among other things, the compiler can catch here:

- type mismatches;
- mismatches between query result shape and `model`;
- invalid route or form declarations;
- forbidden HTML/script constructions;
- unsafe SQL construction or bind errors;
- incompatible auth/cache/policy combinations;
- certain violations of capability boundaries.

This is one of the key differences from a general-purpose scripting approach: a significant class of errors should be found before any HTTP request is made.

14.3 A negative test is as valuable as a positive test

For a security invariant, it is not enough to test that the correct program compiles. You must also prove that the forbidden program *does not* compile.

The repository release checks therefore run fixtures such as:

```
# This must compile successfully:
velran-cli check examples/auth/app.vrn

# This must fail:
velran-cli check \
  tests/fixtures/security/sql-injection-rejected.vrn
```

Typical negative tests cover:

- dynamic SQL suitable for SQL injection;
- raw/script-like XSS attempts;
- optional values used without a presence check;
- invalid form/schema validation;
- user-dependent routes with public cache;
- public routes using object-level authorization guards;
- forbidden upload paths or mixed JSON/form body modes.

A security rule is strong when the project contains **both positive and negative evidence** for it.

A useful repository convention keeps those roles visible. Learner-facing, runnable applications stay under `examples/`; deliberately rejected compiler/security inputs live under `tests/fixtures/`; and lists used by verification tooling live under `tests/manifests/`. A positive example may still participate in `verify.sh`, but it should remain worth reading as

application code. A rejection fixture does not need to look like a good application; its job is to isolate one invariant and fail for the intended reason.

14.4 Four different worlds of failure

When debugging, first decide which layer failed. The same user-visible symptom may require a completely different fix depending on the layer.

Compile-time error The source contract is invalid. `velran-cli check` or the build stops. Fix the code.

Startup/config error The program is valid, but server configuration, a secret, TLS key, or hard policy is invalid. Use `-check-config`, server logs, and effective config.

Request/policy error The server runs, but a request is rejected by input, authentication, authorization, rate-limit, or content-negotiation policy. Use the HTTP status, request ID, access log, and audit log.

Dependency/runtime error DB, Redis, LDAP, filesystem, network, or a resource limit causes the failure. Use readiness, metrics, server logs, and real integration tests.

A bad debugging reflex is to start rewriting source code for every 500/503 symptom. Identify the layer first.

14.5 Test configuration separately

Source checking does not replace server-configuration validation:

```
velran-server \  
  --config /usr/local/etc/velran/server.toml \  
  --check-config
```

To inspect the fully resolved values before deployment:

```
velran-server \  
  --config /usr/local/etc/velran/server.toml \  
  --print-effective-config
```

`-check-config` is not a dependency integration test. A syntactically valid DB URL can still point to an unavailable database; that is why readiness and deployment smoke tests exist separately.

14.6 Migration: status, verify, apply, verify

When the database changes, migration state is part of the test process:

```
velran-cli migrate status \  
  --dir migrations \  
  --db-url-file /run/secrets/velran/migration-db-url  
  
velran-cli migrate verify \  
  --dir migrations \  
  --db-url-file /run/secrets/velran/migration-db-url
```

A good deployment order is:

```

source/config check
-> migration verify
-> migration apply
-> migration verify
-> application rollout
-> readiness
-> smoke test

```

Do not modify an already-applied migration merely to make a test green. Put new changes into a new migration.

14.7 Business tests: prove state, not implementation

A CRUD or CMS action test should not primarily ask whether a certain internal function was called. Assert the externally observable state.

For article publishing, for example:

1. create a draft article;
2. authenticate with the Publisher role;
3. submit the publish action;
4. verify the redirect;
5. read the article back and verify the `Published` state;
6. verify the business-audit record;
7. verify that the public page now shows the new state;
8. for a cached route, verify invalidation too.

This test remains valuable even if queries or module structure are later reorganized internally.

14.8 Database tests: SQLite does not prove everything

A separate temporary SQLite database is convenient for fast development tests. But if production uses PostgreSQL or MariaDB, test against the real backend at least in the release/integration suite.

Particularly sensitive areas include:

- transaction behavior;
- constraints and unique conflicts;
- concurrent mutation and optimistic locking;
- indexes/query plans;
- migration DDL;
- timeouts and connection availability.

A backend-independent typed query contract rules out many mistakes, but it does not make backend behavior identical.

14.9 Authentication and authorization test matrix

For a protected action, test at least:

Case	Expected result
------	-----------------

No session	login/auth rejection
Valid user, missing required role	authorization deny
Required role present	successful operation
MFA-required route, no MFA	MFA rejection
Expired or revoked session	new authentication required
Invalid CSRF token on state change	request rejected
Object not owned by user	object-authorization deny
Admin/allowed actor	successful operation

When using LDAP/LDAPS or Redis, include real dependency integration tests. A mock does not prove TLS hostname validation, network timeout behavior, or Redis fail-closed semantics.

14.10 File, media, API, and cache tests

For these features, happy-path coverage is especially insufficient.

Upload and media

Test at least:

- a valid small file;
- byte-limit overflow;
- image pixel-limit overflow;
- malicious/invalid filename;
- path traversal or symlink escape rejection;
- interrupted-upload cleanup.

JSON/CORS

In addition to valid JSON, test incorrect content type, duplicate keys, unknown fields, **Accept** negotiation, preflight, and the interaction of CSRF and CORS for credentialed state changes.

Cache and rate limiting

Test that user-dependent pages cannot become public cache, that mutations invalidate the right cache generation, that shared state is actually shared across instances, and that Redis failure follows the chosen security fail-closed behavior.

14.11 An HTTP error code is not the same thing as a program bug

The client-visible status is part of the contract. Typical cases are:

Situation	HTTP
Malformed input or request-contract violation	400
No valid authentication on JSON/API route	401
Actor/policy does not allow the operation	403
Resource not found	404
Wrong HTTP method	405
Representation not acceptable	406

Concurrent modification / zero rows for Changed	409
Request/upload too large	413
Wrong or missing body content type	415
Host/trust-boundary mismatch	421
Named-form field error with rerender	422
HTTPS required	426
Rate limit	429
Request header too large	431
Internal unclassified server failure	500
Dependency or request resource temporarily unavailable	503

Never send SQL, secrets, physical filesystem paths, session tokens, or stack traces to the client. Detailed diagnosis stays in server-side logs and is correlated by request ID.

Status and representation are separate contracts

The same business situation may have a different representation on HTML and JSON surfaces. This is not inconsistency: status expresses the error category, while representation expresses the client-facing surface contract.

Situation	HTML route	JSON route
No session	303 See Other to built-in login page	401 + JSON error envelope
Authorization denied	403	403 + JSON error envelope
Optimistic-lock conflict	409 conflict page, no-store	409 + JSON error=conflict
Named-form validation	422, form rerendered with field errors	not a JSON body mode; body contracts are exclusive
Record not found	404	404 + JSON error envelope

A successful mutating HTML action typically returns 303 **See Other** for Post/Redirect/Get. A 303 is therefore not an error; it is part of browser control flow.

14.12 Test exhaustive exceptional states

Velran closed sum types make security-relevant exceptional states testable at compile time. A **match** has no catch-all wildcard, so adding a new enum/sum variant forces every affected match site to be reviewed.

For critical transactions, include tests for all three outcomes:

- **Committed**: the success path is returned and replay state is stable;
- **RolledBack**: the operation did not take effect and the public result is safe;
- **CommitUnknown**: no automatic side-effect retry occurs and the caller receives an explicitly handled safe outcome.

Negative compiler tests should also prove that an idempotent critical transaction without outcome capture, or without exhaustive matching, is rejected.

14.13 Debugging with request IDs

When an HTTP request fails, the best first diagnostic key is the request ID generated by the server. Starting from the client response or access log:

```
grep 'velran-...' /var/log/velran/access.log
grep 'velran-...' /var/log/velran/server.log
grep 'velran-...' /var/log/velran/audit.log
```

The three logs answer different questions:

- access: what arrived and what status ended the request;
- server: what technical/runtime event occurred;
- audit: whether a security or user-policy decision occurred.

Preserve this diagnostic chain in automated test failure output too.

14.14 Deployment smoke test

A green CI run does not prove that the freshly installed service actually started. After rollout, check at least:

```
curl --fail --silent \
  https://example.test/health/live

curl --fail --silent \
  https://example.test/health/ready

curl --fail --silent \
  https://example.test/
```

Keep smoke tests small and non-destructive. If you also test a state-changing endpoint, use isolated test data and idempotent cleanup; do not experiment on production content.

14.15 What does the repository `verify.sh` mean?

Inside the Velran source tree, the full release-evidence entry point is:

```
./verify.sh
```

It combines source checks with Rust workspace tests, learner-facing positive programs from `examples/`, rejection/security fixtures from `tests/fixtures/`, manifest-driven corpus checks from `tests/manifests/`, migration/auth/storage/runtime checks, and release-artifact invariants.

If you modify the Velran implementation itself, this is the release minimum. If you only develop your own `.vrn` application, the daily loop can be smaller, but your release process should still include source check, migration/config validation, relevant integration tests, and post-deployment smoke testing.

14.16 Recommended development cycle

A good baseline for everyday feature development is:

```
1. design domain/model change
2. write migration if needed
3. velran-cli check main.vrn
4. fast application test
5. negative/security cases
6. migration verify
7. relevant integration test
8. velran-server --config server.toml --check-config
9. deploy to staging
10. readiness + HTTP smoke
11. production rollout
12. readiness + smoke + metrics/log review
```

The principle is that fast deterministic failures should happen early; expensive environment-dependent tests should run only after them.

14.17 Debugging decision order

When “it does not work,” proceed consistently:

1. Does it compile? Run `velran-cli check`.
2. Is the config valid? Run `-check-config`.
3. Did the service start? Inspect service and server logs.
4. Is the process alive? Check liveness.
5. Are dependencies reachable? Check readiness.
6. Does the request reach the route? Inspect access log and request ID.
7. Is policy rejecting it? Inspect audit log and trust-boundary status (for example forwarding 400, 401/403/421/426/429).
8. Is runtime/dependency failing? Inspect server log and metrics.
9. Is the business result wrong? Reproduce with isolated test data and inspect persistent state.

This is much faster than simultaneously changing code, proxy configuration, and database state.

14.18 Minimum application release checklist

- `velran-cli check` is green;
- migration history verify is green;
- server config check is green;
- positive and negative tests for critical business flows are green;
- auth/authorization/security regression cases are green;
- integration tests against production-equivalent DB/Redis/TLS are green;
- backup/restore readiness is known;
- staging readiness and smoke are green;
- after rollout, readiness, smoke, logs, and metrics are reviewed.

The goal is repeatable evidence for the important contracts: **what we allow, what we deny, what state we create, and how dependency failure behaves.**

14.19 Practice: derive negative tests from boundaries

Exercise mode: independent. Use the independent method defined in the preface.

For every trusted transition in a feature, write at least one test where the transition must be rejected: malformed input, missing authorization, stale version, exhausted budget, or unavailable dependency. Negative tests are most valuable when they prove the boundary itself rather than only an error string.

14.20 Common mistake: testing only successful handlers

A security-first framework earns much of its value from what it refuses to execute. Keep compile-time rejection, verification failure, and runtime failure tests separate so a regression cannot silently move a protection to a later and weaker phase.

14.21 Running project checkpoint: Secure Notes

Build a compact test matrix for create, view, edit, publish, and delete. Include another user's identifier, stale edit version, invalid input bounds, and an integration failure. The project should now have evidence for both expected success and expected refusal.

Chapter 15

Security hardening: the current secure-by-construction model

Learning path: Fast path: mandatory before shipping. Consolidate the security invariants that must hold across the whole application.

Velran security is not optional middleware: authority is divided across language, compiler, runtime and server. Make security properties compiler invariants where possible; otherwise enforce a secure platform default or fail closed with a stable diagnostic.

First read. Focus on authority/access control, input/browser boundaries, resource safety, and the production security gate. Use the remaining sections as a hardening reference.

15.1 What the secure model does – and does not – decide

Velran can make raw SQL, arbitrary redirect URLs, ambient DB/network authority, unbounded list queries, forged redaction and several other unsafe states unrepresentable. It can prove that a mutation used an authorization proof. It cannot decide whether “Editor may publish this particular article” is the correct business rule. That decision remains in the application’s permission/object/tenant policy.

A useful review rule is: *mechanism should be compiler/runtime-owned; business intent should remain explicit application code.*

15.2 Authority and access control

Every route has an explicit access policy. There is no implicit public endpoint. Authorization is layered:

1. trusted authentication establishes the principal;
2. route policy determines whether the handler may be entered;
3. object authorization proves access to a loaded record;
4. mutation authorization proves that an UPDATE/DELETE key comes from authorized object authority;
5. named permissions and MFA can be part of a handler contract;
6. tenant-scoped data requires active-tenant membership proof;
7. effects/capabilities make DB and outbound-network authority visible rather than ambient.

Validation proof is never authorization proof. A valid `ArticleId` still does not prove that the current principal may mutate that article.

15.3 Critical operations, idempotency and transaction outcome

A critical contract combines security obligations rather than scattering them through boilerplate:

```
critical Payment {
  permission BillingWrite
  mfa
  transaction
  idempotency
  audit PaymentCharged
}
```

Idempotency binds the replay key to both request fingerprint and principal scope. Reusing the key with a different payload becomes a conflict instead of a second side effect.

Database commit acknowledgement is also explicit. A captured transaction returns the closed sum type `Committed | RolledBack | CommitUnknown`, and `match` is exhaustive with no wildcard fallthrough. An idempotent critical transaction must capture and handle the outcome; `CommitUnknown` cannot disappear into a generic retryable DB error.

15.4 Input, injection and browser boundaries

The major injection surfaces are structural rather than convention-based:

- SQL uses compiler-owned statements and typed binds; string-interpolated raw SQL is rejected;
- HTML/template output is structured and sensitivity-aware;
- redirects use typed local route calls rather than string URLs;
- response headers use typed names and bounded validated values;
- filenames, media types and Content-Disposition are typed;
- outbound calls use named integrations and relative literal paths, not arbitrary URLs;
- generated CSP defaults to least privilege, including `connect-src 'none'`;
- compressed inbound HTTP bodies are rejected until a future bounded decompression surface is explicitly designed.

15.5 Trust, domain types and disclosure

Request-boundary values begin untrusted. Domain validation can refine them to validated nominal types, but this does not erase sensitivity or create authorization.

`Secret<T>` and `Sensitive<T>` metadata follows values through calls. Full database/domain models are not generically serialized; public output uses explicit projections. This prevents a newly added model field from silently appearing in an existing API.

For logs and monitoring, `Redacted<T>` cannot be forged with a type annotation. It is produced only by the trusted `redact(...)` builtin, which discards the original representation and yields a safe redacted value.

15.6 Authentication and session hardening

The browser session is platform-owned. Login rotates session authority, logout invalidates it, and auth-generation changes revoke older sessions after security-relevant account changes. Cookies use the hardened platform policy rather than application-selected flags.

Online authentication abuse is limited in three dimensions:

- source + principal;
- principal across sources;
- source across principals.

MFA/recovery attempts have a stricter stage limit. Limiter-store failure is fail closed, and client-visible credential errors remain deliberately generic to reduce account enumeration.

15.7 Credentials, tokens and cryptography

Purpose-specific types prevent accidental cross-use of passwords, token hashes, session tokens, reset tokens, CSRF tokens and cryptographic keys.

Passwords use platform-owned Argon2id policy. Bearer tokens are persisted as purpose-specific hashes and use expiry-aware verification; reset grants, session revocation and session rotation have atomic lifecycle contracts.

Cryptographic keys have purpose and lifecycle. For user-data encryption the high-level primitive is fixed to AES-256-GCM with a platform-generated nonce and versioned authenticated envelope. The lifecycle is:

Active	-> encrypt + decrypt
Retiring	-> decrypt only
Retired	-> neither

Application code cannot select a weaker cipher or supply its own nonce.

15.8 Verified webhooks and safe uploads

A webhook is not an unauthenticated public POST. The route has separate webhook authority: raw-body signature verification, timestamp/replay-window checking and an atomic replay claim happen before the handler runs.

File upload follows a state machine:

multipart bytes	
	-> private staged file
	-> byte-authoritative inspection
	-> verified image
	-> handler success
	-> atomic publish

Client filename and MIME are metadata, not storage or type authority. Public media serving only exposes directories declared by verified image publish routes.

15.9 Resource safety is a security invariant

Velran bounds both individual operations and the whole request:

- request/parser hard limits;
- route resource profiles under a platform ceiling;
- instruction/allocation budgets and hard request deadline;
- static row bounds for database list queries plus runtime row caps;
- bounded request collections;
- upload byte/pixel limits;
- bounded outbound request and response bodies;
- cumulative external-I/O accounting across request and outbound traffic.

General unrestricted collection growth is intentionally not exposed until the same cardinality guarantees can be preserved.

15.10 Production configuration is a security contract

An application can declare strict production requirements:

```
production {  
    https required;  
    debug disabled;  
    hsts required;  
    database tls required;  
}
```

Startup fails closed when the deployment cannot satisfy the contract. Production mode rejects insecure development cookies, unsafe source reload, missing-Origin exceptions and insecure remote database transport. Reload cannot silently change the deployment security contract; changing it requires restart and operator review.

15.11 Supply-chain state is part of the release

The release records more than crate versions. Three artifacts belong together:

```
Cargo.lock  
SUPPLY-CHAIN-CAPABILITIES.txt  
VELRAN-SUPPLY-CHAIN.lock
```

Direct external dependency edges have explicit security capabilities such as network, filesystem, crypto or parser. Current provenance policy accepts checksummed crates.io packages only. The supply-chain envelope seals both the Cargo lock and capability policy, and release tooling verifies `cargo metadata --locked` before sealing or packaging.

15.12 Security events, redaction and alerts

Critical operations already emit typed audit events. The platform also emits structured security events for authentication abuse, MFA failures, access-policy denial, CSRF/origin/CORS rejection, webhook verification failure, idempotency conflict and resource exhaustion.

The event schema does not accept arbitrary request payloads. Principal/source values are redacted in emitted events; bounded in-memory correlation may use raw keys without writing them to logs. Platform-owned burst thresholds and cooldowns turn repeated failures into alerts without allowing alert floods to become a new availability problem.

15.13 Production security gate

Before handing a release to production, verify at least:

1. the real toolchain gates pass: format, locked metadata, check, tests and `./verify.sh`;
2. production policy is present where required and startup compatibility is green;
3. HTTPS, HSTS, secure cookie and trusted-proxy topology are correct;
4. DB runtime credentials are least privilege and remote DB TLS is enforced;
5. auth/session/rate-limit shared state matches the deployment topology;
6. tenant, object and mutation authorization negative tests are green;
7. critical/idempotent operations handle `CommitUnknown` explicitly;
8. webhook verification/replay and upload publish-state tests are green;
9. outbound integrations resolve only through reviewed named egress policy;
10. resource/deadline limits are configured and observed;
11. security-event redaction and burst alerts are verified;
12. supply-chain and release-manifest state are current and verified.

15.14 Review questions for every new feature

Ask five questions:

1. **What comes from outside?** Which fields, files, identifiers or upstream data are untrusted?
2. **Where does it become typed or validated?** Which boundary creates the proof?
3. **Who owns authority?** Session, database record, tenant membership, trusted config or named capability?
4. **What happens in ambiguous failure?** Is there a fail-closed, rollback, cleanup or explicit unknown state?
5. **What bounds the resource?** Bytes, rows, items, time, rate, CPU or cumulative I/O?

If any answer is vague, the feature's security model is unfinished.

15.15 Verified companion examples

Security hardening is easier to review against executable fixtures. The canonical companions here are `examples/resource-profile/app.vrn`, `examples/rate-limit/app.vrn`, and `examples/typed-security-events/main.vrn`.

15.16 Practice: map controls to threat paths

Exercise mode: independent. Use the independent method defined in the preface.

Choose three concrete threats and trace which independent controls stop or limit each one. Prefer layered answers: typed input plus authorization plus transaction invariants is stronger than a single validation check expected to do everything.

15.17 Common mistake: treating hardening as a final deployment phase

Hardening that depends on application structure must be designed before production. Route policy, capability boundaries, object authorization, bounded inputs, and named egress are architecture, not post-release configuration polish.

15.18 Running project checkpoint: Secure Notes

Review Secure Notes as an attacker would: identifiers, form fields, publication transitions, media references, JSON endpoints, and outbound integration. Document which boundary rejects each invalid path and which event should be observable when the rejection matters operationally.

Part IV

Production and operations

Chapter 16

Cache, Redis, rate limiting, and multi-instance operation

Learning path: Complete path / scale-up chapter. Read when measurement shows that caching, Redis, or horizontal scaling is needed.

First read. Learn what may be cached, how cache generations invalidate safely, and how shared sessions/rate limits fail closed. Treat backend tuning and topology details as reference.

With multiple instances, session state, rate limits, and public cache need explicit ownership. Velran keeps Redis runtime-owned: application code declares policy but receives no general key/value API. Business source of truth remains in DB/AppFs.

16.1 Why not a general Redis client?

V1 has no language operation that lets a program read or write arbitrary Redis keys. Redis is compiler/runtime-owned infrastructure:

Use	Who owns the key and rules?
session	runtime/auth layer
TOTP replay	runtime/auth layer
login rate limit	runtime/auth layer
route rate limit	runtime + trusted policy config
public page cache	compiler/runtime
business data	not Redis; application DB/AppFs

Do not store wiki articles, CMS workflow state, or billing records in Redis. Those are business source of truth and belong in the application DB accessed through typed SQL. After Redis loss, the system should rebuild from durable state: sessions may disappear, cache may rebuild, and rate-limit windows may restart.

16.2 Production Redis configuration

The Redis URL is a secret and should come from a file:

```
[redis]
url_file = "/run/secrets/velran/redis-url"
```

```
allow_insecure = false
```

The file may contain a TLS URL such as:

```
rediss://velran:SECRET@redis.internal.example/0
```

Keep `allow_insecure = false` as the production baseline. Network reachability to Redis is not an authorization model by itself: also restrict it with network ACL/firewall, and make credentials readable only by the Velran service user.

The runtime uses separate namespaces for separate purposes. Public cache, for example, lives under an `velran-cache` namespace; the application does not choose arbitrary key prefixes.

16.3 Shared sessions across server instances

An in-memory session store is a development convenience, not a multi-instance production model:

```

      +--> Velran A --+
Browser -> proxy |      |--> Redis
      +--> Velran B --+

```

If the session lived only in A, a request routed to B would lose authentication. Use a shared session backend; sticky sessions are not a failover or rollout strategy.

In production, Redis shares auth state including:

- sessions;
- TOTP replay protection;
- login rate-limit state.

The same browser therefore does not need to hit the same server instance every time.

The important local-auth exception

Sessions can be shared, but the V1 local-auth identity store is SQLite and targets single-node identity. For multiple application instances, the current shared identity backend is LDAP/LDAPS. Do not confuse these two layers:

Redis session tells the server which trusted principal and auth state an already-authenticated request belongs to;

identity backend verifies credentials and account/role state during login.

Multiple instances still need a common identity source even with shared Redis sessions; in V1, LDAP is the production path for that.

16.4 Public page cache

Public cache avoids repeating the same query and HTML render for every request to a public, user-independent GET page.

Declaration:

```
route frontPage GET "/"
  public cache public ttl 60
  => frontPage;
```

The developer declares only the caching intent and route TTL. The runtime constructs a canonical hashed key from, among other things, the route name, route generation, path, sorted typed query input, and representation.

Application code therefore cannot do this:

```
cache.set("homepage:" + userInput, rendered)
```

There is no general cache API. This prevents key-naming drift, accidental PII in keys, and a proliferation of incompatible invalidation strategies.

16.5 What may go into public cache?

The name `cache public` is literal. Only GET routes whose responses do not depend on user session or request-specific security state should be cached.

The compiler forbids dependencies such as:

```
csrfToken
authPrincipal
authMfaVerified
```

This is critical because a successful cached response may also be public-cacheable at the HTTP layer:

```
Cache-Control: public, max-age=<ttl>
```

Account pages, admin interfaces, personalized navigation, and pages consuming flash messages therefore do not belong in public cache.

16.6 Operator-controlled cache boundaries

A route cannot request an arbitrarily large TTL. Trusted server configuration sets hard ceilings:

```
[cache]
max_ttl_secs = 3600
max_entries = 10000
max_bytes = 67108864
allow_memory = false
```

`max_ttl_secs` caps any route-requested TTL. Entry and byte limits bound the in-memory development fallback; memory cache is not a production scaling strategy.

If the application uses public cache but Redis is not configured, the server does not start under the production default. Memory cache can be enabled explicitly for development, but do not let that escape into production unnoticed.

16.7 Cache invalidation: generations, not key search

A state-changing route can declare which cached routes become invalid:

```
route publish POST "/publish"  
  public invalidate cache frontPage article  
  => publish;
```

Invalidation does not run Redis `SCAN` and does not delete a discovered key list. The runtime increments a route generation. New requests then build keys from the new generation, making old entries unreachable to new traffic.

This has two important consequences:

- invalidation is effectively $O(1)$, independent of how many old cache keys exist;
- with shared Redis, every instance observes the same new generation, so invalidation is cluster-safe.

Old entries may remain physically in Redis until TTL expiry, but logically they are no longer part of the active cache namespace.

16.8 Stampede protection and its limits

After a cache miss, many identical requests may arrive at once. On a single instance, Velran uses single-flight-like behavior so only one request rebuilds a given cache key; others wait and then read cache again.

The boundary matters: V1 has no distributed single-flight lease. Two different Velran instances may rebuild the same expired key concurrently. Therefore a cacheable page query must still be reasonably cheap and well indexed. Cache is not an excuse for an uncontrolled expensive database operation.

16.9 Redis failure during cached requests

A public-cache backend failure must not silently switch the application to a different consistency model. When the cache backend is unavailable, Velran may return a service failure such as `cache_unavailable` for cached routes instead of implicitly allowing every request to fall through to the database without bound.

This fail-closed production behavior is useful: a Redis outage should not silently trigger a DB thundering herd. Readiness can return 503 for DB/Redis dependency failures so the load balancer can remove the instance from traffic.

16.10 Route rate limiting

A route does not embed numeric limits in source; it requests a trusted policy by name:

```
route api GET "/api/status"  
  public rate publicApi  
  => api;
```

Concrete numbers live in operator configuration:

```
[policy.publicApi]
limit = 120
window_secs = 60
scope = "ip_route"
```

This is a deliberate separation. The application can say that an endpoint needs the `publicApi` policy, but cannot arbitrarily override production resource and abuse policy.

16.11 Rate-limit scopes

V1 scopes are:

Scope	Bucket subject
<code>ip</code>	effective client IP
<code>route</code>	one shared route bucket for all clients
<code>ip_route</code>	effective client IP + route
<code>user</code>	authenticated canonical principal
<code>user_route</code>	canonical principal + route

`user` and `user_route` on a public route are startup errors because there is no trusted authenticated principal from which to derive the bucket.

For IP scopes, Velran does not blindly trust `X-Forwarded-For`. It first determines the effective client IP using trusted-proxy policy. Correct proxy-CIDR configuration is therefore also a rate-limit security concern behind Apache/Nginx.

16.12 Fixed-window model across instances

The route limiter uses fixed-window buckets. In Redis, all instances atomically increment the same bucket, so the limit is global rather than per instance.

For example with two servers:

```
limit = 120 / 60 sec

not: 120 requests on A + 120 requests on B
but: 120 requests in the shared bucket
```

The subject appears in the Redis key only as a stable hash, so raw usernames or IP addresses are not embedded directly as keys.

When the limit is exceeded:

```
HTTP/1.1 429 Too Many Requests
Retry-After: 60
```

If the Redis limiter fails, policy fails closed:

```
503 rate_limiter_unavailable
```

This is safer than automatically opening the endpoint without limits during a Redis failure.

16.13 Memory backends are development exceptions

The trusted production baseline is:

```
[rate_limit]
policies_file = "/usr/local/etc/velran/rate-limits.toml"
allow_memory = false

[cache]
allow_memory = false
```

Memory rate limiting and memory cache are explicit development escape hatches. With multiple processes or hosts, each instance would see separate memory and neither limits nor cache invalidation would have the same global semantics as Redis.

If you test locally with memory backends, always validate the production configuration before deployment:

```
velran-server --config /usr/local/etc/velran/server.toml --check-config
```

16.14 A typical multi-instance topology

```

      +---> Velran A  --+---> application DB
Internet -> proxy --|      |
      +---> Velran B  --+---> Redis
              |
              +-----> shared AppFs (*)
```

The asterisk matters: if the application uses uploaded files/AppFs as business state, multi-instance infrastructure must make that data root consistently available to every instance. The V1 AppFs API provides confinement and safe file operations; infrastructure still decides how the data root is shared correctly.

Component roles are:

- **reverse proxy/load balancer:** TLS, routing, instance selection;
- **Velran instance:** near-stateless application process with bounded local runtime state;
- **application DB:** durable structured business source of truth;
- **AppFs:** durable file/media business state when used;
- **Redis:** rebuildable shared session/cache/rate-limit/replay infrastructure.

16.15 Rolling restart and cache generation

A new release rollout does not require manually deleting old cache keys. Route generation is part of the cache key, so a new release/compiler generation creates a new namespace. Explicit business invalidation still comes from state-changing route declarations.

During a rolling restart, the proxy should send traffic only to ready instances. Use SIGTERM graceful drain. The orchestrator's termination grace period should exceed the server's configured shutdown grace period.

Sticky sessions are not required when the shared session backend is configured correctly.

16.16 What should you monitor?

Cache and rate limiting are not invisible optimizations. Minimum production monitoring includes:

- `velran_cache_hits_total` and `velran_cache_misses_total`;
- 429 responses and `velran_rate_limit_denials_total`;
- `rate_limiter_unavailable` events;
- readiness 503 events on Redis/DB dependency failures;
- Redis latency, connection, and memory state in infrastructure monitoring;
- application DB latency during cache-miss periods.

Do not use raw IP, username, session ID, tenant, request ID, or full URL as metric labels. The bounded-cardinality rule from the observability chapter applies here too: metric labels must not grow without bound per user or request.

16.17 Redis in recovery

Redis is runtime state in this architecture: sessions, cache, rate-limit, and replay-protection state may live there, but it is not business source of truth. Recovery therefore focuses not on restoring cache, but on ensuring that the system rebuilds this state safely and predictably after Redis loss.

The full backup/restore order and which state must actually be backed up are covered in Chapter 19.

16.18 Practical example: cached news front page with publishing

The public front page is user-independent and can be cached:

```
#[page]
fn frontPage(ctx: PageContext, db: Db)
  -> Result<Html, PageError> {
  let articles = latestArticles(db)?;
  return Ok(html {
    <main>
      @for article in articles {
        <a @href(article, article.slug)>
          {{ article.title }}
        </a>
      }
    </main>
  });
}

route frontPage GET "/"
  public cache public ttl 60
  => frontPage;
```

Publishing is an authenticated state change and is not cacheable, but it invalidates the front page:

```
route publish POST "/admin/publish/:id<i64>"
```

```
auth role Publisher
invalidate cache frontPage
=> publish;
```

The mental model remains simple:

1. the database is the source of truth for articles;
2. the public HTML cache is only a fast copy;
3. after the publish transaction, the new generation causes the next GET to build a fresh page;
4. any Velran instance may perform that rebuild;
5. the resulting response goes into shared Redis cache.

16.19 Checklist

Before a multi-instance production deployment, verify:

- Redis URL comes from a secret file and TLS/security policy is appropriate;
- shared sessions eliminate the need for sticky sessions;
- local-auth SQLite is not treated as a shared multi-instance identity store;
- public cache exists only on truly user-independent GET routes;
- every relevant state-changing route declares explicit cache invalidation;
- cache TTL does not exceed the operator maximum;
- production uses `allow_memory = false` for cache and rate limiter;
- rate-limit scope derives from trusted identity/effective client IP;
- 503/readiness behavior has been tested for Redis outage;
- cache hit/miss and rate-limit denial metrics are monitored;
- Redis state is not treated as a required business backup component.

16.20 Practice: classify cacheable data

Exercise mode: independent. Use the independent method defined in the preface.

For each candidate cache entry, name the authoritative source, key scope, lifetime, invalidation trigger, and authorization context. If any of these is unclear, delay caching until the contract is explicit.

16.21 Common mistake: caching before defining consistency

A cache can make stale or cross-user data much faster. Performance work must preserve the same ownership and visibility rules as the uncached path, with invalidation tied narrowly to authoritative state changes.

16.22 Running project checkpoint: Secure Notes

Cache only a read path whose visibility rules are already stable. Prove that publishing, editing, deleting, or changing authorization cannot leave another user seeing a stale or incorrectly scoped representation.

Chapter 17

Logging, audit, metrics, and troubleshooting

Learning path: Complete path / operations depth. Read when you own production diagnosis, telemetry, and audit evidence.

Security invariant: telemetry is not a data dump

Logs and traces must preserve the minimum evidence needed to operate and investigate the system without becoming a second store for secrets, credentials, tokens, or raw request bodies.

Production diagnostics must correlate a request, route, status, security decision, and any durable business audit event.

First read. Focus on request IDs, log classes, redaction/cardinality, health, and the troubleshooting workflow. Treat proxy, rotation and retention details as operational reference.

Velran provides four separate surfaces for this:

- structured **server log** for technical and lifecycle events;
- structured **access log** for HTTP requests;
- structured **audit log** for security and user activity;
- a separate-listener **Prometheus metrics** endpoint.

Applications may additionally store a transactional **business audit trail** in their database. The audit log and business audit are not the same thing.

17.1 The role of the three log classes

Recommended production configuration:

```
[observability]
metrics_listen = "127.0.0.1:9090"
allow_public_metrics = false
access_log = true

[logging]
server_file = "/var/log/velran/server.log"
access_file = "/var/log/velran/access.log"
```

```
audit_file = "/var/log/velran/audit.log"
stderr = true
```

Each file serves a different purpose.

server.log Startup, shutdown, dependency failures, technical and internal runtime events. Check this first if the server does not start, readiness fails, or a background component reports trouble.

access.log One record per HTTP request. This is the traffic and latency trace: method, route, status, duration, client IP, input/output bytes, request ID.

audit.log Security and user activity: login/logout, auth or MFA failure, policy denial, rate-limit denial, resource-profile startup audit, and authenticated state-changing actions.

`access_log = false` disables only the access log. Security/user audit remains active.

17.2 Request ID: following one request end to end

Every successfully parsed application, static, or health request receives a server-generated identifier:

```
X-Request-Id: velran-...
```

An access-log entry may look like:

```
{"event":"http_request","request_id":"velran-...",
 "method":"GET","route":"articleShow","status":200,
 "duration_ms":7,"client_ip":"203.0.113.5",
 "bytes_in":0,"bytes_out":812}
```

A client-provided `X-Request-Id` is not authority. Velran generates its own request ID so an attacker cannot force the server to use an attacker-chosen correlation identifier.

The basic troubleshooting workflow is therefore:

```
# 1. find the request ID in the access log
grep 'velran-...' /var/log/velran/access.log

# 2. search for the same ID in the technical log
grep 'velran-...' /var/log/velran/server.log

# 3. for security cases, search the audit log too
grep 'velran-...' /var/log/velran/audit.log
```

Together the three logs provide the complete technical picture.

17.3 What may and may not be logged?

A log is not a debug-data dumping ground. In production, sensitive request contents especially must not be logged automatically.

Velran logs must not contain:

- Authorization headers;

- Cookie headers or session IDs;
- CSRF tokens;
- TOTP secrets or recovery codes;
- DB, Redis, or LDAP credentials;
- secret-file contents;
- complete request bodies or form payloads.

For durable business evidence, use explicit narrow audit fields rather than request-body logging.

17.4 Security audit and HTTP denials

For trust-boundary denials, Velran emits a dedicated `security_audit` event with the same request ID. This includes invalid/untrusted forwarding metadata returning 400, security decisions returning 401/403/421/426/429, and more precise categories for CSRF/CORS/origin denials. This lets operators distinguish, for example:

- `proxy/invalid_forwarding`: malformed or untrusted forwarding metadata;
- `auth/unauthorized`: missing authentication on an API route;
- `csrf/denied`, `cors/denied`, `origin/denied`: browser trust-boundary denial;
- `policy/forbidden`: route- or object-authorization denial;
- `host/mismatch`, `transport/https_required`: Host/TLS boundary failure;
- `rate_limit/denied`: limiter denial.

A key developer distinction is that 403 does not necessarily mean a program bug. It may be the correct result of security policy. Always inspect the audit event and route policy too.

17.5 Business audit: durable business history

Operational logs describe events; durable business state changes belong in a transactional business audit record.

Example:

```
enum ArticleStatus {
    Review
    Published
}

transaction db {
    Article.publishChanged(tx, id, version)?;
    audit Article id action publish;
        from article.status
        to ArticleStatus.Published
}
```

The audit runs in the same DB transaction as the mutation. If the audit record cannot be written, the business change rolls back too. This is stronger than writing a separate log line.

Velran writes to the `_velran_business_audit` runtime table fields such as:

- runtime-generated event ID and UTC timestamp;
- HTTP request ID;
- canonical authenticated actor;
- source action;

- object type and object identifier;
- static business-action name;
- optional previous and new domain state.

A business audit statement does not replace authorization:

```
let invoice = Invoice.byId(db, id?);
authorize invoice owner ownerUsername or role Accountant;

transaction db {
  Invoice.approve(tx, id, version?);
  audit Invoice id action approve;
    from invoice.status
    to InvoiceStatus.Approved
}
```

The correct order is: **identity -> authorization -> mutation -> audit in the same transaction.**

17.6 Platform security events, redaction, and burst alerts

Security monitoring is a structured platform boundary. Automatic events cover authentication/MFA abuse, policy/browser-boundary denial, webhook failure, idempotency conflict, and resource exhaustion.

The event schema intentionally does not accept arbitrary request payload or message text. Emitted principal/source fields are redacted. If application code must create a safe representation of sensitive text for a permitted monitoring sink, use the trusted `redact(...)` builtin; `Redacted<T>` cannot be created by annotation or cast.

The server also maintains bounded in-memory correlation and platform-owned burst thresholds with cooldowns. When a threshold is crossed, it emits a structured `threshold_exceeded` alert event. Raw correlation keys may exist transiently in memory but are not written into the security event.

This monitoring layer does not replace enforcement. Rate limiting, authorization, replay prevention and resource limits still block the request; alerts add detection and incident-response evidence.

17.7 Prometheus metrics on a separate management socket

Metrics are not an application route. They run on a separate listener:

```
[observability]
metrics_listen = "127.0.0.1:9090"
allow_public_metrics = false
```

Query from the same host:

```
curl http://127.0.0.1:9090/metrics
```

The listener serves only GET/HEAD `/metrics`. It does not run application routes, sessions, or auth handlers.

Typical metrics include:

```

velran_requests_total
velran_responses_5xx_total
velran_auth_failures_total
velran_csrf_failures_total
velran_policy_denials_total
velran_request_timeouts_total
velran_runtime_budget_exceeded_total
velran_rate_limit_denials_total
velran_readiness_failures_total
velran_request_bytes_in_total
velran_response_bytes_out_total
velran_active_connections
velran_log_fallback_total
velran_request_duration_ms_bucket
velran_route_requests_total{route="..."}
velran_route_5xx_total{route="..."}
velran_route_duration_ms_sum{route="..."}

```

`velran_policy_denials_total` tracks security-policy denials: in addition to 403/421/426 decisions, a 400 explicitly classified as forwarding-metadata failure counts too. A normal malformed-request 400 does not.

The most important rule is **bounded cardinality**. The `route` label is a compiler-owned route name or fixed reserved name. Raw URLs, path parameters, IP addresses, usernames, tenants, session IDs, and request IDs must never become metric labels. Otherwise an attacker could create an unbounded number of time series.

17.8 Who scrapes metrics behind Apache or Nginx?

The public web proxy and the metrics listener are separate trust boundaries. A typical layout is:

```

Internet
  |
  v
Apache/Nginx :443
  |
  v
127.0.0.1:8080 Velran server

Prometheus agent
  |
  v
127.0.0.1:9090 Velran metrics

```

Do not automatically proxy the metrics port out through the public domain. If a remote Prometheus server must reach it, use a private management network, ACL, or separate authenticated monitoring proxy.

17.9 Health: liveness and readiness are different

Built-in server endpoints are:

```
GET /health/live
HEAD /health/live

GET /health/ready
HEAD /health/ready
```

Liveness only proves that the server event loop is alive. It does not call DB, Redis, or LDAP.

Readiness reports whether configured runtime dependencies required for service are reachable. DB or Redis failure/timeout may produce 503 **Service Unavailable**.

Therefore in a load balancer:

- do not remove an instance based on **liveness** because of a brief DB outage;
- but an instance may be removed temporarily based on **readiness**.

A health endpoint is not a business monitoring API; it deliberately exposes minimal status.

17.10 Logrotate and SIGHUP

Velran writes file logs in append mode. Let the operating system handle log rotation in production. The repository includes `examples/logrotate/velran`.

Simplified example:

```
/var/log/velran/server.log /var/log/velran/access.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 0640 velran velran
    sharedscripts
    postrotate
        systemctl kill -s HUP --kill-who=main velran.service || true
    endscript
}
```

The meaning of **SIGHUP** is deliberately bounded. In behind-proxy mode it reopens log files and transactionally rebuilds domain/application hosting state; it is not a general process-level `server.toml` reconfiguration mechanism. Application source changes normally use the automatic source-reload supervisor instead.

For configuration changes:

```
/usr/local/bin/velran-server \
--config /usr/local/etc/velran/server.toml \
--check-config

systemctl restart velran.service
```

This is more auditable and predictable than a partially successful hot reload.

17.11 What if the log file is not writable?

File logging uses a bounded writer queue. If the queue fills, a file sink fails, or the writer is unavailable, Velran falls back to stderr and increments:

```
velran_log_fallback_total
```

Treat increases as alertable conditions. Possible causes include:

- permission or filesystem failure;
- full disk;
- broken logrotate/reopen;
- sustained log-processing pressure.

The parent directory of a log path must exist. On Unix, the runtime opens log files with mode 0640 and without following symlinks; the operator must still configure directory permissions correctly.

17.12 Practical production troubleshooting workflow

When investigating 5xx responses or “the page is slow,” do not guess. Use a reproducible order.

1. **Liveness:** is the process itself running?
2. **Readiness:** are DB/Redis reachable?
3. **Access log:** which route, status, duration, and request ID are involved?
4. **Server log:** does the same request ID show a dependency/runtime error?
5. **Audit log:** did security policy or rate limiting reject the request?
6. **Metrics:** is this isolated or a trend in 5xx/latency?
7. **Reverse proxy:** did the failure occur before, inside, or after Apache/Nginx?

Quick checks:

```
curl -i https://www.example.com/health/live
curl -i https://www.example.com/health/ready
# For proxy deployments, also test the internal upstream:
curl -i http://127.0.0.1:8080/health/ready
curl -s http://127.0.0.1:9090/metrics | grep velran_responses_5xx_total
journalctl -u velran.service --since "10 min ago"
```

If the loopback Velran endpoint is healthy but the public domain is not, inspect Apache/Nginx, TLS, or forwarded-header policy next. If loopback readiness is also 503, investigate backend dependencies first.

17.13 Minimum monitoring baseline

Even a small production deployment should alert on at least:

- readiness persistently not ready;
- sudden growth in 5xx rate/count;
- meaningful request-latency degradation;
- unusual spikes in auth/policy/rate-limit denials;
- increases in `velran_runtime_budget_exceeded_total`;

- increases in `velran_log_fallback_total`;
- process restart loops or unexpected shutdown.

Thresholds should not be hardcoded by the language. Operators set them according to traffic, SLOs, and business criticality.

17.14 Retention, SIEM, and audit protection

Retention is an organizational and legal policy. Velran does not impose one universal retention period.

Local `audit.log` is an important operational trace, but by itself it is not a strong tamper-evident archive because a host administrator or actor with sufficient filesystem rights can modify it. For higher assurance, forward it to a centralized access-controlled or append-only log/SIEM system.

The business-audit database table is likewise not protected against direct modification by a DB administrator. Together the two layers provide a strong application trace, but compliance environments still benefit from an independent external log archive.

17.15 Checklist

Before production startup, verify:

- separate server/access/audit log paths exist;
- log-directory permissions are restrictive;
- request bodies and secrets are not logged;
- the metrics listener is not accidentally public;
- Prometheus labels have bounded cardinality;
- liveness and readiness are treated separately;
- logrotate triggers SIGHUP, while config changes trigger restart;
- `velran_log_fallback_total` is monitored;
- business workflows write business audit in the same DB transaction;
- audit-log forwarding and retention have a documented decision.

17.16 Practice: design an evidence trail

Exercise mode: independent. Use the independent method defined in the preface.

Take one important request and list the operational facts needed to reconstruct what happened without logging secrets or full sensitive payloads. Include request identity/correlation, outcome, latency, policy rejection, and relevant domain audit identifiers.

17.17 Common mistake: logging data instead of events

More log volume is not automatically more observability. Prefer structured events that explain decisions and outcomes. Avoid copying credentials, tokens, request bodies, or sensitive business records into logs simply because they are available.

17.18 Running project checkpoint: Secure Notes

Define observable events for failed authorization, publish transitions, integration failures, and unexpected server errors. Keep business audit and operational telemetry distinct but correlatable.

Chapter 18

Production deployment and service lifecycle

Learning path: Fast path: core for production. Learn the release/activation contract and safe deployment workflow.

Security invariant: verify before activation

Compile and validate the complete candidate generation before it can become active. If verification or activation fails, keep the previous known-good generation serving traffic.

Production diagnostic: SEC-PROD-002

Meaning: The application declares an incomplete production policy, so the strict deployment contract cannot be proven.

Fix: Declare all required production invariants: HTTPS required, debug disabled, HSTS required, and database TLS required.

This chapter does not repeat the configuration reference or observability details. From a production perspective, it summarizes how a verified release becomes a running service, how release switching works, and how the process lifecycle remains controlled. Complete backup, restore, and disaster recovery are covered separately in the next chapter.

18.1 The application production contract

Production security is partly declared by the Velran program itself:

```
production {  
    https required;  
    debug disabled;  
    hsts required;  
    database tls required;  
}
```

This is not a second secret/configuration system. It states requirements; the server/operator configuration still supplies certificates, paths and trusted networks. In other words, the Velran source says *what must be true*, while trusted deployment configuration says *how this environment makes it true*. Startup fails if the real topology cannot satisfy the contract. Source

reload cannot silently change it; changing production security requirements requires a restart and explicit operator review.

18.2 Release path

Recommended order:

```
./verify.sh
-> cargo build --locked --release
-> immutable release directory
-> velran-server --config ... --check-config
-> velran-cli migrate verify
-> approved velran-cli migrate apply
-> velran-cli migrate verify
-> atomic current symlink switch
-> controlled restart
-> /health/live
-> /health/ready
-> log / metric / audit review
```

The built `velran-server` and `velran-cli` binaries, application source, and migrations should all be attributable to the same release identifier. Do not modify files one by one inside a live `/srv/velran/current` directory. Prepare a new immutable release directory and switch the symlink atomically.

18.3 systemd service

`velran-server` is a long-lived service process. `velran-cli` is not a daemon: it starts for a migration, auth administration, or another operator task and exits when the task is complete.

```
[Service]
User=velran
Group=velran
ExecStart=/usr/local/bin/velran-server \\\
  --config /usr/local/etc/velran/server.toml
Restart=on-failure
NoNewPrivileges=true
PrivateTmp=true
```

Run the service under an unprivileged account. Under `systemd`, let `systemd` own the process-level cgroup ceilings such as `MemoryMax`, `CPUQuota`, and `TasksMax`; do not simultaneously configure Velran to write the same cgroup controls. The goal is one clear authority for each resource boundary.

For direct TLS on ports 80/443, grant only the bind-service capability (`CAP_NET_BIND_SERVICE`). Behind a reverse proxy on `127.0.0.1:8080`, omit it.

18.4 Controlled restart and readiness

After changing process-level configuration, run `-check-config` first, then perform a controlled restart. In behind-proxy mode, `SIGHUP` reopens logs and transactionally rebuilds domain/application hosting state, but listener, database/Redis/auth connections, cgroup policy, logging

sinks, and other process-level settings still require restart.

Application-source changes use the candidate-activation path described below: compile and validate away from request handling, then swap the domain atomically. A rejected candidate leaves the previous generation active.

During a rolling deployment, send traffic only to ready instances: start the new instance, wait for readiness, then remove the old one. Health diagnostics are covered in the observability chapter.

18.5 Migration and rollback are not the same operation

Rolling back an application binary or source release may be simple; rolling back a database schema is not. Prefer expand/contract migrations in production: first introduce a backward-compatible schema, then remove obsolete structures only in a later release after the rollback window has closed.

Do not tie automatic reverse SQL to switching the application symlink back. If a migration transformed or deleted data, the real recovery point may be the verified backup.

18.6 Recovery gate before deployment

Before a destructive or schema-compatibility-sensitive release, require a known recovery point and recent restore evidence. Backup contents, DB/AppFs consistency, RPO/RTO, and the rollback-versus-restore decision tree are covered in the next chapter. Deployment is responsible for ensuring that this gate is satisfied before the release switch.

18.7 Minimum post-deploy checks

1. the new process is running and ready;
2. the public home page or a smoke route responds through the real edge path;
3. there is no new error spike in the server log;
4. the access-log status distribution looks normal;
5. the audit log is writable;
6. DB/Redis dependency metrics have not degraded;
7. the release hash and database migration state are known for a rollback decision.

18.8 Release invariants

A production release should preserve a small set of invariants regardless of deployment tooling:

- build and verification use the locked dependency graph;
- the application source, migrations, server/CLI binaries and approved configuration version are attributable to one release record;
- production secrets remain outside the immutable release tree;
- a candidate application is fully compiled and validated before it can become active;
- activation is atomic: requests observe either the old generation or the new generation, never a half-installed mixture;
- a failed candidate leaves the previous known-good generation serving traffic;
- database migration state and application rollback compatibility are known before the switch.

Operationally: a candidate either passes the complete gate and activates atomically, or it does not activate. Native compilation failure is a deployment failure, never a fallback signal.

18.9 What can reload and what requires restart

Source reload is intentionally narrower than process reconfiguration. Treat these as different operational events.

Change	Normal mechanism	Reason
Velran application source/module graph	compile candidate, validate, atomic generation swap	application-local code generation
Immutable release symlink target	watched source path triggers candidate generation	preserves atomic release layout
Log rotation in supported topology	SIGHUP/log reopen	no application-policy change
Listener/TLS/process topology	config check + controlled restart	process-level authority
DB/Redis/auth connection configuration	config check + controlled restart	connection/service lifecycle
Cgroup/resource authority	controlled restart	process-level isolation boundary
Production security requirements	operator review + restart	source reload must not silently weaken policy

This distinction prevents a convenient source reload mechanism from turning into an implicit process-configuration channel.

18.10 Candidate activation: the operator rule

Treat activation as a two-step boundary: *compile/validate first, activate second*. Failure leaves the known-good generation serving traffic; never edit the active generated artifact in place. Application developers can stop at this contract.

Under the hood: how native activation preserves the boundary

This subsection is optional for application developers. The source-reload supervisor watches the logical entrypoint and complete transitive module graph. After the file set remains stable for the debounce period, compilation happens away from request handling. The candidate is checked against the same route, resource, cache, authentication and hosting constraints used at startup.

Internally, the accepted source becomes verified executable IR, generated safe Rust, a `rustc`-compiled immutable `cdylib`, and only then a new runtime generation. Only after those checks succeed is the domain runtime replaced atomically. Requests that already hold the previous runtime finish on that generation; later requests use the new one. Public-cache generations advance at the same activation boundary, so stale rendered content does not survive merely because its previous TTL has not expired.

There is no VM/interpreter fallback if native compilation or activation fails. This is an

implementation detail with an operational consequence: failure remains visible and the known-good generation remains authoritative.

18.11 Pre-deploy evidence

Before switching traffic or the active release, record enough evidence to answer four questions later:

1. **What was built?** release identifier, source revision/artifact identity and hashes.
2. **What dependencies and capabilities were approved?** locked Cargo graph plus supply-chain capability/provenance verification.
3. **What schema is expected?** migration set and successful migration verification.
4. **How do we recover?** known backup/restore readiness and the compatibility boundary for application rollback.

This evidence is deliberately secret-free. It proves provenance and operational state without copying credentials into release records.

18.12 Deployment anti-patterns

Avoid the following shortcuts:

- running an unlocked release build that silently updates dependency resolution;
- editing files directly inside the active release directory;
- treating application rollback as automatic database rollback;
- letting the server apply migrations implicitly during startup;
- putting secrets in command-line arguments, release manifests or source files;
- sending traffic before readiness has succeeded;
- accepting a failed candidate by falling back to a different execution engine.

18.13 Practice: rehearse an activation failure

Exercise mode: operational scenario. Use the operational-scenario method defined in the preface.

Prepare a candidate release that intentionally fails one pre-activation check. Verify that the active generation remains unchanged and that the failure is visible to operators. Then rehearse a successful activation and rollback.

18.14 Common mistake: equating build success with deployability

A compiled artifact is only one piece of release evidence. Configuration, migrations, secrets, TLS material, resource policy, health checks, and activation semantics must all be valid before traffic depends on the generation.

18.15 Running project checkpoint: Secure Notes

Freeze `examples/secure-notes/checkpoints/18-production/main.vrn` as the production candidate. Package Secure Notes as a production candidate. Record the exact verification

commands, migration state, configuration check, activation evidence, and rollback point. The goal is a repeatable release procedure rather than a one-off successful deployment.

Chapter 19

Backup, restore, and disaster recovery

Learning path: Fast path: core for production. Define recoverability before production data becomes irreplaceable.

A backup is useful only if the application is **provably restartable from a known recovery point**. Velran business state may span the application database, the local-auth identity store, and AppFs, so define the complete recovery state, its consistency boundary, and how restore is regularly proven.

19.1 Start with a state inventory

Before production, list everything that belongs to application state. Typically:

- the application database;
- the local-auth SQLite database, if local auth is used;
- the AppFs `data_root`, including uploaded media and other runtime data;
- the immutable release identifier and artifact hash;
- the migration set and checksums;
- the version or hash of trusted configuration and policy files;
- backup time, application version, and database backend.

Secret files belong to a separate secret-management boundary. The recovery runbook must know where the required DB, Redis, TLS, or LDAP credentials can be restored from, but **secret values must never be written into a plaintext release or backup manifest**.

19.2 What is not business source of truth?

In V1, Redis stores session, rate-limit, and public-cache state. Do not treat that state as part of the business backup contract.

An empty Redis is a safe and acceptable disaster-recovery baseline:

- sessions disappear, so users sign in again;
- the public cache rebuilds;
- rate-limit windows restart.

If the same Redis cluster stores another application's business data, backing up that data belongs to that other system's recovery contract.

19.3 RPO and RTO are business decisions before they are technical settings

Record two values before implementing the backup mechanism:

RPO – Recovery Point Objective the maximum acceptable data-loss window;

RTO – Recovery Time Objective the maximum acceptable time to restore service.

If the RPO is 24 hours, a daily backup may be sufficient in principle. If the RPO is measured in minutes, database-level WAL/binlog/PITR or equivalent infrastructure may be required. The Velran runtime cannot invent organizational RPO/RTO values; operators must translate those requirements into backup technology.

19.4 Safe V1 baseline: a controlled maintenance backup

The database and AppFs may jointly represent business state. A DB row may, for example, refer to an uploaded image. A DB dump and AppFs archive taken at unrelated times are therefore not automatically a consistent recovery point.

The simplest auditable V1 baseline is:

```
traffic drain
-> SIGTERM / graceful shutdown
-> stopped state verified
-> application DB backup
-> local-auth DB backup, if used
-> AppFs data_root snapshot
-> recovery manifest + hashes
-> service start
-> readiness
```

This introduces a short write pause but gives a clear consistency boundary. An online backup should be called consistent only when the DB/storage infrastructure can provide snapshots tied to the same recovery point, or when the application data model demonstrably tolerates different snapshot times. The runtime itself does not promise a distributed snapshot.

19.5 Application DB: use the backend's native backup tool

Velran does not attempt to hide native database backup mechanisms. Operators should use the supported tooling of the selected backend, preferably with a dedicated backup credential.

PostgreSQL

A typical logical backup:

```
pg_dump --format=custom --no-owner --no-acl \\  
--file app.dump "$DATABASE_URL"
```

For an isolated restore test:

```
createdb velran_restore_test
pg_restore --no-owner --no-acl \\\
--dbname velran_restore_test app.dump
```

Do not put production secrets in shell history. For large databases, physical backup and PITR may be the better choice; their runbooks belong to the PostgreSQL platform.

MariaDB

For a transactional InnoDB schema, a typical logical backup is:

```
mariadb-dump --single-transaction \\\
--routines --triggers app > app.sql
```

Restore into an isolated database:

```
mariadb velran_restore_test < app.sql
```

`--single-transaction` does not magically make non-transactional tables consistent. For a mixed schema, keep the controlled-maintenance baseline.

SQLite

Do not call a plain `cp` of a live SQLite database a backup. Use SQLite's backup mechanism or take a filesystem snapshot while the application is stopped.

```
sqlite3 /srv/velran/data/app.db \\\
".backup '/backup/app.db'"
```

Always restore into a separate test path. Do not discover whether a backup works by restoring over production.

19.6 The local-auth DB is a security-sensitive backup

When local auth is enabled, the identity store is a separate SQLite boundary. It may contain password hashes, TOTP secrets, and recovery hashes, so treat its backup as a **secret**.

```
sqlite3 /srv/velran/auth/users.db \\\
".backup '/backup/local-auth.db'"
```

Store the backup encrypted, with narrow access and documented retention. After restore, verify file and parent-directory ownership and permissions. The identity store cannot be reconstructed from the application database.

19.7 AppFs: the entire data root may be durable state

Do not back up only the media subdirectory that is published over HTTP. The entire declared AppFs `data_root` may belong to the recovery state.

With the service stopped, for example:

```
tar -C /srv/velran -cpf data-root.tar data
```

After restore, preserve or restore ownership and permissions. The service account should be able to write only the roots declared writable by configuration; do not weaken AppFs confinement with symlinks.

19.8 Every recovery point needs a manifest

Backup objects alone do not identify which release and migration state they belong to. Create a secret-free manifest:

```
backup_id=2026-09-04T180000Z
release_id=2026-09-04.1
server_sha256=...
app_source_sha256=...
migrations_sha256=...
config_sha256=...
db_backend=postgresql
app_db_backup_sha256=...
local_auth_backup_sha256=...
data_root_backup_sha256=...
```

The manifest should contain hashes and identifiers, not credential values. The included `deploy/release-record.sh` helps record release/source/migration/config state in read-only mode.

19.9 Restore drill: prove the backup

A successful backup job does not prove recoverability. Regularly run a restore drill in an isolated environment.

```
new empty restore DB
-> application DB restore
-> local-auth restore, if used
-> AppFs restore into separate root
-> select exact immutable release
-> restore-test config on isolated host/port
-> velran-server --check-config
-> velran-cli check main.vrn
-> velran-cli migrate verify
-> start velran-server on isolated listener
-> /health/live + /health/ready
-> application-specific smoke test
-> log and audit verification
```

`deploy/restore-verify.sh` automates part of the non-mutating config/source/migration checks. It does not start a production listener and does not run `migrate apply`.

Record the drill result with date, backup ID, and release ID. “The backup completed” is not a release gate; you need recent restore evidence appropriate to the organization’s RPO/RTO requirement.

19.10 Make a recovery decision before an upgrade

A controlled production upgrade should follow an order such as:

```
immutable release artifact + hashes
-> migration verify
-> compatibility decision
-> consistent backup + recent restore evidence
-> migration apply
-> migration verify
-> atomic release switch
-> controlled restart
-> live + ready + smoke
-> logs + metrics + audit review
```

Record migration compatibility at release level. The key question is whether the previous application can still run against the new schema.

19.11 Expand/contract keeps the fast rollback window open

If the new schema is backward-compatible, a faulty application release can be switched back to the previous immutable artifact without restoring the database.

Recommended pattern:

1. **expand**: add a nullable/defaulted column, table, or index while keeping the old application working;
2. deploy the new application;
3. perform controlled data backfill if needed;
4. only in a later release **contract**: remove the old column or constraint after the rollback window has closed.

A destructive rename/drop or immediate incompatible shape change can close the app-only rollback option. Mark that explicitly before release.

19.12 Rollback and restore are different operations

Faulty application, backward-compatible schema

```
traffic drain
-> previous immutable release
-> controlled restart
-> live + ready + smoke
```

There is no database restore, so business data created after the faulty deployment is preserved.

Faulty application, but a forward fix is possible

Prefer a new immutable release or a forward migration. Do not tie automatic reverse SQL to switching the release symlink back.

Schema or data is corrupted

This is disaster recovery:

```
stop writes
-> choose and record recovery point
-> restore DB + auth DB + AppFs consistently
-> select matching release and config
-> verify + smoke
-> controlled return to traffic
```

Writes after the selected recovery point may be lost. This is a business incident decision, not a technical footnote: the data-loss window must be explicitly accepted and documented.

19.13 Forbidden shortcuts

- calling a plain `cp` of a live SQLite DB a backup;
- backing up only the application DB when AppFs also contains business state;
- considering backup successful without a restore drill;
- tying an automatic down migration to application rollback;
- testing restore on the production database;
- giving the normal application service the migration credential;
- writing secret values into release or backup manifests;
- making Redis session/cache restoration a prerequisite for disaster recovery.

19.14 Production recovery checklist

A system is recovery-ready only if at least the following are true:

1. the state inventory is documented;
2. RPO and RTO are recorded;
3. backup encryption, retention, and access policy are known;
4. there is an identifiable recent successful restore drill;
5. migration compatibility for the release is documented;
6. the app-only rollback procedure has been exercised;
7. the owner and approval path for destructive restore are known;
8. the effect of losing Redis is accepted;
9. the recovery runbook identifies the source of release/config/secret restoration;
10. live, ready, smoke, log, and audit checks are performed after an incident.

The core principle of recovery is: **a backup is not a file; it is a proven ability to restore.** Without restore drills, RPO and RTO are assumptions.

19.15 Practice: prove a restore, not a backup

Exercise mode: operational scenario. Use the operational-scenario method defined in the preface.

Restore a copy of the application state into an isolated environment and verify record counts, critical business invariants, media references, and application startup. Measure the restore time and record any manual step that could block an incident response.

19.16 Common mistake: backing up components independently without a recovery model

Database rows, uploaded files, configuration, and release identity may form one recoverable state even when stored separately. A backup plan must define how those pieces line up at restore time.

19.17 Running project checkpoint: Secure Notes

Perform a full Secure Notes recovery rehearsal including persistent data and any media attachments. Confirm that a restored generation serves the same ownership and publication semantics as before the simulated failure.

Part V

Reference and operator quick guide

Chapter 20

The Velran CLI and the daily developer/operator workflow

Learning path: Fast path: operator reference. Use the CLI consistently for check, build, verification, and operational workflows.

Velran installs two programs: the long-lived `velran-server` process and the short-lived `velran-cli` operator tool. Treat them consistently as separate responsibilities. The server serves HTTP and enforces runtime policy; the CLI checks source, manages migrations, and performs local-auth administration.

This chapter introduces no new language features. It shows how daily work becomes a repeatable command flow from editing through production verification.

20.1 Which tool for which task?

Task	Tool	Notes
Check Velran source	<code>velran-cli check</code>	Compiler-level check; does not start an HTTP server.
Migration status/verify/apply	<code>velran-cli migrate</code>	DB URL comes from a secret file.
Local-auth store administration	<code>velran-cli auth</code>	Operator action on the SQLite identity store.
Server configuration preflight	<code>velran-server</code>	<code>-check-config</code> : validates config, app, and local policy/resource files.
Inspect effective configuration	<code>velran-server</code>	<code>-print-effective-config</code> ; secret values remain redacted.
Run the web application	<code>velran-server</code>	Long-lived process behind a proxy or direct HTTPS.
Workspace release build	<code>cargo build</code>	Build both binaries with <code>-locked -release</code> .
Verify the whole platform	<code>./verify.sh</code>	Part of Velran repository release evidence, not a command every application must run.

Production examples install the programs here:

```
/usr/local/bin/velran-cli  
/usr/local/bin/velran-server
```

With a normal shell `PATH`, the examples below can use the short program names.

20.2 The CLI is not an alternative server

`velran-cli` does not open a listener, serve routes, or maintain session state. It performs one command and exits. This is useful from a security perspective: migrations and user administration do not require an administrative HTTP endpoint.

Conversely, `velran-server` does not run migrations automatically at startup. Schema changes remain explicit operator decisions.

20.3 The first command: `velran-cli check`

During daily development, make this the first gate after every meaningful source change:

```
velran-cli check main.vrn
```

On success, the CLI prints a short summary of models, queries, pages, actions, and routes seen by the compiler. On failure, do not start the server yet; fix the compile-time contract first.

`check` catches issues such as:

- unknown types or invalid language syntax;
- query projection and model-shape mismatch;
- invalid typed bind/decode contracts;
- route/policy or resource-profile reference errors that are provable at compile time;
- forbidden raw HTML/SQL-style constructs.

It does not prove that the DB or Redis is reachable over the network and does not replace HTTP integration tests.

20.4 Running from the source tree versus using the installed CLI

When developing the Velran repository itself, the same program can be run through Cargo:

```
cargo run --locked -p velran-cli -- check main.vrn
```

This is a developer convenience. In production runbooks, use the immutable installed release binary:

```
/usr/local/bin/velran-cli check /srv/velran/app/main.vrn
```

Do not mix the two forms in one deployment. A production operation should be clearly attributable to one release artifact.

20.5 Migration workflow

The migration command provides three operations:

status shows applied and pending migrations;

verify validates migration history and checksum contracts;

apply acquires the migration lock and applies pending migrations.

The DB URL is not a plaintext command-line secret. Put it in a one-line secret file:

```
/run/secrets/velran/db-url
```

Typical verification order:

```
velran-cli migrate status \\  
  --dir migrations \\  
  --db-url-file /run/secrets/velran/db-url  
  
velran-cli migrate verify \\  
  --dir migrations \\  
  --db-url-file /run/secrets/velran/db-url
```

For an approved schema change:

```
velran-cli migrate apply \\  
  --dir migrations \\  
  --db-url-file /run/secrets/velran/db-url \\  
  --lock-timeout-secs 30  
  
velran-cli migrate verify \\  
  --dir migrations \\  
  --db-url-file /run/secrets/velran/db-url
```

`--lock-timeout-secs` is a positive integer. If the migration lock cannot be acquired within that time, fail the operation; do not allow parallel schema changes.

--allow-insecure-db is not a production convenience switch

The migration CLI follows the server's secure-by-default model. The `--allow-insecure-db` flag is only for deliberate, controlled environments. In production, use proper TLS/database security rather than a permanent bypass.

20.6 Local-auth administration

If the application uses local auth, the CLI also owns identity-store administration. Initialize the store first:

```
velran-cli auth init \\  
  --db-url-file /run/secrets/velran/local-auth-db-url
```

Create a user:

```
velran-cli auth user-add \\  
  --db-url-file /run/secrets/velran/local-auth-db-url \\  
  --username user
```

```
--username alice \\  
--password-file /run/secrets/velran/bootstrap-password \\  
--role Editor
```

If no `-role` is supplied, a new user receives the `User` role. Supply multiple `-role` options to assign multiple roles.

Change a password or account state:

```
velran-cli auth password-set \\  
  --db-url-file /run/secrets/velran/local-auth-db-url \\  
  --username alice \\  
  --password-file /run/secrets/velran/new-password  
  
velran-cli auth disable \\  
  --db-url-file /run/secrets/velran/local-auth-db-url \\  
  --username alice  
  
velran-cli auth enable \\  
  --db-url-file /run/secrets/velran/local-auth-db-url \\  
  --username alice
```

Replace roles:

```
velran-cli auth roles-set \\  
  --db-url-file /run/secrets/velran/local-auth-db-url \\  
  --username alice \\  
  --role Editor --role Publisher
```

`roles-set` treats the supplied role list as the new complete state. Do not treat it as an incremental “add one role” operation.

20.7 TOTP administration is especially sensitive

Enroll TOTP:

```
velran-cli auth totp-enroll \\  
  --db-url-file /run/secrets/velran/local-auth-db-url \\  
  --username alice \\  
  --recovery-count 8
```

The command prints the TOTP secret, the `otpauth` URI, and one-time recovery codes. Do not run it in CI logs, shell transcripts, or shared terminals. The operator process must securely hand off and store the secret and recovery codes immediately.

Disable TOTP:

```
velran-cli auth totp-disable \\  
  --db-url-file /run/secrets/velran/local-auth-db-url \\  
  --username alice
```

20.8 A secret file is also an input contract

The CLI does not read DB-URL and password files without limits. The current secret-URL file contract requires:

- a regular file;
- not a symlink;
- at most 16 KiB;
- exactly one non-empty line.

A password file must also be a regular non-symlink file; it is limited to 4096 bytes, contains one line, and the password itself must be 12–1024 bytes.

Do not try to bypass the contract with process substitution, symlinks, or multiline secret templates. The CLI boundary is deliberately narrow.

20.9 Server preflight follows CLI checks

After source and migration checks, validate the production configuration with the server:

```
velran-server \\  
  --config /usr/local/etc/velran/server.toml \\  
  --check-config
```

This reads the trusted TOML, compiles the application, checks route-rate/resource policy references and TLS configuration, and validates local preconditions for declared storage/static roots.

To audit precedence or effective policy:

```
velran-server \\  
  --config /usr/local/etc/velran/server.toml \\  
  --print-effective-config
```

The output is intentionally secret-redacted. It is still operator information, so retain automation logs only when justified.

20.10 Daily development cycle

A good baseline for a small or medium application is:

```
edit .vrn source  
-> velran-cli check main.vrn  
-> velran-cli migrate verify  
-> if schema changed: migrate status/apply/verify on dev DB  
-> velran-server --config server.dev.toml --check-config  
-> start velran-server  
-> HTTP/form/JSON smoke tests  
-> automated application tests
```

The point of the order is to catch cheaper, more deterministic failures first. There is little value in running HTTP smoke tests against source that already fails at compiler level.

20.11 Application release workflow

A minimal production operator flow is:

```
immutable release selected
-> velran-cli check deployed main.vrn
-> velran-server --check-config
-> velran-cli migrate status
-> velran-cli migrate verify
-> backup/recovery gate
-> approved velran-cli migrate apply, if pending
-> velran-cli migrate verify
-> controlled service restart/switch
-> /health/live
-> /health/ready
-> application smoke test
-> logs + metrics + audit review
```

Migration `apply` is the only schema-mutating step in this sequence; treat it as a separate approval point.

20.12 The Velran platform release workflow is different

When building the Velran repository itself, Rust workspace release evidence is part of the process:

```
./verify.sh
cargo build --locked --release \\\
  -p velran-server -p velran-cli
```

The resulting binaries are:

```
target/release/velran-server
target/release/velran-cli
```

Install them with:

```
sudo install -m 0755 target/release/velran-server \\\
  /usr/local/bin/velran-server
sudo install -m 0755 target/release/velran-cli \\\
  /usr/local/bin/velran-cli
```

Your own Velran web application's daily workflow does not need to verify the language's entire Rust workspace every time. Your own source, migration, configuration, and integration tests are the relevant gates.

20.13 Automation: separate read-only and mutating steps

In CI/CD, separate safely repeatable checks from state-changing operations.

Read-only/preflight:

```
velran-cli check main.vrn
```

```

velran-cli migrate status \
  --dir migrations --db-url-file /run/secrets/velran/db-url
velran-cli migrate verify \
  --dir migrations --db-url-file /run/secrets/velran/db-url
velran-server --config /usr/local/etc/velran/server.toml \
  --check-config
velran-server --config /usr/local/etc/velran/server.toml \
  --print-effective-config
health/readiness GET

```

Mutating/operator-approved:

```

velran-cli migrate apply \
  --dir migrations --db-url-file /run/secrets/velran/db-url
velran-cli auth init \
  --db-url-file /run/secrets/velran/local-auth-db-url
# Other auth commands also receive their explicit --db-url-file
# plus their own --username/--password-file/--role options.

```

Do not normally put auth-administration commands into the automatic application deployment pipeline. They are identity-administration operations with a separate audit and access boundary.

20.14 Common failures

check succeeds but the server does not start The compile-time contract is valid, but configuration, TLS, filesystem, or policy startup validation failed. Run `-check-config` and inspect the server log.

-check-config succeeds but readiness is red Local configuration is valid, but a runtime dependency such as DB or Redis is unreachable or unhealthy.

migrate verify fails Do not reflexively run `apply`. Resolve the history/checksum discrepancy first.

The auth CLI cannot open the DB Verify that the URL points to the local-auth SQLite store and that the store has been initialized.

Secret-file error Do not relax the file contract; provide a regular, non-symlink, one-line secret file.

20.15 Operator memory line

The command details above are deployment-specific; the invariant is the order:

```

source -> schema -> config -> approved mutation
-> service -> health -> evidence

```

Keep the concrete commands in a versioned runbook. Do not rely on memory for flags, secret-file paths, unit names, or approval points.

20.16 Practice: make the safe path the shortest path

Exercise mode: operational scenario. Use the operational-scenario method defined in the preface.

Write down the commands a developer or operator uses for check, test, release verification, configuration validation, and startup. Remove undocumented shortcuts and prefer commands that fail closed when lockfiles, policies, or release evidence are stale.

20.17 Common mistake: relying on remembered command sequences

If a safe release depends on a person remembering five optional flags, the workflow is fragile. Put invariant-preserving command sequences into versioned scripts and keep their output understandable enough for review.

20.18 Operator checkpoint: Secure Notes

Write a one-page command runbook for checking, building, verifying, and starting Secure Notes. A new operator should be able to follow it without knowing which steps are security-critical by memory alone.

Chapter 21

The `server.toml` configuration reference

Learning path: **Complete path / reference.** Search this chapter when configuring the server; it is not required as linear reading.

First reading. Learn precedence/fail-closed rules, the production baseline, and which trust domain owns each setting. Use the option tables as deployment reference.

Earlier chapters used server configuration in a task-oriented way. This chapter gives the current production configuration from an operator’s perspective in one place. The reference is based on the actual parser and `config/server.toml.sample`; if they ever differ, parser behavior is authoritative.

21.1 Loading, precedence, and fail-closed rules

Recommended production startup:

```
velran-server \\  
  --config /usr/local/etc/velran/server.toml
```

Precedence is:

```
built-in defaults < server.toml < explicit CLI override
```

The TOML file is trusted configuration, but parsing remains fail-closed:

- unknown section or key: startup error;
- duplicate key or table: TOML parse error;
- the config file must be a regular non-symlink file;
- maximum size is 1 MiB;
- filesystem paths coming from TOML must be absolute;
- secrets must not appear in plaintext fields: DB/Redis/LDAP/local-auth credentials are supplied only through `*_file` references.

Validate without starting a listener:

```
velran-server \\  
  --validate
```

```
--config /usr/local/etc/velran/server.toml \\  
--check-config
```

Inspect the secret-free effective view:

```
velran-server \\  
  --config /usr/local/etc/velran/server.toml \\  
  --print-effective-config
```

21.2 How to read the tables

Built-in default means the value that applies with no configuration. The **production sample** is the recommended starting value in `config/server.toml.sample`. They are not always identical: the sample deliberately opts into larger production budgets in several places.

21.3 [server] and [tls]

Key	Built-in default	Meaning / production note
<code>server.app</code>	none	Absolute path to the application's <code>main.vrn</code> . Sample: <code>/srv/velran/current/main.vrn</code> .
<code>server.listen</code>	127.0.0.1:8080	Listener address. Usually loopback behind a reverse proxy; may be public for direct TLS.
<code>server.insecure_dev_cookies</code>	false	Development HTTP only. Keep false in production.
<code>server.unix_socket</code>	none	Optional absolute Unix-domain socket path; Unix only and requires <code>server.behind_proxy=true</code> .
<code>server.behind_proxy</code>	false	Enables explicit trusted reverse-proxy semantics. Cannot be combined with backend TLS.
<code>server.expected_build_id</code>	none	Optional 64-lowercase-hex deployment pin. Startup fails closed if the running server build ID differs.
<code>tls.cert_file</code>	none	Absolute path to the TLS certificate chain.
<code>tls.key_file</code>	none	Absolute path to the TLS private key.
<code>tls.handshake_timeout_ms</code>	5000	TLS handshake timeout ceiling.
<code>tls.http_redirect_listen</code>	none	Optional HTTP listener used only for HTTPS redirects.
<code>tls.public_host</code>	none	Canonical/public host for redirect and Host/Origin validation; also required in trusted HTTPS reverse-proxy mode.

Behind a reverse proxy

Behind Apache or Nginx, Velran typically does not listen on a public interface:

```
[server]
app = "/srv/velran/current/main.vrn"
listen = "127.0.0.1:8080"
insecure_dev_cookies = false

[tls]
public_host = "www.example.com"
```

The proxy terminates public TLS. Velran certificate/key settings are omitted, but `tls.public_host` is still required for trusted Host/Origin pinning. The `web.trusted_proxy_cidrs` list defines which upstreams may supply authoritative forwarding metadata. In this production mode, a non-TLS Velran listener is allowed only on loopback.

21.4 [database], [redis], and [auth]

Key	Default	Meaning
<code>database.url_file</code>	none	Secret file containing the DB URL. There is no plaintext <code>database.url</code> .
<code>database.allow_insecure</code>	false	Explicitly permits an unencrypted remote DB connection; normally do not use in production.
<code>redis.url_file</code>	none	Redis URL from a secret file.
<code>redis.allow_insecure</code>	false	Explicitly permits insecure Redis.
<code>auth.ldap_url</code>	none	LDAP/LDAPS endpoint.
<code>auth.ldap_search_base</code>	none	LDAP search base.
<code>auth.ldap_username_attribute</code>	uid	LDAP attribute used as the login name.
<code>auth.ldap_service_bind_dn_file</code>	none	Service bind DN from a secret file.
<code>auth.ldap_service_bind_password_file</code>	none	Bind password from a secret file.
<code>auth.totp_secrets_file</code>	none	TOTP secret-store file.
<code>auth.roles_file</code>	none	External role-mapping file.
<code>auth.memberships_file</code>	none	Optional tenant/user membership authority file. Absolute path; loaded as trusted local policy input.
<code>auth.local_auth_db_url_file</code>	none	Local-auth identity-store URL from a secret file.
<code>auth.require_totp</code>	false	Whether TOTP is required after a successful first factor.
<code>auth.login_max_attempts</code>	8	Maximum source+principal password failures per window.
<code>auth.login_principal_max_attempts</code>	16	Maximum password failures for one principal across sources.
<code>auth.login_source_max_attempts</code>	64	Maximum authentication failures from one source across principals.
<code>auth.mfa_max_attempts</code>	5	Maximum second-factor failures per source+principal/principal window.

<code>auth.login_window_secs</code>	300	Authentication abuse-control window in seconds.
-------------------------------------	-----	---

Do not configure LDAP and local auth as two parallel primary identity sources in one deployment. Keep identity authority unambiguous. Redis contains session, cache, and rate-limit runtime state; it is not the user source of truth.

21.5 [web]

Key	Default	Meaning
<code>web.trusted_proxy_cidrs</code>	empty list	Only these source addresses may provide authoritative forwarded client/protocol metadata.
<code>web.allow_missing_origin</code>	false	Policy for unsafe requests without Origin. The production default is fail-closed.
<code>web.cors_origins</code>	empty list	Exact-origin allowlist. Wildcards are not valid for credentialed CORS.
<code>web.cors_allow_credentials</code>	false	Allows cross-origin session cookies/credentials; must be considered together with CSRF policy.
<code>web.webhook_secrets_dir</code>	none	Absolute non-symlink directory containing verified-webhook secrets. Required when the application declares verified webhook routes.
<code>web.egress_policy_file</code>	none	Absolute outbound-integration/egress policy file. Required when application code declares outbound integration effects.

21.6 [storage] and [static_assets]

Key	Built-in default	Meaning
<code>storage.data_root</code>	none	AppFs runtime-data root. Sample: <code>/srv/velran/data</code> .
<code>storage.fs_mode</code>	<code>rwc</code>	AppFs capability mode. Grant only the operations that are required.
<code>storage.max_upload_bytes</code>	16 MiB	Maximum bytes for one upload.
<code>storage.max_image_pixels</code>	40,000,000	Decoded image pixel ceiling.
<code>static_assets.root</code>	none	Read-only static root. Sample: <code>/srv/velran/current/public</code> .
<code>static_assets.url_prefix</code>	<code>/assets/</code>	URL prefix for the static namespace.
<code>static_assets.max_asset_bytes</code>	8 MiB	Maximum size of one static asset.

<code>static_assets.max_age_secs</code>	300	Cache lifetime for non-fingerprinted assets.
<code>static_assets.immutable_max_age_secs</code>	31536000	Cache lifetime for fingerprinted immutable assets.
<code>static_assets.precompressed</code>	true	Use pre-generated Brotli/gzip variants.

Do not use the same directory for `data_root` and the static root: one is mutable application state, the other is a release-owned read-only artifact.

21.7 [lifecycle], [observability], and [logging]

Key	Default	Meaning
<code>lifecycle.health_live_path</code>	<code>/health/live</code>	Process liveness endpoint.
<code>lifecycle.health_ready_path</code>	<code>/health/ready</code>	Dependency-aware readiness endpoint.
<code>lifecycle.health_dependency_timeout_ms</code>	1000	Timeout for readiness dependency probes.
<code>lifecycle.shutdown_grace_ms</code>	30000	Graceful-shutdown drain window.
<code>observability.metrics_listen</code>	none	Prometheus listener; sample: <code>127.0.0.1:9090</code> .
<code>observability.allow_public_metrics</code>	false	Explicitly allows a public metrics listener.
<code>observability.access_log</code>	true	Emit HTTP access events.
<code>logging.server_file</code>	none	Error/system log file.
<code>logging.access_file</code>	none	Access log file.
<code>logging.audit_file</code>	none	Security/user-activity audit log.
<code>logging.stderr</code>	true	Stderr log output.

The operator creates the parent log directory. An existing log target must not be a symlink or non-regular file. In behind-proxy mode `SIGHUP` reopens logs and transactionally rebuilds domain/application hosting state; process-level settings still require restart.

21.8 [rate_limit] and [cache]

Key	Default	Meaning
<code>rate_limit.policies_file</code>	none	Route rate-limit policy file. Sample: <code>/usr/local/etc/velran/rate-limits.toml</code> .
<code>rate_limit.allow_memory</code>	false	Explicitly allows in-memory fallback. Keep false for multi-instance production.
<code>cache.max_ttl_secs</code>	3600	Maximum route TTL for public cache.
<code>cache.max_entries</code>	10000	In-memory cache entry ceiling.
<code>cache.max_bytes</code>	64 MiB	Total in-memory cache byte ceiling.
<code>cache.allow_memory</code>	false	Explicitly allows memory cache without Redis.

<code>cache.singleflight_wait_timeout_ms</code>	5000	Maximum waiter time for a coalesced public-cache fill; zero is rejected.
---	------	--

When multiple `velran-server` instances run, shared session/cache/rate-limit state must be carried by Redis. Memory fallback is appropriate only deliberately, usually in a single-instance environment.

21.9 [native]

Native execution is the production execution path; configuration controls the trusted compiler invocation and cache, not application-supplied arbitrary compiler flags.

Key	Default	Meaning
<code>native.enabled</code>	<code>true</code>	Enables native execution. Current application activation requires the native path.
<code>native.required</code>	<code>false</code>	Fail startup/activation if no native shard can be built and loaded. Production sample sets <code>true</code> .
<code>native.rustc</code>	discovered	Optional absolute trusted <code>rustc</code> path; otherwise use configured environment/PATH discovery.
<code>native.cache_dir</code>	app-local	Optional absolute native artifact cache directory. Cache hits remain integrity/hash verified.
<code>native.optimization</code>	<code>release</code>	<code>interactive</code> or <code>release</code> ; production uses <code>release</code> .
<code>native.target_cpu</code>	none	Optional validated CPU baseline, for example <code>native</code> for a host-local cache.
<code>native.cpu_features</code>	none	Optional validated feature list such as <code>+sse2,+avx2</code> ; pin only when deployment compatibility is understood.
<code>native.debug_rustc_repro</code>	<code>false</code>	Diagnostic reproduction aid; do not enable as a normal production setting.

Security rule: performance settings may change code generation, but they do not bypass verified lowering, package/capability checks, or native-cache integrity verification.

21.10 [reload]

Key	Default	Meaning
<code>reload.enabled</code>	<code>true</code>	Enables the shared source-graph watcher.
<code>reload.poll_interval_ms</code>	1000	Metadata polling interval.

<code>reload.debounce_ms</code>	250	Debounce interval before compiling a candidate generation.
<code>reload.debug_compile_errors</code>	false	Development diagnostic output while the last valid generation keeps serving. Production security policy rejects this mode.

21.11 [limits]

Request and runtime limits are defense-in-depth controls. The sample sets several values explicitly so the operator makes an auditable production-budget decision.

Key	Built-in default	Sample / meaning
<code>limits.max_header_bytes</code>	16 KiB	Sample uses the same value.
<code>limits.max_body_bytes</code>	256 KiB	Same in sample; uploads also have a separate storage limit.
<code>limits.max_connections</code>	4096	Concurrent-connection ceiling.
<code>limits.max_requests_per_connection</code>	100	Keep-alive request ceiling.
<code>limits.read_timeout_ms</code>	5000	Request read timeout.
<code>limits.request_timeout_ms</code>	15000	Total request budget.
<code>limits.write_timeout_ms</code>	5000	Response write timeout.
<code>limits.max_header_count</code>	64	Header-count ceiling.
<code>limits.max_form_fields</code>	64	Maximum number of form fields.
<code>limits.max_form_field_bytes</code>	8 KiB	Byte ceiling for one form field.
<code>limits.max_json_depth</code>	32	Maximum nested JSON depth at the bounded JSON boundary.
<code>limits.max_json_string_bytes</code>	64 KiB	Maximum decoded bytes in one JSON string.
<code>limits.max_json_array_items</code>	4096	Maximum items in one JSON array.
<code>limits.max_json_object_fields</code>	1024	Maximum fields in one JSON object.
<code>limits.max_instructions</code>	100000	Sample: 5000000. Verified execution/instruction budget; increases should be audited.
<code>limits.max_runtime_alloc_bytes</code>	32 MiB	Sample: 256 MiB. Runtime allocation ceiling.
<code>limits.session_ttl_secs</code>	28800	8 hours.
<code>limits.max_sessions</code>	100000	Session ceiling.
<code>limits.max_process_memory_bytes</code>	none	Sample: 1 GiB; process address-space limit.
<code>limits.resource_profiles_file</code>	none	Sample: <code>/usr/local/etc/velran/resource-profiles.toml</code> .

Why separate request, runtime, and process limits?

`max_body_bytes` limits network input, `max_runtime_alloc_bytes` limits runtime-accounted allocations for one request, and `max_process_memory_bytes` limits the entire process address space. None replaces the others.

21.12 [cgroup]

Cgroup controls add a Linux production defense-in-depth layer above process-level limits.

Key	Default	Meaning
<code>cgroup.dir</code>	none	Absolute path to an already prepared cgroup directory.
<code>cgroup.memory_max_bytes</code>	none	Cgroup memory ceiling; the systemd sample does not set this.
<code>cgroup.memory_swap_max_bytes</code>	none	Optional swap ceiling for delegated-cgroup deployments.
<code>cgroup.cpu_percent</code>	none	Optional CPU budget percentage for delegated-cgroup deployments.
<code>cgroup.pids_max</code>	none	Optional PID/thread ceiling for delegated-cgroup deployments.

Creating and delegating the cgroup directory is an operator task; the server is not a system-administration tool. In the repository's systemd baseline, the service manager is the cgroup authority (`MemoryMax`, `CPUQuota`, `TasksMax`), so the `[cgroup]` section is intentionally not enabled. Use Velran-side cgroup writes only with a separately delegated writable cgroup v2 directory. Do not configure both authorities at once.

21.13 Production baseline

A typical minimum configuration behind a reverse proxy:

```
[server]
app = "/srv/velran/current/main.vrn"
listen = "127.0.0.1:8080"
insecure_dev_cookies = false

[tls]
public_host = "www.example.com"

[database]
url_file = "/run/secrets/velran/database-url"

[redis]
url_file = "/run/secrets/velran/redis-url"

[web]
trusted_proxy_cidrs = ["127.0.0.1/32", "::1/128"]
allow_missing_origin = false

[storage]
data_root = "/srv/velran/data"
```

```
[logging]
server_file = "/var/log/velran/server.log"
access_file = "/var/log/velran/access.log"
audit_file = "/var/log/velran/audit.log"
stderr = true

[limits]
resource_profiles_file = "/usr/local/etc/velran/resource-profiles.toml"
```

This is only a baseline. Auth, static assets, metrics, rate limiting, TLS, and cgroup settings must match the deployment topology.

21.14 Automatic source reload

The global `[reload]` section defines `enabled`, `poll_interval_ms`, `debounce_ms`, and the development-only `debug_compile_errors`. Each multi-domain entry may override them in `[domains.reload]`. The default policy enables metadata polling. A single shared supervisor watches the compiler-reported source graph with `mtime + size`, including the configured logical entriypoint path for atomic symlink deployments. Failed candidates leave the old generation active; successful commits atomically replace only the affected domain and invalidate its public-cache generations. Use `-no-source-reload` for a process-wide maintenance override.

21.15 Change workflow

For production configuration changes, this book uses one recommended order:

1. modify the version-controlled configuration template;
2. change secrets in separate files, not in TOML;
3. run `velran-server -check-config`;
4. inspect the secret-free `-print-effective-config` summary;
5. perform a controlled restart;
6. verify live/ready endpoints, logs, metrics, and audit output.

`SIGHUP` is a bounded hosting reload in behind-proxy mode, not a general process reconfiguration mechanism. Application source changes normally need neither `SIGHUP` nor restart: the `[reload]` supervisor watches the entriypoint and transitive module graph and transactionally swaps a successfully compiled domain generation.

21.16 Practice: review configuration by trust domain

Exercise mode: operational scenario. Use the operational-scenario method defined in the preface.

Group configuration into listener/TLS, storage, authentication/session, resource limits, egress, observability, and native build/runtime concerns. Review each group with the owner who understands its failure mode rather than treating the file as one flat list of keys.

21.17 Common mistake: copying a large configuration and changing only obvious values

A copied setting may carry assumptions about proxies, paths, limits, or credentials that do not match the new environment. Start from the documented minimum and add only settings whose operational meaning is understood.

21.18 Operator checkpoint: Secure Notes

Produce the minimal production configuration for Secure Notes and annotate which values are environment-specific. Keep secrets in referenced files and verify the final configuration with the server's check mode before deployment.

Chapter 22

Developer reference and where to go next

Learning path: Complete path / reference map. Use this to find the authoritative source for a language, security, or operations question.

22.1 Canonical documentation map

This book is the long-form development guide. The current high-level security status is `SECURITY-STATUS.md`; detailed canonical topic documentation lives in `docs/`. Hungarian operator/developer companions remain under `docs/hu/` where a translated topic exists.

Topic	Canonical project documentation
Current security architecture	<code>SECURITY-STATUS.md</code> , <code>docs/99-security-architecture-and-status.md</code>
Secure-by-construction foundation	<code>docs/57-secure-by-construction-foundation.md</code>
Domain types and bounded collections	<code>docs/62-domain-types-and-bounded-collections.md</code>
Nominal types and refinement	<code>docs/63-nominal-domain-types.md</code> , <code>docs/64-domain-value-refinement.md</code>
Public projections and secret flows	<code>docs/65-explicit-public-projections.md</code> , <code>docs/66-secret-consumer-contracts.md</code>
Passwords, tokens and sessions	<code>docs/67-typed-credentials-and-passwords.md</code> through <code>docs/74-platform-owned-browser-sessions.md</code>
Permissions, MFA and critical operations	<code>docs/75-function-level-permissions.md</code> through <code>docs/77-critical-operation-contracts.md</code>
Resource profiles and bounded request input	<code>docs/78-route-budget-profiles.md</code> , <code>docs/79-bounded-request-collections.md</code>
Public error boundary	<code>docs/80-public-error-boundaries.md</code>
Tenant authority and isolation	<code>docs/81-tenant-membership-authority.md</code> , <code>docs/82-compiler-enforced-tenant-isolation.md</code>
Idempotent critical operations	<code>docs/83-idempotent-critical-operations.md</code>
Verified webhooks	<code>docs/81-verified-webhook-integrity.md</code>
Safe upload state machine	<code>docs/84-safe-file-upload-state-machine.md</code>

Effect/capability model	docs/85-effect-capability-foundation.md
Typed outbound integrations	docs/86-typed-outbound-integration-calls.md
Production security policy	docs/87-production-configuration-policy.md
Typed HTTP metadata	docs/88-typed-http-metadata.md
Crypto key purpose/lifecycle	docs/89-crypto-key-purpose-lifecycle.md
Generated CSP/security headers	docs/90-generated-security-headers.md
Supply-chain capabilities/provenance	docs/92-supply-chain-capability-provenance.md
Authentication abuse protection	docs/93-authentication-abuse-protection.md
Closed sum types/exhaustive match	docs/94-sum-types-and-exhaustive-match.md
Transaction outcome semantics	docs/95-transaction-outcome-semantics.md
Authenticated encryption/key lifecycle	docs/ 96-authenticated-encryption-key-lifecycle.md
Advanced resource safety	docs/97-advanced-resource-safety.md
Typed security events	docs/79-typed-security-events.md
Bounded JSON boundary	docs/100-bounded-json-boundary.md
Security monitoring/redaction	docs/98-security-monitoring-and-redaction.md
Dependency security	docs/19-dependency-security.md
Modules/namespaces	docs/21-modules-and-namespaces.md
Math/timing, string/list/dict/regex	docs/44-math-and-timing.md, docs/46-string-builtins.md through docs/49-regular-expressions.md
Optimistic locking	docs/24-optimistic-locking.md
IPv6-ready egress	docs/32-ipv6-egress.md
Debian package/install	docs/36-debian-package.md
Reverse proxy/multidomain/reload	docs/37-multi-domain-hosting.md through docs/39-automatic-source-reload.md
Maintainability/architecture gates	docs/50-maintainability-and-clean-code.md

22.2 Daily secure development checklist

1. Keep authority explicit: route policy, DB capability, tenant, permission/MFA and named outbound integration.
2. Refine untrusted input to nominal/domain types; do not confuse validation with authorization.
3. Use object/mutation authorization before protected writes.
4. Use critical contracts for operations that need permission, MFA, transaction, audit or idempotency.
5. For retry-sensitive transactions, capture and exhaustively handle `Committed`, `RolledBack`, and `CommitUnknown`.
6. Never reintroduce raw SQL, raw HTML, arbitrary redirect/header/filename or arbitrary outbound URL shortcuts.
7. Keep file uploads staged/private until inspection and explicit publish.
8. Keep secrets behind typed secret/credential/key boundaries and use `redact(...)` for safe monitoring representation.
9. Keep DB rows, collections, files, outbound bodies and request time bounded.

10. Before platform release run locked Cargo metadata/check/tests, `./verify.sh`, supply-chain verification and release-manifest verification.

22.3 How to use the documentation set

Use the documentation layer that matches the question:

Document class	Best use	Authority
This book	learning path and end-to-end development workflow	explanatory, aligned with canonical docs
<code>docs/*.md</code>	feature contracts, invariants and focused examples	canonical topic documentation
<code>docs/hu/*.md</code>	Hungarian companions for day-to-day development/operations	translated companion where present
<code>SECURITY-STATUS.md</code>	current high-level security architecture/status	current architecture summary
<code>RUST_FIRST_LANGUAGE_POLICY.md</code>	language/backend direction and native execution boundary	current language policy
<code>history/development-notes/</code>	historical iterations, experiments and superseded design context	non-normative history

If sources disagree, prefer the current canonical feature document and verified compiler/runtime behavior, then update the stale text. Historical notes explain design history; they are not current syntax or deployment instructions.

22.4 Task-oriented reading paths

For common work, the following reading order is faster than scanning the repository numerically:

- **Build a CRUD application:** Chapters 3–7, then the forms, optimistic-locking and domain-type documents.
- **Build a JSON/API service:** Chapter 9, bounded JSON, typed HTTP metadata and public error boundary.
- **Add authentication-sensitive mutations:** Chapter 7, function permissions, MFA elevation, critical-operation contracts and idempotency.
- **Accept files/images:** Chapter 8 plus the safe upload state-machine document.
- **Call external services:** Chapter 13 plus secure routing/egress and typed outbound integration documents.
- **Prepare production:** Chapters 15–23, production configuration policy, supply-chain provenance, recovery and the production checklist.
- **Investigate a security invariant:** start with `SECURITY-STATUS.md`, then follow the focused document named for that invariant.

22.5 Executable companion examples

The book intentionally contains both complete programs and focused fragments. For code you want to copy, modify, or use as a regression fixture, start with a repository example that

participates in project verification:

Task	Verified companion	What to study
Small modular application	<code>examples/ starter-project/</code>	modules, typed pages/actions, CSRF and typed route helpers
CRUD and typed SQL	<code>examples/crud/app.vrn</code>	<code>Vec<T></code> , <code>Option<T></code> , transactions, validation and object authorization
Authentication policy	<code>examples/auth/app.vrn</code>	authenticated route policy and session-aware application flow
Files and uploads	<code>examples/upload/app.vrn</code>	typed upload boundary and safe file handling
JSON/API	<code>examples/json-api/app.vrn</code>	typed JSON request/response and API route behavior
Components/layouts	<code>examples/components/app.vrn</code>	components, layouts and typed HTML composition
Wiki/CMS	<code>examples/wiki/main.vrn</code>	slugged content, Markdown rendering, layouts and mutation flow
SQL-backed Markdown	<code>examples/ markdown-sql-cache/</code>	typed ID lookup, server-side safe Markdown rendering and optional public response cache
Optimistic locking	<code>examples/ optimistic-locking/main.vrn</code>	versioned updates and conflict handling
Outbound integration	<code>examples/ outbound-native/app.vrn</code>	named, typed outbound calls and egress boundary
Security events	<code>examples/ typed-security-events/</code>	typed security-event emission and monitoring boundary

A short listing in the book may deliberately omit unrelated declarations so the concept remains visible. Such a fragment is explanatory, not a promise that the fragment alone forms a complete program. The companion paths above are the preferred copy/paste starting points. Verification-only rejection cases belong under `tests/fixtures/`, not in the learner example catalog; `tests/manifests/` contains corpus metadata. The `tools/check-book-velran-examples.sh` guard protects the book against known legacy syntax, missing route policy, missing example paths and English/Hungaria Velran-listing drift.

22.6 Keeping examples trustworthy

Documentation examples follow production-source rules: current Rust-like syntax, explicit route policy, typed/bounded input, compiler-owned SQL, no raw HTML/redirect/egress shortcuts, and native-only execution. Prefer a matching verified `examples/` fixture when available. A documentation change is complete only after language-corpus review and EN/HU structural parity checks.

22.7 Practice: choose the right authority

Exercise mode: operational scenario. Use the operational-scenario method defined in the preface.

For a language question, identify whether the answer belongs in the compiler-verified language surface, the web framework contract, deployment documentation, or historical notes. Use the most authoritative current source first and treat translated or historical material according to its stated role.

22.8 Common mistake: resolving documentation conflicts by guessing

When two documents disagree, do not silently combine them. Prefer the canonical current specification or verified behavior, then update the stale document so the conflict does not remain for the next reader.

22.9 Reader checkpoint: Secure Notes

Pick one design decision from Secure Notes and locate the current authoritative documentation that supports it. This is a practical test that the documentation map is usable, not merely complete.

Appendix A

Appendix and production checklist

A.1 Current verification note

The current Velran source tree has completed the repository-level Verified Development Milestone, including formatting, the workspace test suite, `./verify.sh`, and local test serving. Before a production release, repeat the lock/provenance refresh and full repository verification on the exact release tree and collect the target-environment deployment, recovery, and operational evidence. Repository verification is necessary, but it is not a substitute for production-environment release evidence.

Learning path: Fast path: final release gate. Use the checklist to prove release readiness rather than relying on memory.

This chapter introduces no new language elements. It is a compact operations aid assembled from the preceding chapters: directory layout, reverse-proxy examples, a systemd unit, logrotate configuration, a pre-release gate, and a troubleshooting map. Treat the examples as starting points; adapt domains, certificates, permissions, and resource limits to your environment.

A.2 Recommended production directory layout

Velran is locally installed software, so this book consistently uses the `/usr/local` hierarchy for programs and configuration. Application state and logs live separately.

```
/usr/local/bin/  
  velran-server  
  velran-cli  
  
/usr/local/etc/velran/  
  server.toml  
  rate-limits.toml  
  resource-profiles.toml  
  
/run/secrets/velran/  
  database-url  
  redis-url  
  local-auth-database-url  
  
/srv/velran/  
  current/                # current application release
```

```

releases/
  2026-09-04.1/
data/                                # AppFs / durable application data

/var/log/velran/
  server.log
  access.log
  audit.log

```

Keep release contents read-only where possible. The server should have write permission only where durable state or logs actually need to be written. Secret files must not live inside the application release directory or source control.

A.3 Complete minimal Apache reverse-proxy example

This example assumes one Apache reverse proxy. Apache owns public TLS; Velran listens only on loopback.

The corresponding minimum Velran configuration is:

```

[server]
listen = "127.0.0.1:8080"
insecure_dev_cookies = false

[tls]
public_host = "www.example.com"

[web]
trusted_proxy_cidrs = ["127.0.0.1/32", "::1/128"]

```

In reverse-proxy production mode the certificate and key stay on the proxy, but trusted `public_host` is still required for Host/Origin pinning. The proxy must produce authoritative `X-Forwarded-Proto: https` metadata.

```

<VirtualHost *:80>
  ServerName www.example.com
  Redirect permanent / https://www.example.com/
</VirtualHost>

<VirtualHost *:443>
  ServerName www.example.com

  SSLEngine on
  SSLCertificateFile \
    /etc/letsencrypt/live/www.example.com/fullchain.pem
  SSLCertificateKeyFile \
    /etc/letsencrypt/live/www.example.com/privkey.pem

  ProxyPreserveHost On
  ProxyPass          / http://127.0.0.1:8080/
  ProxyPassReverse   / http://127.0.0.1:8080/

  RequestHeader set X-Forwarded-Proto "https"
</VirtualHost>

```

Typical Apache modules are `proxy`, `proxy_http`, `headers`, and `ssl`. The Velran trusted-proxy list should contain only the real proxy. For a loopback proxy this normally means `127.0.0.1/32` and, if needed, `::1/128`. An Internet-sized trusted-proxy range is a configuration error.

A.4 Complete minimal Nginx reverse-proxy example

```
server {
    listen 80;
    server_name www.example.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name www.example.com;

    ssl_certificate
        /etc/letsencrypt/live/www.example.com/fullchain.pem;
    ssl_certificate_key
        /etc/letsencrypt/live/www.example.com/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto https;
        proxy_set_header X-Forwarded-For $remote_addr;
    }
}
```

Here Nginx directly sees the client, so it creates the forwarded client address itself. Do not pass through arbitrary Internet-supplied `X-Forwarded-*` values as authority. With multiple proxies, design the trust chain explicitly.

A.5 systemd service example

The repository contains the complete example at `examples/systemd/velran.service`. Its essential shape is:

```
[Unit]
Description=Velran application
Wants=network-online.target
After=network-online.target

[Service]
Type=simple
User=velran
Group=velran
WorkingDirectory=/srv/velran/current
ExecStartPre=/usr/local/bin/velran-server \
    --config /usr/local/etc/velran/server.toml \
    --check-config
ExecStart=/usr/local/bin/velran-server \
    --config /usr/local/etc/velran/server.toml
```

```

ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure
RestartSec=2s
TimeoutStopSec=45s

MemoryMax=1G
MemorySwapMax=0
CPUQuota=200%
TasksMax=256
LimitNOFILE=8192
# For direct TLS on 80/443 add only:
# AmbientCapabilities=CAP_NET_BIND_SERVICE
# CapabilityBoundingSet=CAP_NET_BIND_SERVICE
NoNewPrivileges=yes
PrivateTmp=yes
PrivateDevices=yes
ProtectSystem=strict
ProtectHome=yes
ProtectKernelTunables=yes
ProtectKernelModules=yes
ProtectControlGroups=yes
RestrictSUIDSGID=yes
LockPersonality=yes
ReadWritePaths=/srv/velran/data /var/log/velran

[Install]
WantedBy=multi-user.target

```

Systemd hard limits and Velran request/runtime resource policy are complementary defense layers. In the systemd baseline, systemd itself is the process cgroup authority, so do not also enable Velran's `[cgroup]` writer mode. For direct binds to 80/443 grant only `CAP_NET_BIND_SERVICE`; behind a reverse proxy on a high loopback port, omit it. `TimeoutStopSec` should exceed the configured graceful-shutdown window. In behind-proxy mode `SIGHUP` can transactionally reload domain/application hosting state, while process-level configuration changes still require a controlled restart; application source alone is normally activated by the automatic source-reload supervisor.

A.6 logrotate example

```

/var/log/velran/server.log /var/log/velran/access.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 0640 velran velran
    sharedscripts
    postrotate
        systemctl kill -s HUP --kill-who=main \
            velran.service >/dev/null 2>&1 || true
    endscript
}

/var/log/velran/audit.log {

```

```

daily
rotate 90
compress
delaycompress
missingok
notifempty
create 0640 velran velran
sharedscripts
postrotate
    systemctl kill -s HUP --kill-who=main \
        velran.service >/dev/null 2>&1 || true
endscript
}

```

Retention is a legal and risk decision. Audit logs usually deserve longer retention and central forwarding; the numbers above are examples, not universal requirements.

A.7 Pre-release production gate

The following list is intentionally short and executable. The earlier chapters provide the rationale.

1. **Source and build:** identify the release commit; pin `Cargo.lock`; build the platform with `cargo build -locked -release -p velran-server -p velran-cli`; pass `velran-cli check` on application source.
2. **Migration:** run `migrate status` and `migrate verify` before production; apply approved schema changes explicitly; verify again afterward.
3. **Configuration:** pass `config-check` on the real production `server.toml`; review the redacted effective configuration.
4. **Secrets:** keep DB/Redis/auth secret files out of the repository, with narrow filesystem permissions, and out of logs and release records.
5. **HTTP edge:** verify domain, TLS, and Host handling; the trusted-proxy list contains only real proxies.
6. **Auth:** production cookie policy is active; local-auth/LDAP configuration is verified; admin/MFA recovery is documented.
7. **CSRF and CORS:** state-changing browser requests are CSRF protected; CORS allowlists are explicit; credentialed CORS has no wildcard origin.
8. **Database:** the application DB user is least privilege; pool/budget matches backend capacity; important query indexes are understood.
9. **Storage/upload:** AppFs confinement is correct; byte/pixel limits are active; the application cannot write release content.
10. **Redis/cache/rate limit:** multi-instance shared security/runtime state lives in Redis; Redis outage behavior is known and tested.
11. **Egress:** only named outbound targets are allowed; host/CIDR/port/TLS/timeout/size policy is explicit; secrets do not appear in URLs or logs.
12. **Resources:** request/runtime limits and process hard limits are consistent; cgroup authority is either systemd or a separately delegated Velran cgroup, not both.
13. **Observability:** server/access/audit logs are writable; request IDs work; metrics and readiness are reachable from the correct management network; minimum alerts are defined.
14. **Backup/recovery:** recent verified DB + AppFs + identity backups exist; a recovery manifest exists; restore drills are real rather than theoretical.

15. **Deploy/rollback:** release artifact and checksum are known; the previous release can be restored; schema changes have an explicit app-rollback versus full-restore decision.
16. **Post-start evidence:** liveness, readiness, and at least one business smoke request succeed; logs show no startup fallback or unexpected policy failure.

Platform-release security evidence

When releasing the Velran platform itself, do not call the tree release-ready unless the exact source tree passes:

```
cargo fmt --all -- --check
cargo metadata --locked --format-version 1 >/dev/null
cargo check --workspace --locked
cargo test --workspace --locked
./verify.sh
```

The supply-chain envelope must match both `Cargo.lock` and `SUPPLY-CHAIN-CAPABILITIES.txt`. Release packaging must not regenerate or bless a Cargo-stale lockfile. Verify the release manifest after all documentation/book changes as well.

A.8 Quick troubleshooting map

Symptom	First place to look	Next question
Service does not start	config-check, server log	Config/secret/path/policy or dependency readiness?
502/504 at proxy	proxy log, Velran listener/readiness	Is the process running, loopback port correct, readiness green?
400/415	access log + route body contract	Malformed input, wrong schema, or wrong Content-Type?
401	auth/session audit event	No session, expired, invalidated, or MFA missing?
403	route/object authorization, CSRF/CORS	Who is the actor, and which policy denied access?
404/405/406	route map + request method/Accept	Does the route exist, and is the method/representation acceptable?
409	optimistic-lock/business state	Did the record change, or did a Changed query modify zero rows?
413/431	request hard limits	Did body/upload or headers exceed configured limits?
421/426	Host/proxy/TLS policy	Are public host, trusted proxy, and effective HTTPS correct?
422	named-form rerender	Which field failed typed decode or validation?
429	rate-limit metrics/audit	Which limiter and identity/IP bucket was exhausted?

Symptom	First place to look	Next question
500	server log by request ID	Was there a real internal invariant failure?
503	readiness + server log	Did DB/Redis/auth/storage or a request resource limit make service temporarily unavailable?
DB error	DB readiness + query context	Connection, timeout, schema drift, lock, or constraint?
File/media error	AppFs path/permissions/limits	Is the storage root writable, with confinement and byte/pixel limits correct?
External API error	egress policy + adapter log	DNS, CIDR, port, TLS, timeout, or response-size denial?
Logs disappear after rotation	log fallback metric, service log	Did SIGHUP arrive, and are new-file permissions correct?

A.9 What to carry into production

Five rules summarize the operational model:

- expose HTTP only after typed data, explicit policy, and bounded external input are defined;
- keep stable policy in trusted configuration and secrets in secret files;
- make schema mutation, release activation, and rollback explicit operator decisions;
- treat proxy, Redis, DB, AppFs, and outbound integrations according to their declared authority boundaries;
- verify before release, then prove health, observability, backup, and restore with evidence.

A.10 Velran web developer competency checklist

This checklist answers a different question from the production gate: *can you build and operate a small Velran web application without copying a recipe blindly?* A checked item should mean that you can perform the task, explain the relevant boundary, and diagnose the most likely failure mode.

Core application development

- [] I can read an existing Velran project and identify pages, actions, queries, routes, modules, migrations, and configuration boundaries.
- [] I can add a page or action handler and connect it to a route with an explicit access policy.
- [] I can choose between query, form, JSON, and multipart input and keep externally controlled strings and collections bounded.
- [] I can model application data with Rust-like structs, enums, `Option<T>`, `Vec<T>`, and `Result<T,E>` without falling back to legacy Velran spellings.
- [] I can add a database migration, write a typed query, and keep database constraints aligned with application-level validation.

- [] I can implement create, read, update, and delete flows with transactions where multiple writes form one logical operation.
- [] I can recognize when optimistic locking or another concurrency rule is required instead of silently overwriting newer state.
- [] I can split a growing application into modules without weakening route, data, or capability boundaries.
- [] I can expose a typed JSON endpoint and distinguish browser-oriented form flows from API-oriented request/response flows.
- [] I can render database-backed Markdown on the server, keep Markdown as the source of truth, and apply public response caching only when the route is truly non-personalized and its freshness contract permits it.
- [] I can use the canonical repository examples and Secure Notes checkpoints to verify my design instead of inventing unsupported syntax from memory.

Security boundaries

- [] I understand that authentication proves who the actor is, while authorization decides whether that actor may perform this operation on this object.
- [] I can choose deliberately between **public**, authenticated, MFA, role, permission, webhook, and critical-operation route policies.
- [] I can enforce ownership or another object-level authorization rule before reading or changing a protected object.
- [] I can push principal/tenant filtering into the database query when a collection must never contain another actor's records.
- [] I can explain why unbounded external input, unrestricted upload/media processing, and unrestricted outbound networking are security boundaries rather than convenience features.
- [] I can keep CSRF, CORS, trusted-proxy, Host, cookie, and credential handling separate in my mental model instead of treating them as one generic “web security” switch.
- [] I can interpret the most common compiler/security diagnostics and fix the violated invariant rather than suppressing the check.
- [] I can review a change and state which capability, filesystem, database, network, or identity boundary it expands.

Build, verification, and production

- [] I can run the normal development loop: edit, **velran-cli check**, targeted tests, build, and the repository verification gates appropriate to the change.
- [] I understand the native path from verified source through executable IR and generated Rust to **rustc**, immutable generation, and atomic activation at the level needed to diagnose a failed build or activation.
- [] I can distinguish application reloadable state from configuration or infrastructure changes that require a process restart.
- [] I can prepare and verify a migration instead of coupling schema mutation to server startup.
- [] I can read server, access, and audit logs using a request ID and use readiness/metrics to separate application failure from dependency failure.
- [] I can deploy with an explicit release artifact, checksum, configuration review, smoke test, and rollback decision.
- [] I can explain what belongs in backup, restore a consistent DB/AppFs/identity state in isolation, and treat a restore drill as evidence rather than documentation alone.
- [] I can run **./verify.sh**, understand which gate failed, distinguish learner examples from

rejection fixtures/manifests, and avoid “fixes” that merely weaken the guard.

How to use the checklist

Do not aim to memorize every command. The target is independent reasoning. If an item is uncertain, return to the corresponding Secure Notes checkpoint, change one thing deliberately, run the relevant verifier, and explain why the resulting behavior is safe. When every item above is demonstrable on a small application, you have the practical baseline expected from a Velran web developer; later chapters and reference material then deepen rather than replace that baseline.

A.11 Practice: run the checklist as evidence collection

Exercise mode: operational scenario. Use the operational-scenario method defined in the preface.

Do not tick checklist items from memory. Attach a command result, configuration review, test result, restore rehearsal, or other concrete evidence to every item that can affect release safety.

A.12 Common mistake: treating a checklist as a substitute for engineering judgment

A checklist catches known classes of omission; it cannot prove that an unfamiliar architecture is safe. Stop the release when an item is ambiguous and resolve the design question instead of interpreting the checkbox optimistically.

A.13 Final Secure Notes checkpoint

Use this appendix to perform a release-readiness review of the final checkpoint in `examples/secure-notes/checkpoints/23-release-readiness/main.vrn`. Trace evidence from source and lockfile through its baseline migration, compiler verification, configuration, native activation, backup, and restore procedure. Any item that cannot be evidenced becomes the next engineering task before production activation.

Closing note

The purpose of this book is to teach the language features as parts of a working web application rather than as isolated syntax. Detailed feature-level reference material remains available in the project's `docs/` directory; this book provides the day-to-day development path.